

Deep Learning

CS-878

Week-06

Image Classification



[This image](#) by [Nikita](#) is
licensed under [CC-BY2.0](#)

(assume given a set of possible labels)
{dog, cat, truck, plane, ...}

—————→ cat

Computer Vision Tasks

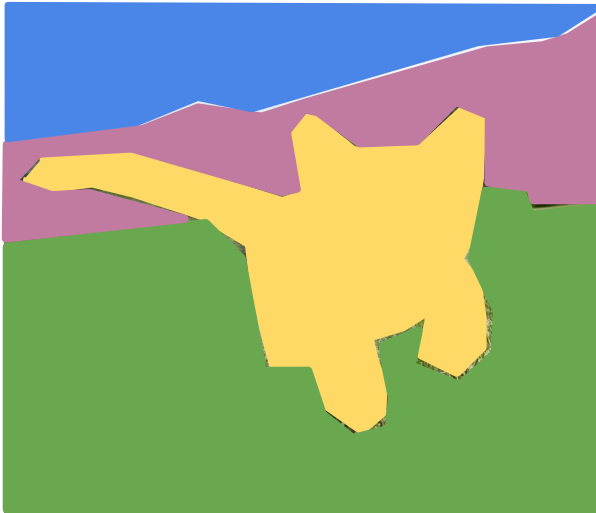
Classification



CAT

No spatial extent

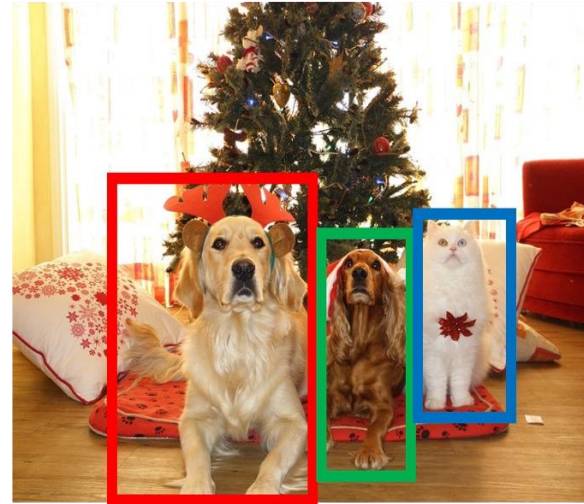
Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation



DOG, DOG, CAT

Multiple Object

[This image is CC0 public domain](#)

Semantic Segmentation

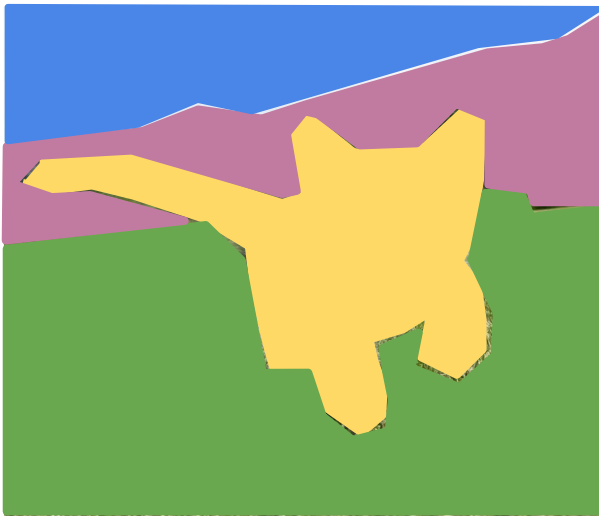
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation

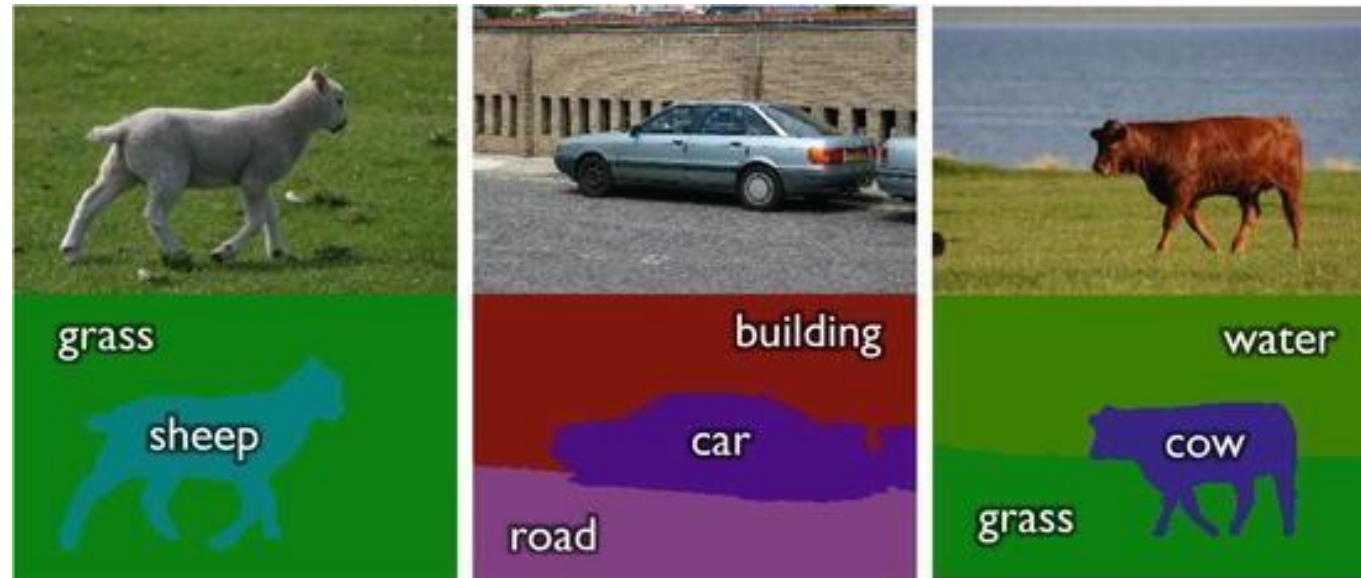


DOG, DOG, CAT

Semantic Segmentation

Assign a label to **each pixel** in an image:

- Pixel-level image annotation/analysis (vs. object-level analysis)



Common datasets: PASCAL VOC (2012) and MSCOCO

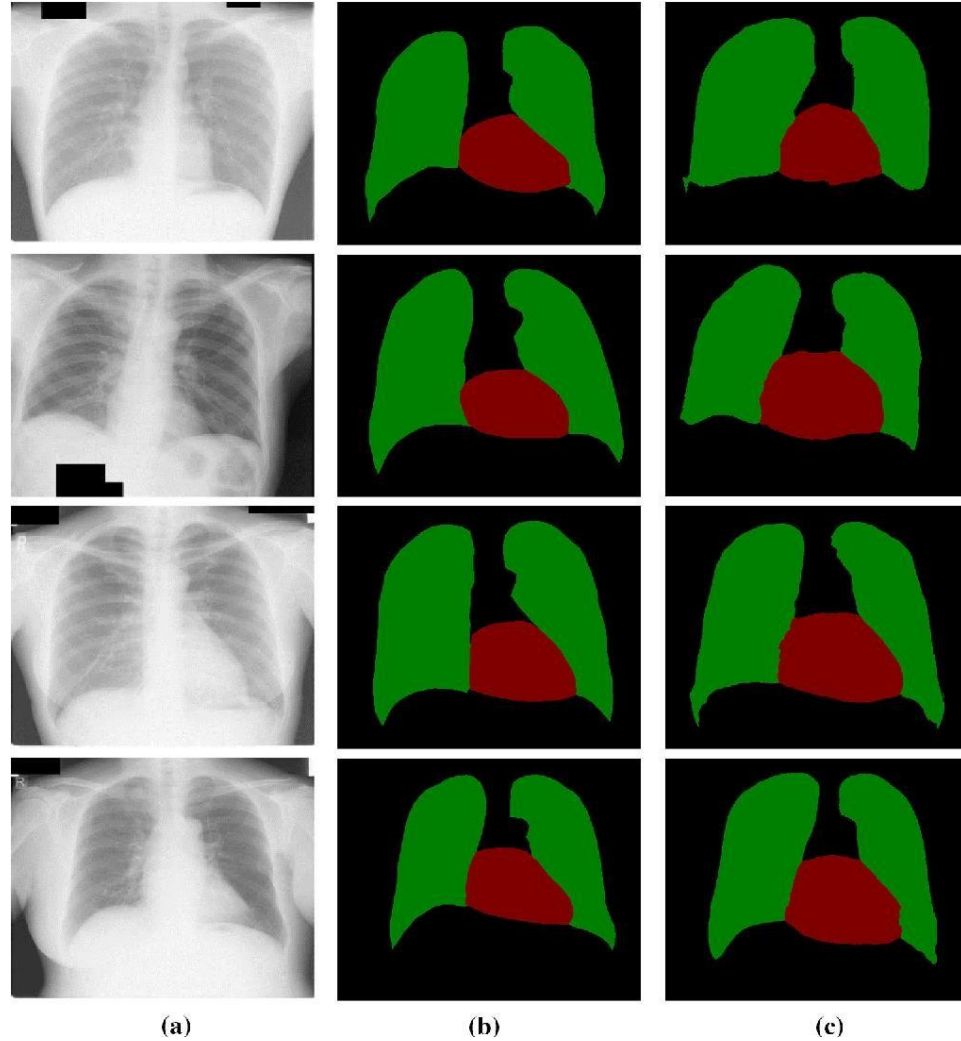
Semantic Segmentation

- A key part of Scene Understanding
- Applications
 - Autonomous navigation
 - Assisting the partially sighted
 - Medical diagnosis
 - Image editing



Semantic Segmentation

- Applications
 - Medical diagnosis



Segmentation Tasks

A sample image from the PASCAL VOC2011 dataset



Original (input) Image



Semantic (class) Segmentation

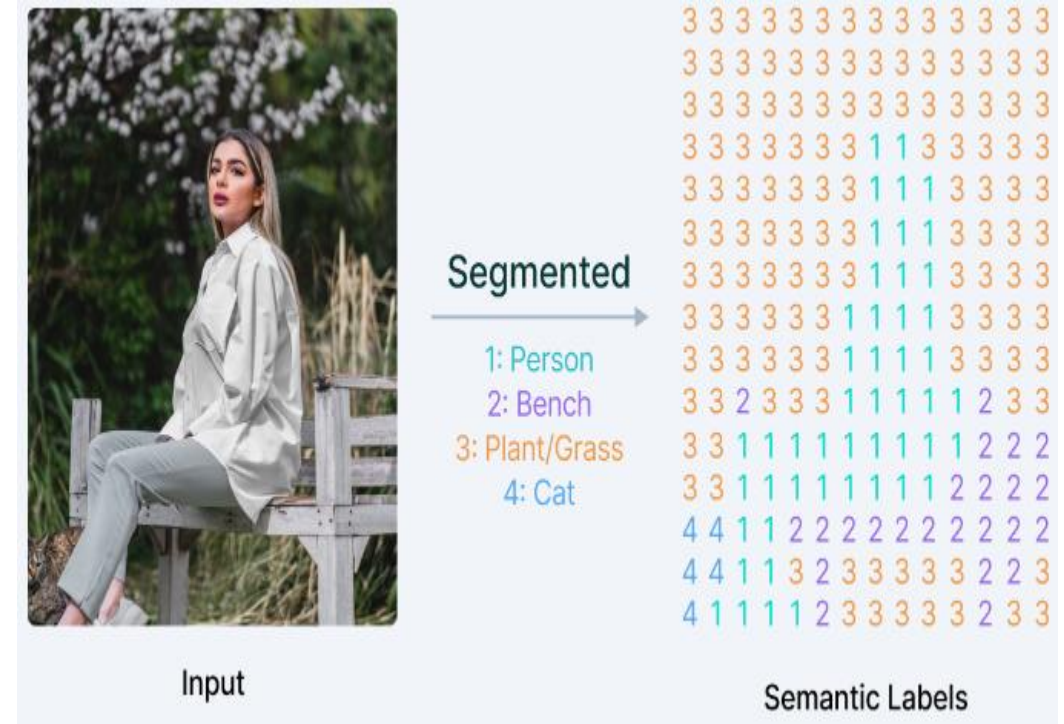
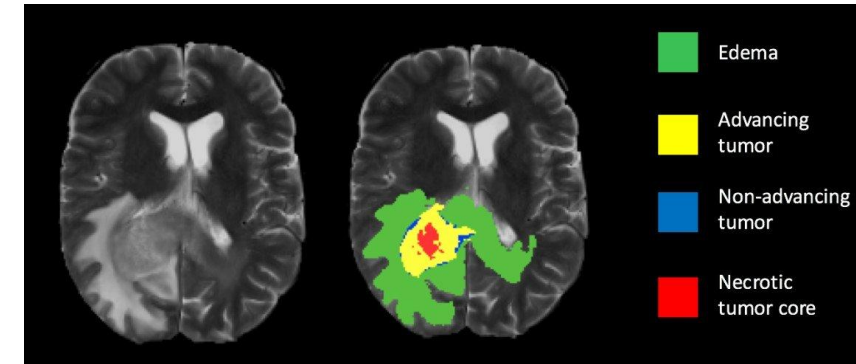
Semantic Segmentation

With semantic segmentation, we are interested in the ***precise*** location of the object.

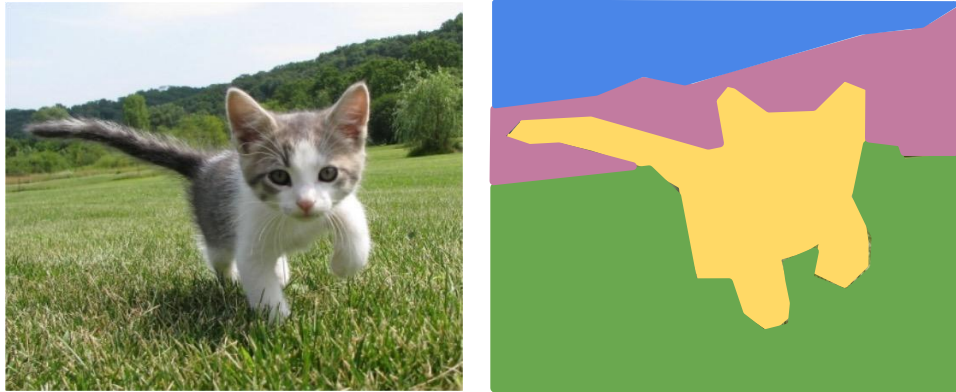
For example, for a 16x16 image, we would get 256 outputs arranged in 16x16 matrix.

These outputs would tell that ***which pixel belongs to which particular class.***

For just one object in an image, a pixel could either belong to the ***object*** or the ***background.***

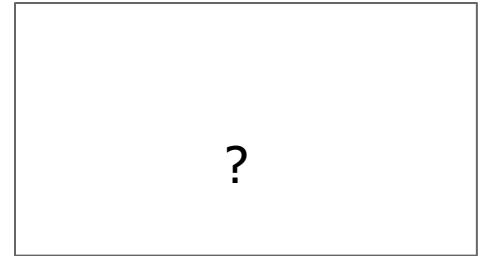
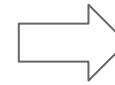


Semantic Segmentation: The Problem



GRASS, CAT, TREE,
SKY, ...

Paired training data: for each training image, each pixel is labeled with a semantic category.



At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Sliding Window

Full image



Semantic Segmentation Idea: Sliding Window

Full image

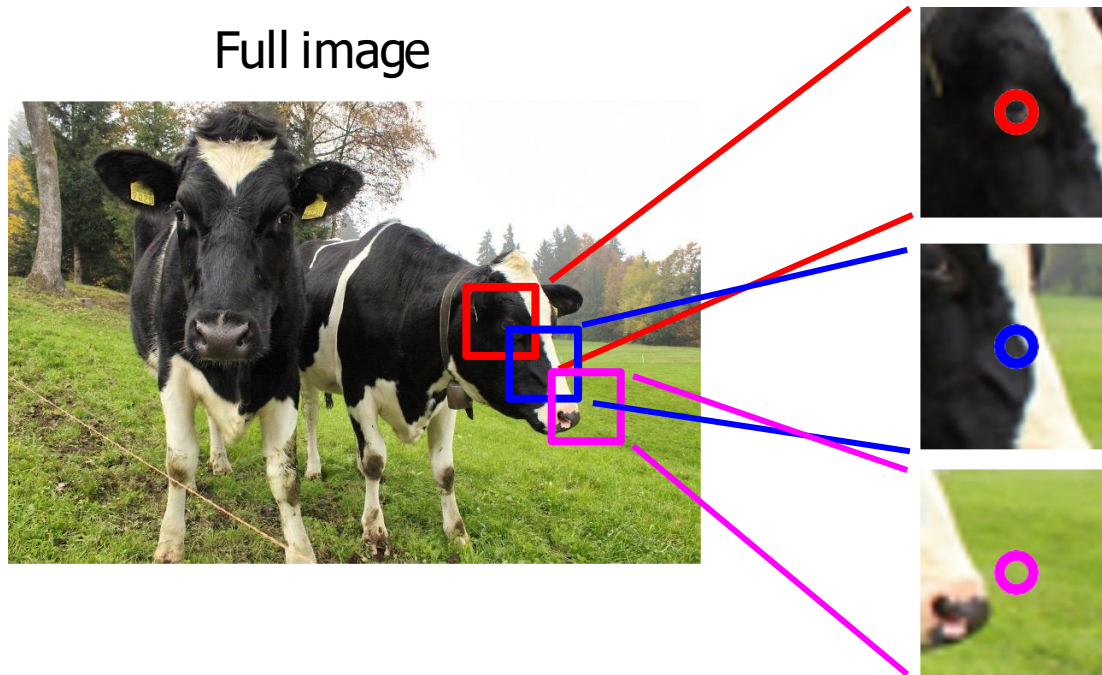


?

Impossible to classify without context

Q: how do we include context?

Semantic Segmentation Idea: Sliding Window



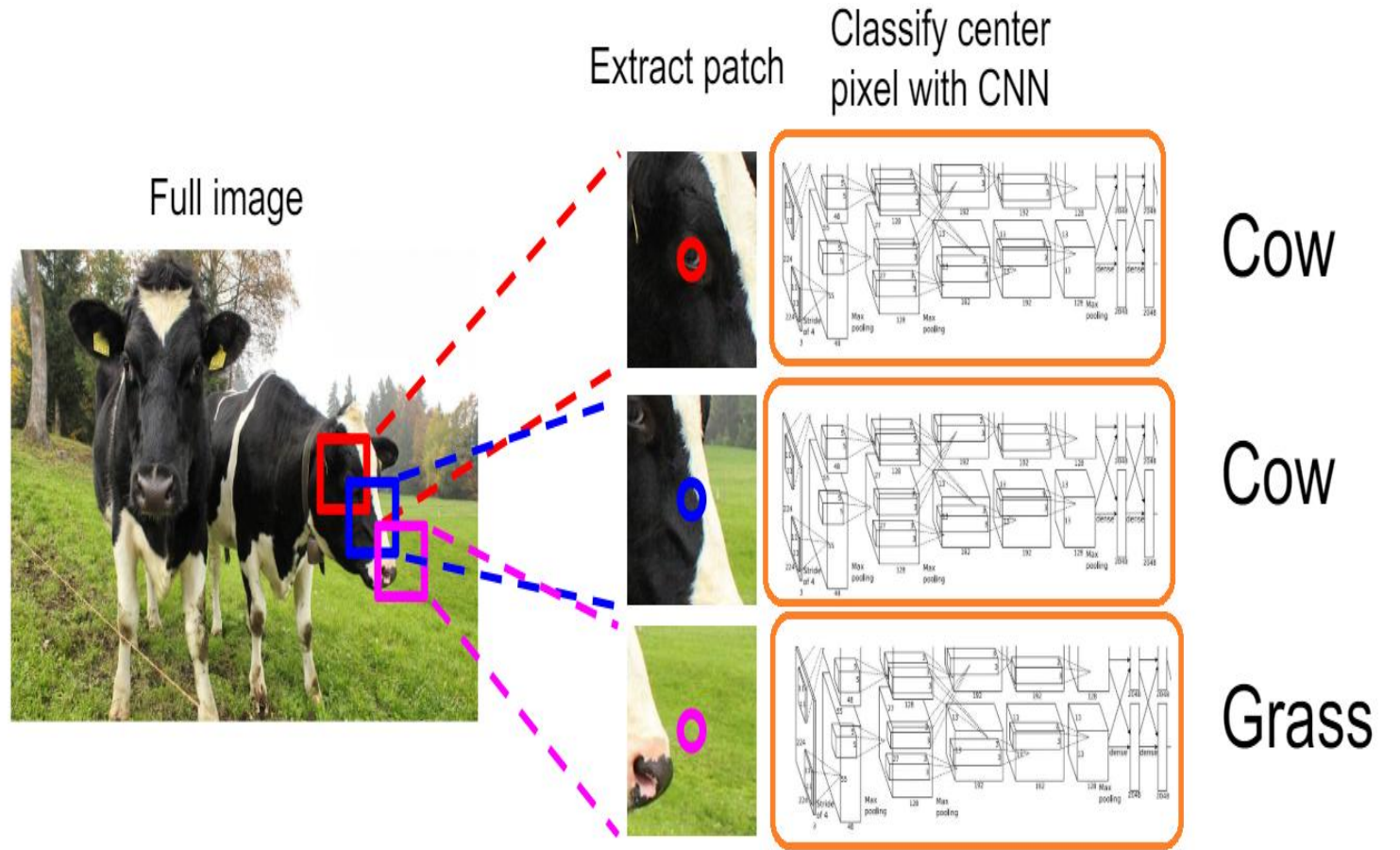
Q: how do we model this?

Semantic Segmentation

One straight-forward strategy is to **modify** our classification network

Run the classification network **for each pixel** in the image using **sliding window**

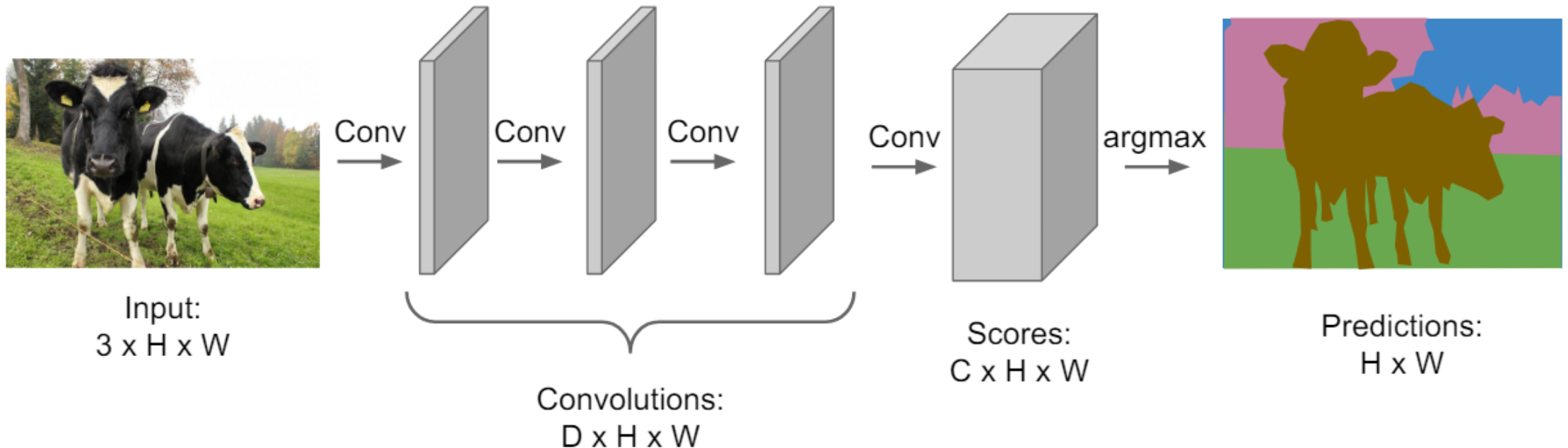
Problems with **training and inference**



Semantic Segmentation

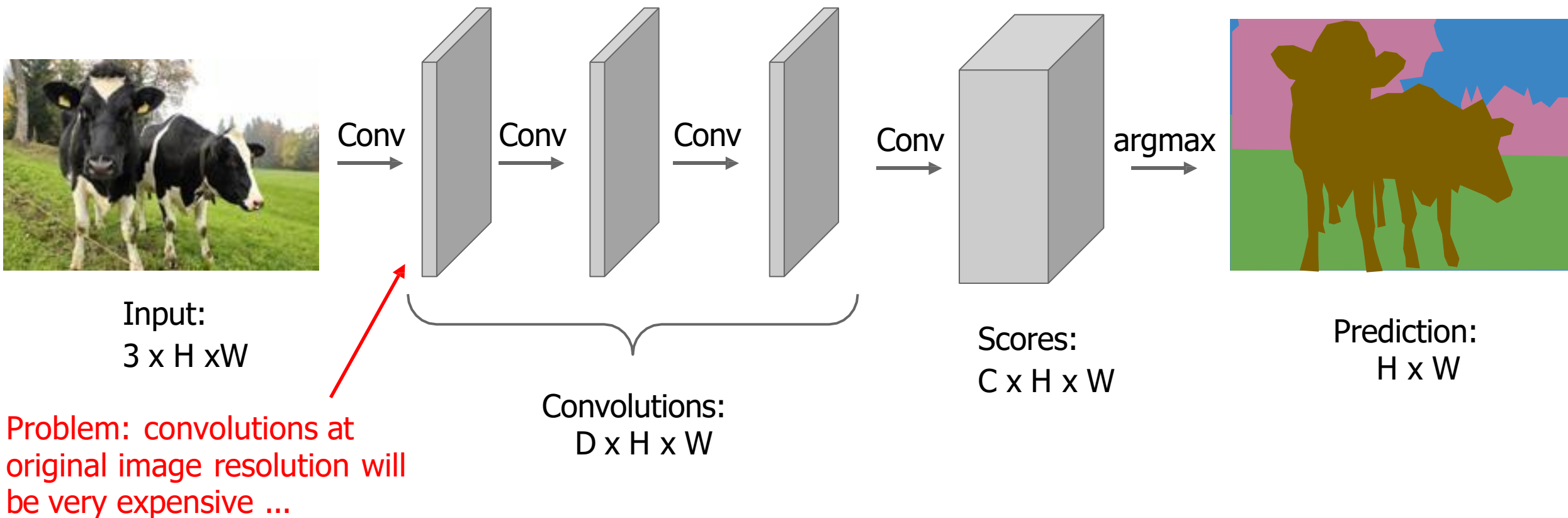
Another strategy is to ***modify*** our classification network by ***keeping the feature map size same*** throughout the network

Problems in ***training and inference*** persist



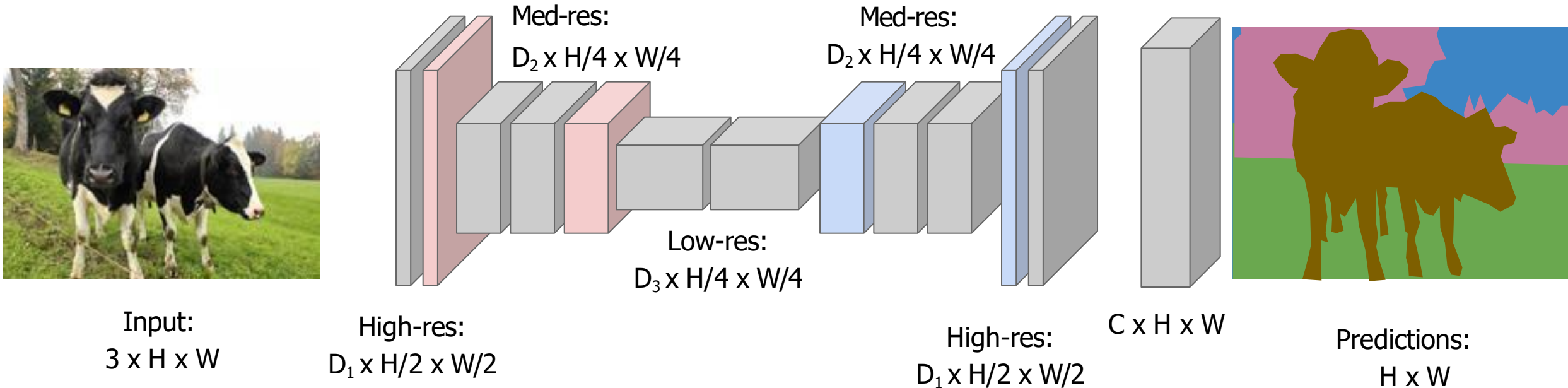
Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Semantic Segmentation Idea: Fully Convolutional

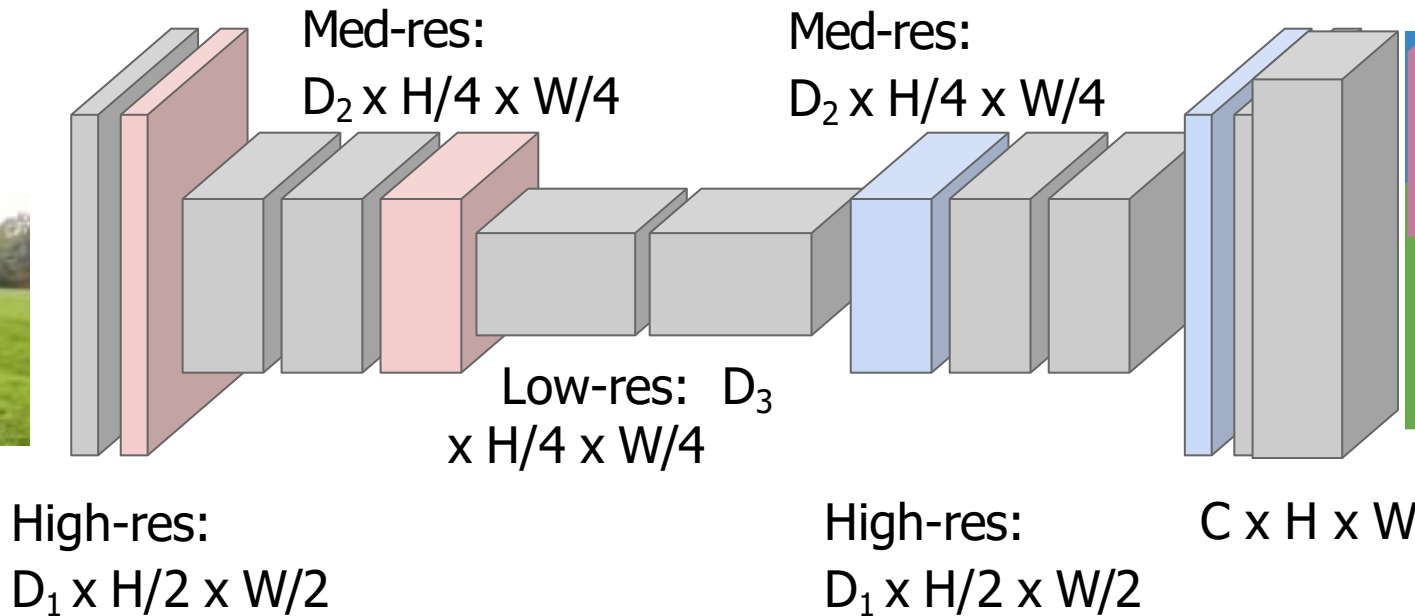
Downsampling:
Pooling, strided
convolution

Design network as a bunch of convolutional layers, with
downsampling and **upsampling** inside the network!

Upsampling:
???



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

In-Network upsampling: “Unpooling”

Nearest Neighbor

1	2
3	4

Input: 2 x 2



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Output: 4 x 4

“Bed of Nails”

1	2
3	4

Input: 2 x 2



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Output: 4 x 4

In-Network upsampling: “Max Unpooling”

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4



5	6
7	8

Output: 2 x 2



...

Rest of the network

Max Unpooling

Use positions from pooling layer

1	2
3	4

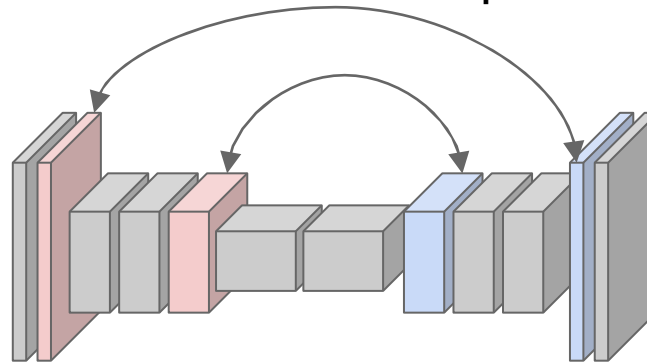
Input: 2 x 2



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

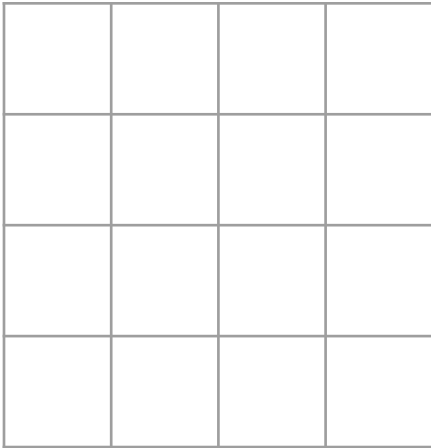
Output: 4 x 4

Corresponding pairs of
downsampling and
upsampling layers

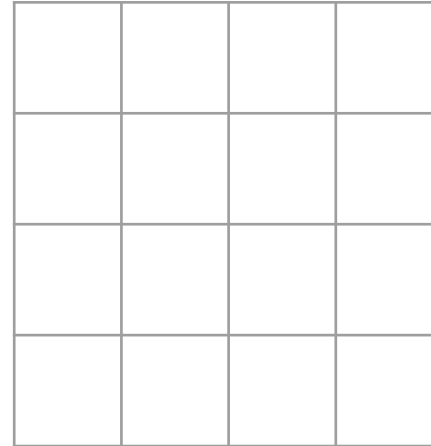


Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1



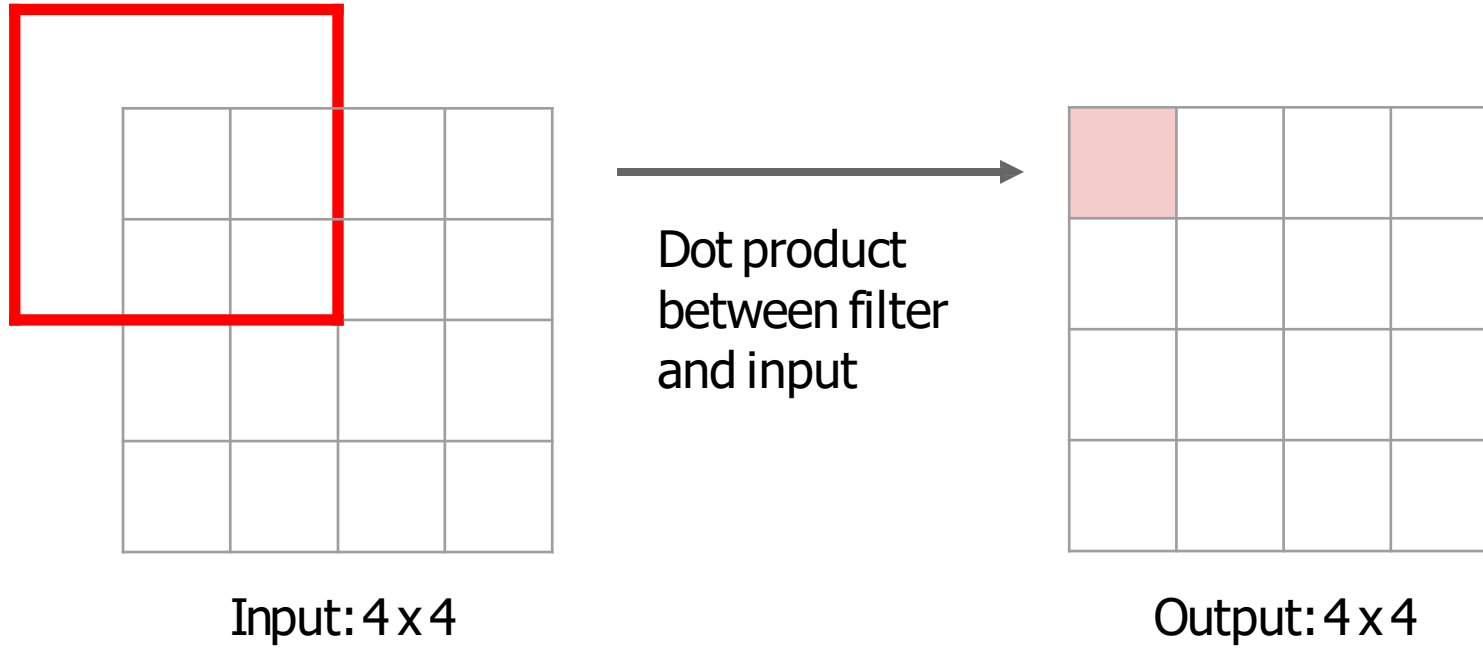
Input: 4 x 4



Output: 4 x 4

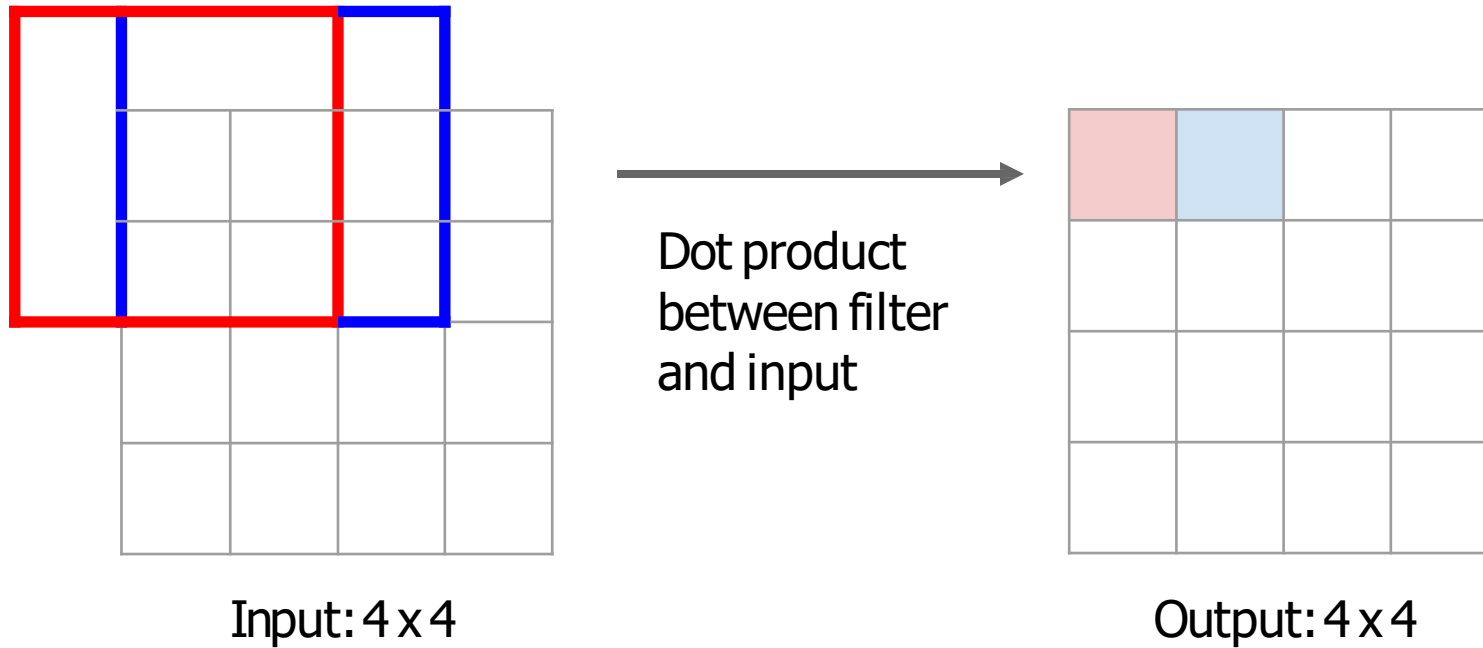
Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1



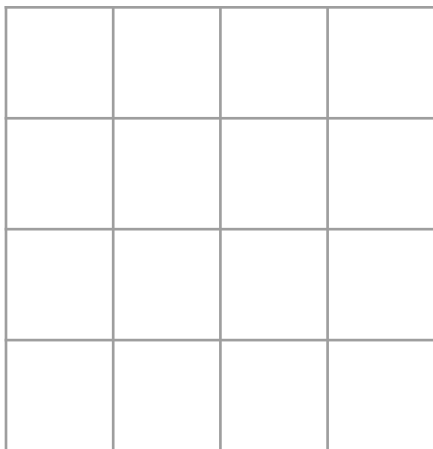
Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1

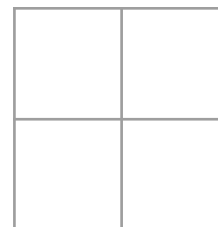


Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1



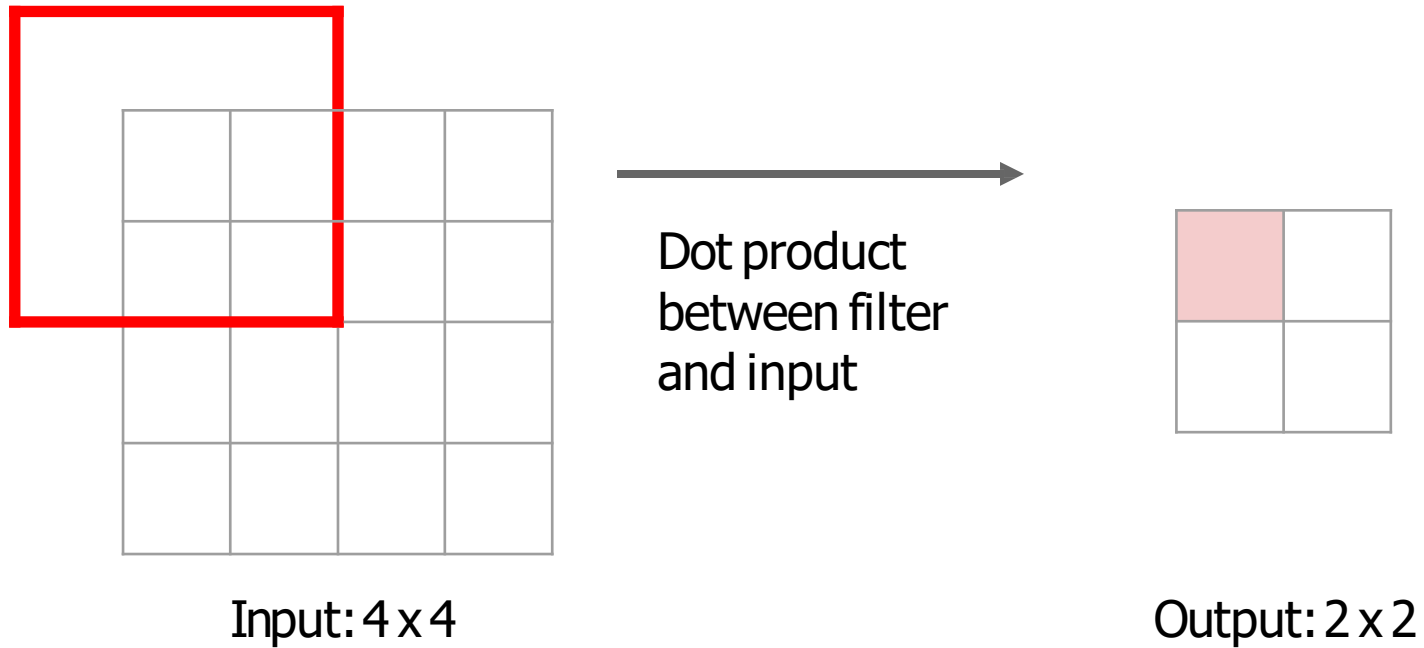
Input: 4 x 4



Output: 2 x 2

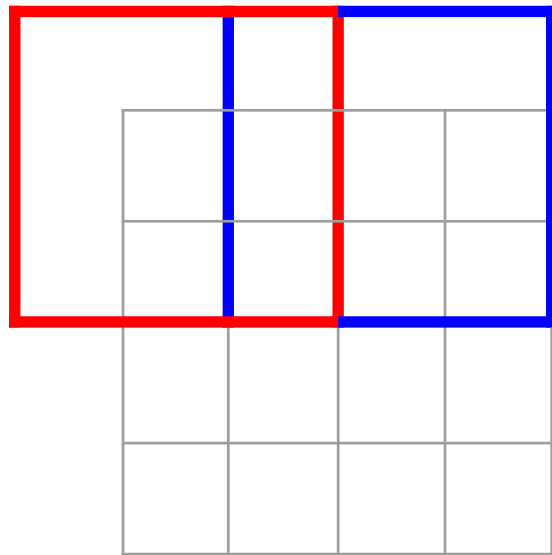
Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Learnable Upsampling

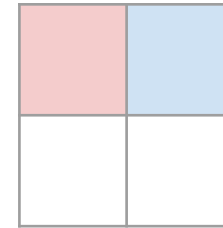
Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4



Dot product
between filter
and input



Output: 2 x 2

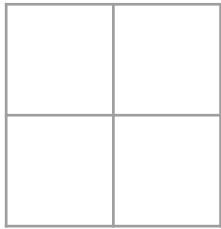
Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

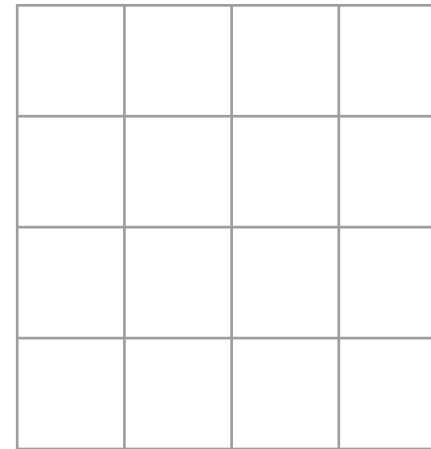
We can interpret strided convolution as "learnable downsampling".

Learnable Upsampling: Transposed Convolution

3 x 3 transposed convolution, stride 2 pad 1



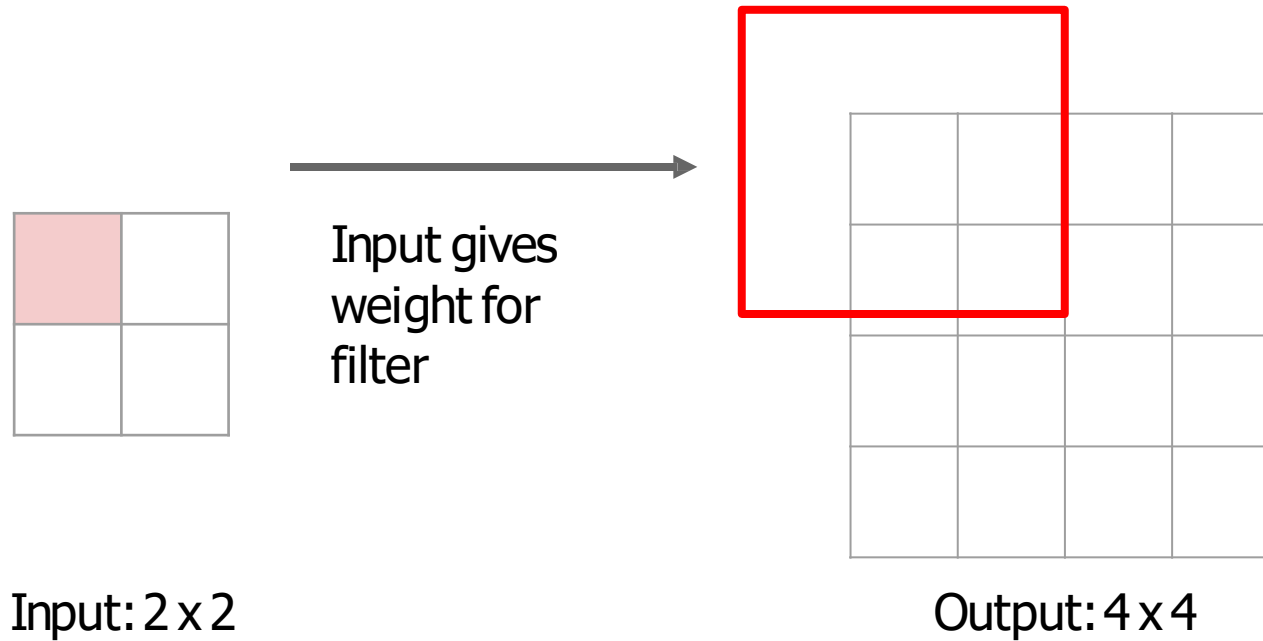
Input: 2 x 2



Output: 4 x 4

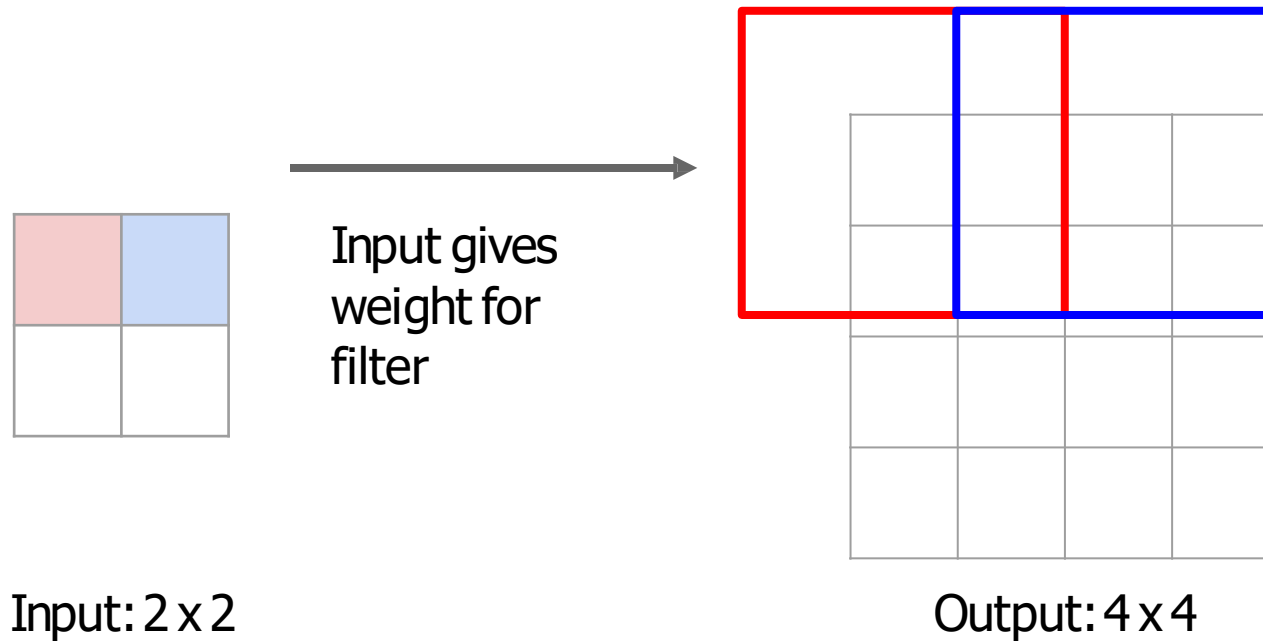
Learnable Upsampling: Transposed Convolution

3 x 3 transposed convolution, stride 2 pad 1



Learnable Upsampling: Transposed Convolution

3 x 3 transposed convolution, stride 2 pad 1

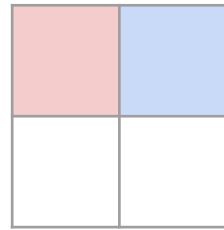


Filter moves 2 pixels in the output for every one pixel in the input

Stride gives ratio between movement in output and input

Learnable Upsampling: Transposed Convolution

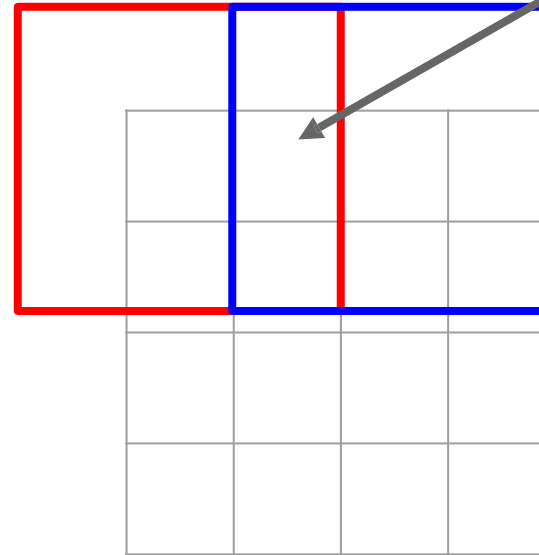
3 x 3 transposed convolution, stride 2 pad 1



Input: 2 x 2



Input gives
weight for
filter



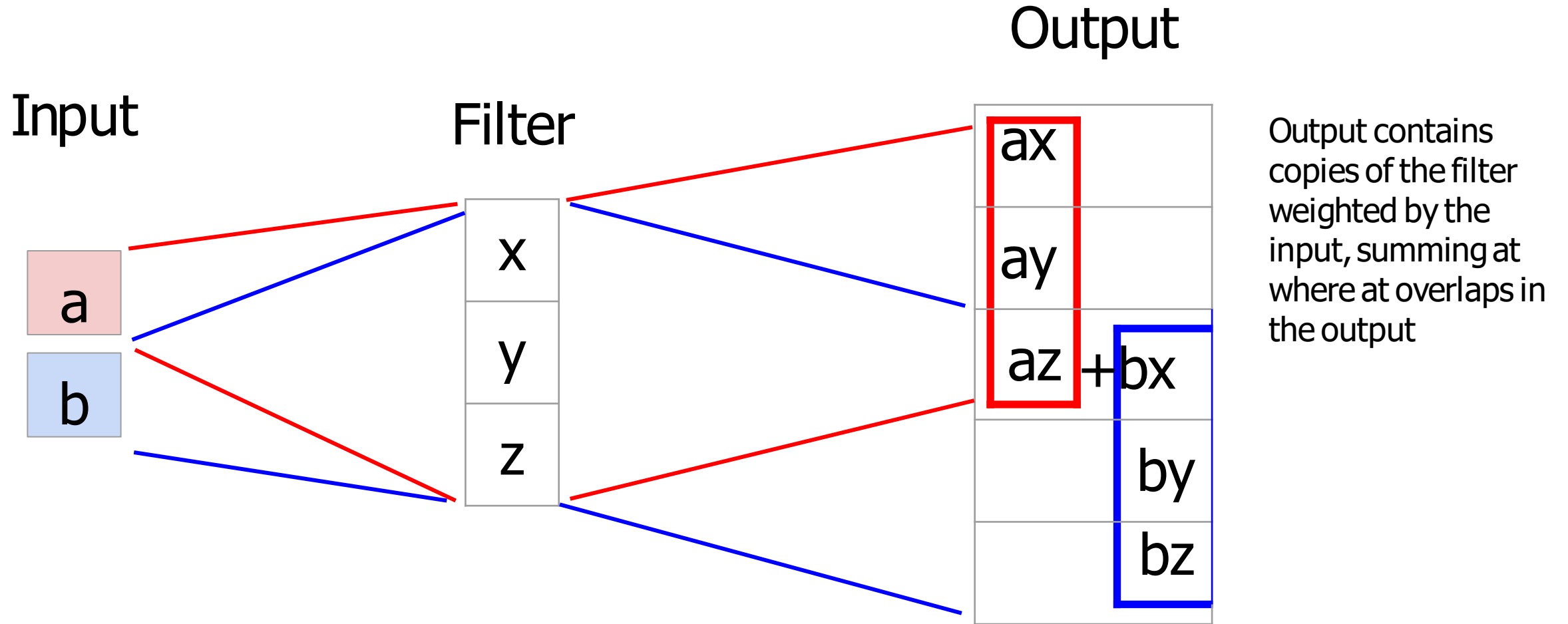
Output: 4 x 4

Sum where
output overlaps

Filter moves 2 pixels in the
output for every one pixel
in the input

Stride gives ratio between
movement in output and
input

Learnable Upsampling: 1D Example



Transpose Convolution Example

Input

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}$$

Stride = 1

Padding = 0

Final Output (after summing overlaps)

$$Y = \begin{bmatrix} 1 & 2 & 2 & 4 \\ 3 & 5 & 8 & 8 \\ 2 & 7 & 5 & 2 \\ 6 & 8 & 3 & 4 \end{bmatrix}$$

Place $1 \cdot K$ starting at (0,0):

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}$$

Place $2 \cdot K$ starting at (0,1):

$$2K = \begin{bmatrix} 2 & 0 & 4 \\ 0 & 2 & 0 \\ 4 & 0 & 2 \end{bmatrix}$$

Place $3 \cdot K$ starting at (1,0):

$$3K = \begin{bmatrix} 3 & 0 & 6 \\ 0 & 3 & 0 \\ 6 & 0 & 3 \end{bmatrix}$$

Place $4 \cdot K$ starting at (1,1):

$$4K = \begin{bmatrix} 4 & 0 & 8 \\ 0 & 4 & 0 \\ 8 & 0 & 4 \end{bmatrix}$$

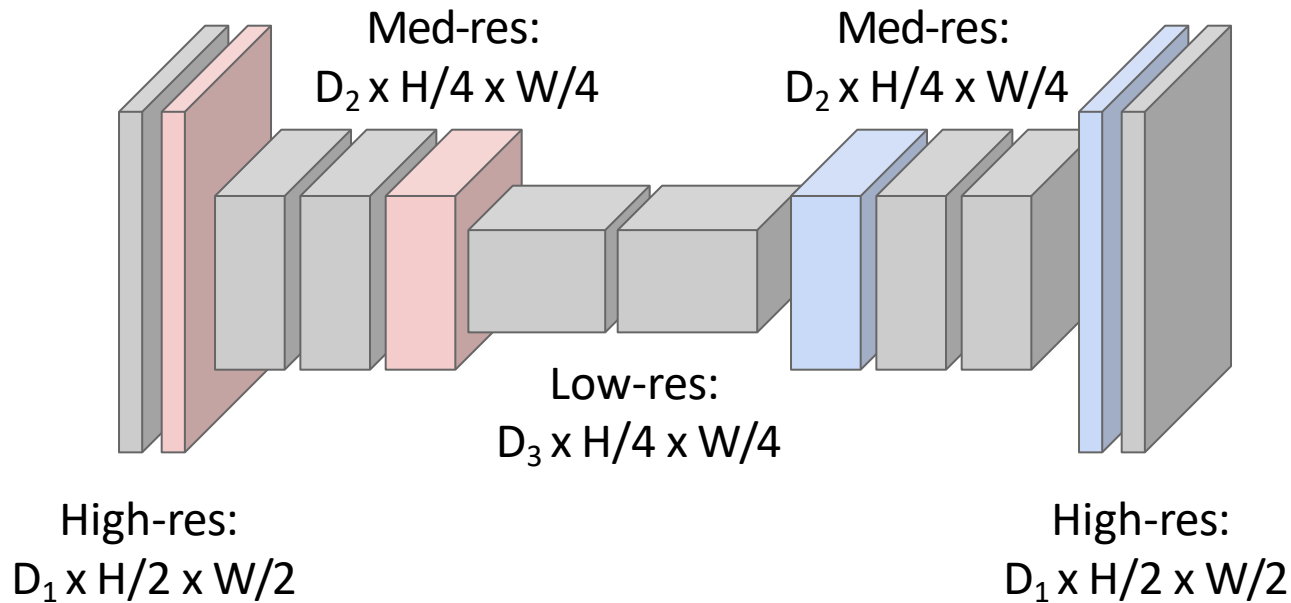
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with
downsampling and **upsampling** inside the network!

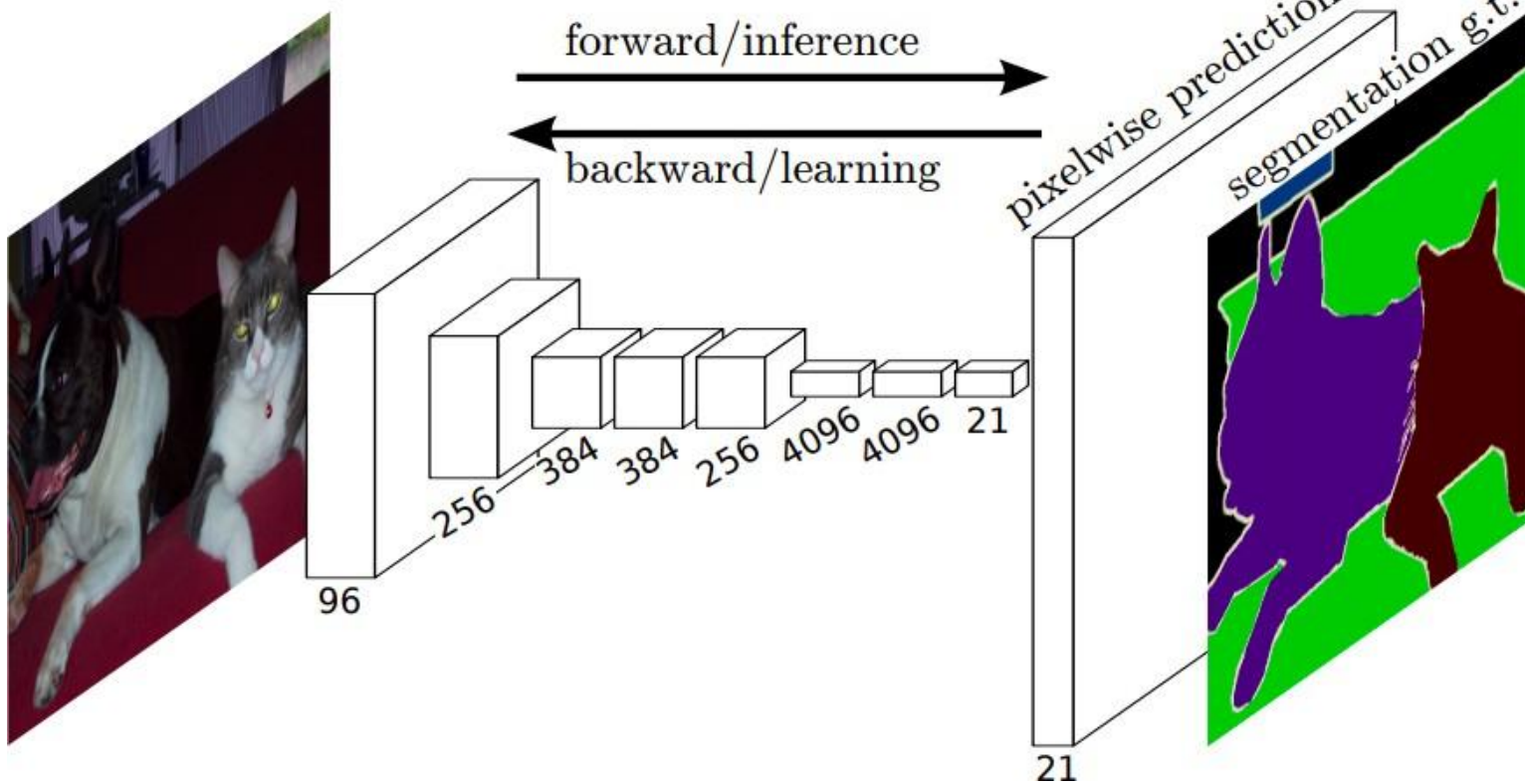
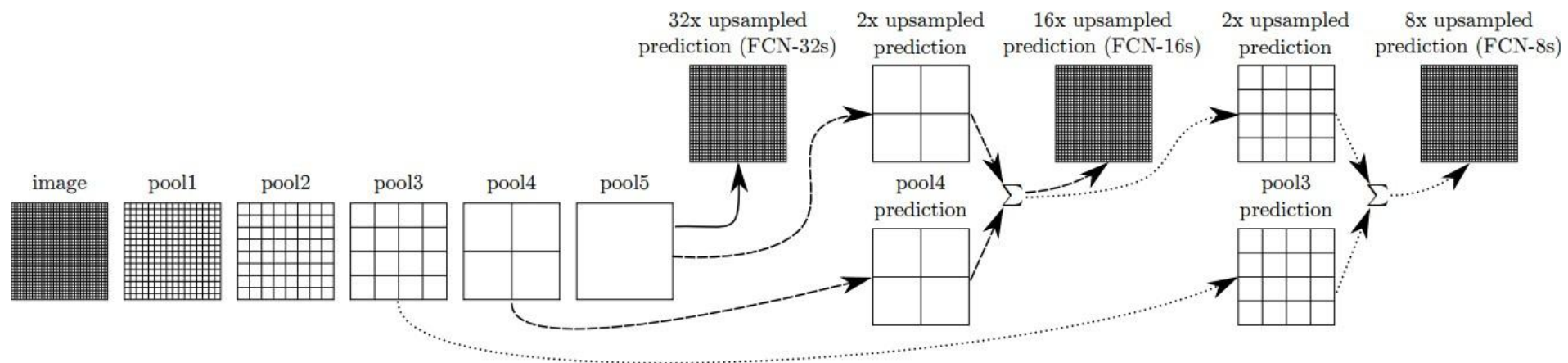


Upsampling:
Unpooling or strided
transposed convolution

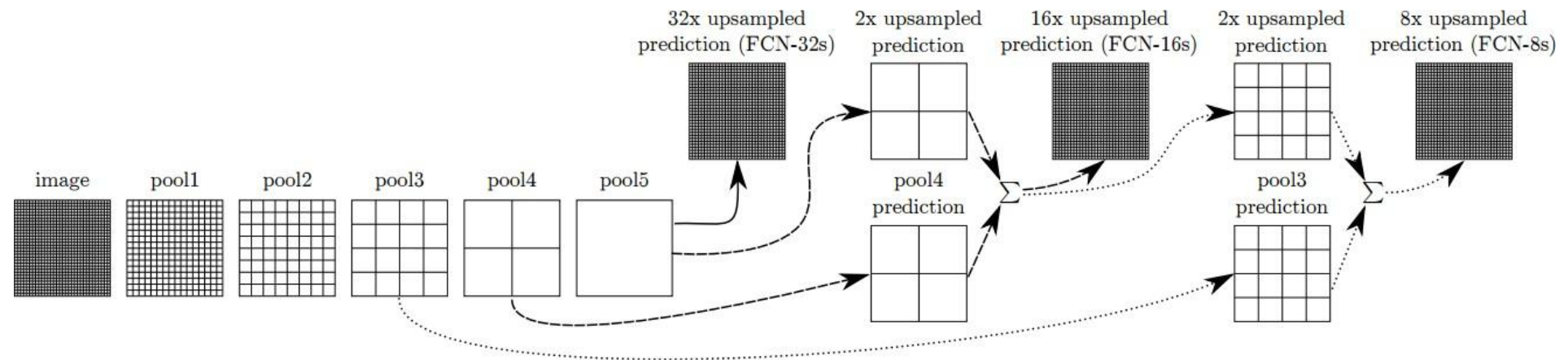


Predictions:
 $H \times W$

FCN



FCN

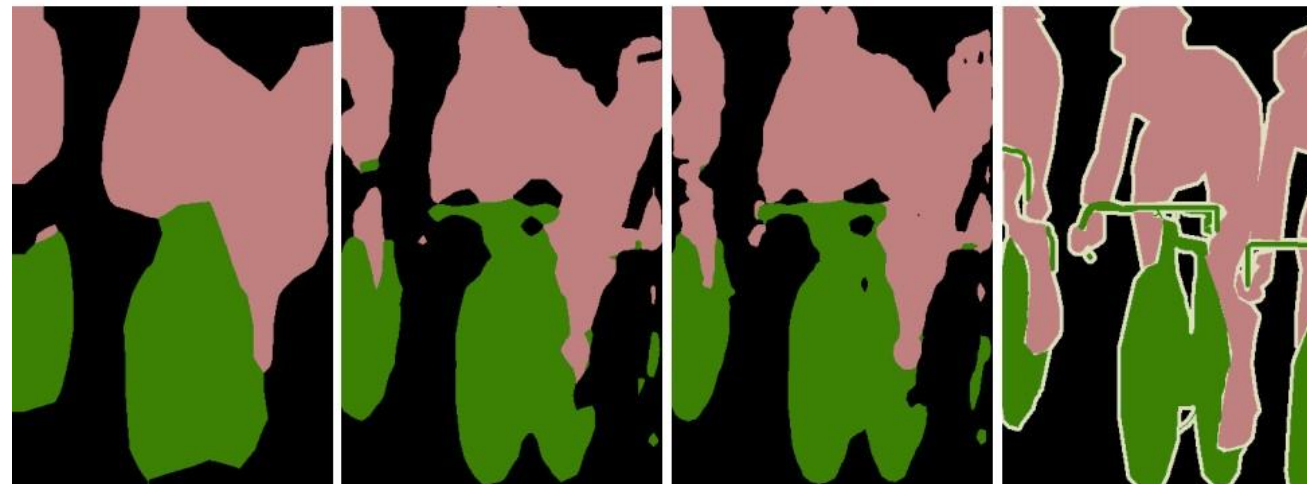


FCN-32s

FCN-16s

FCN-8s

Ground truth

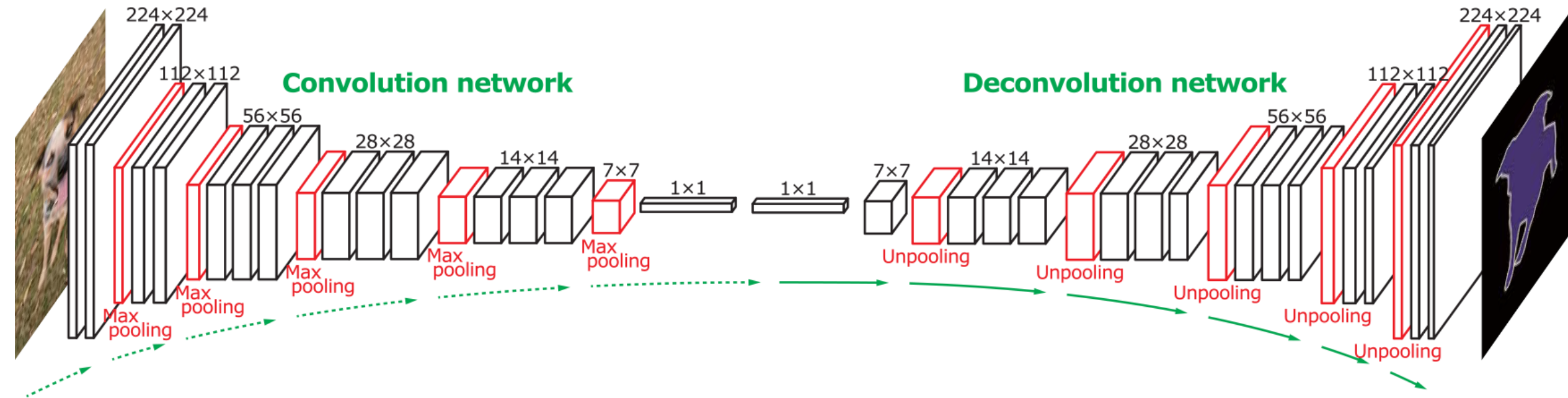


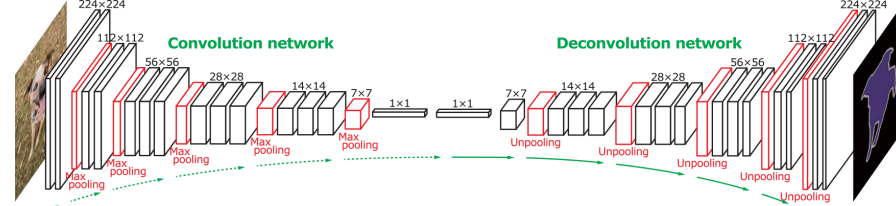
No skip connection

1-skip connection

2-skip connections

Deconvolution Network for Semantic Segmentation



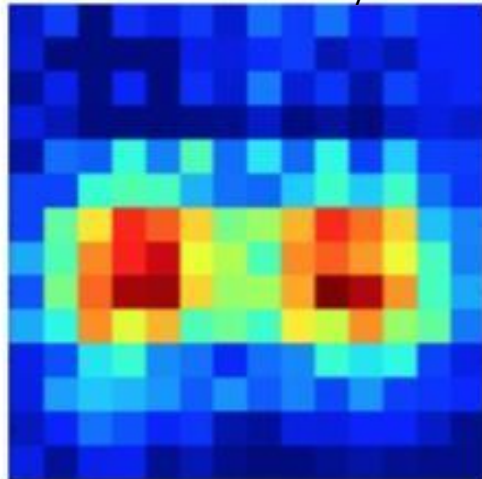


Input image



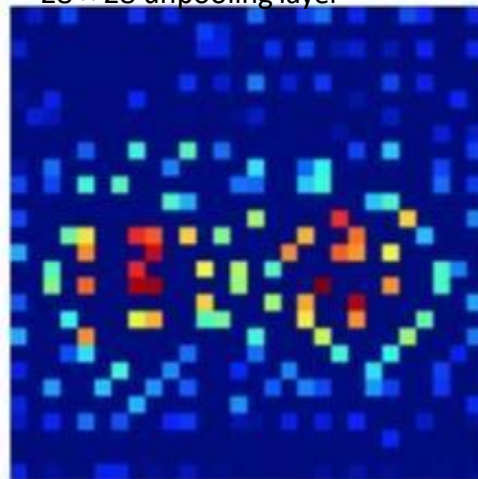
(a)

14 × 14 deconvolutional layer



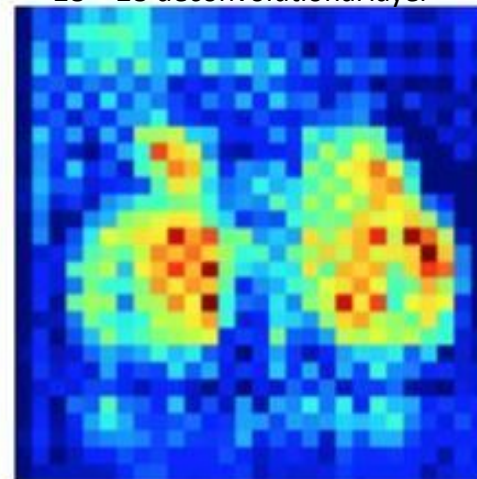
(b)

28 × 28 unpooling layer



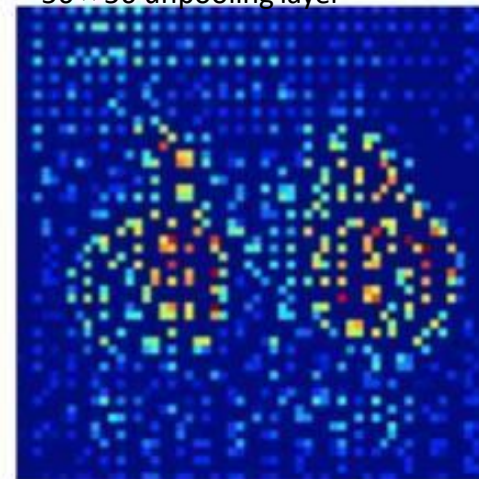
(c)

28 × 28 deconvolutional layer



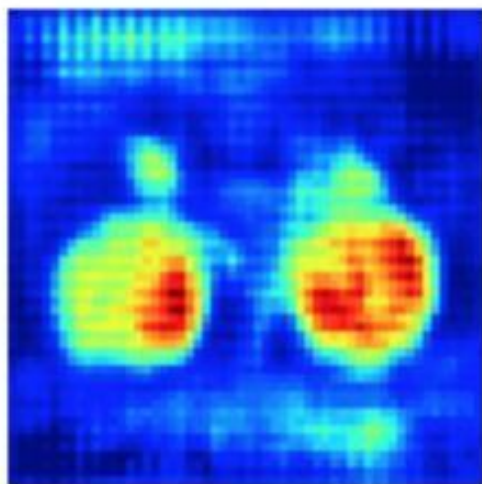
(d)

56 × 56 unpooling layer



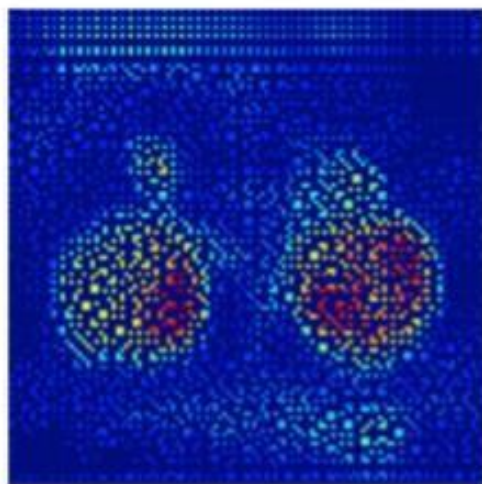
(e)

56 × 56 deconvolutional layer



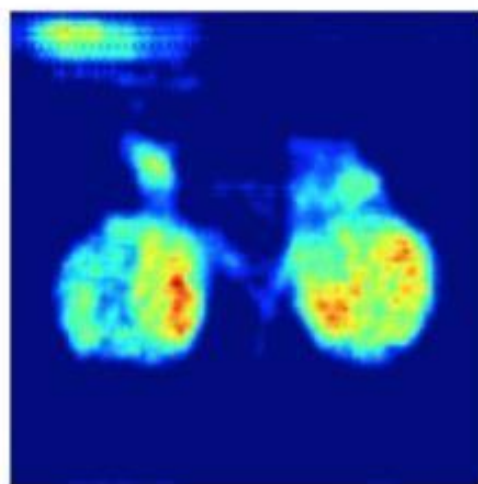
(f)

112 × 112 unpooling layer



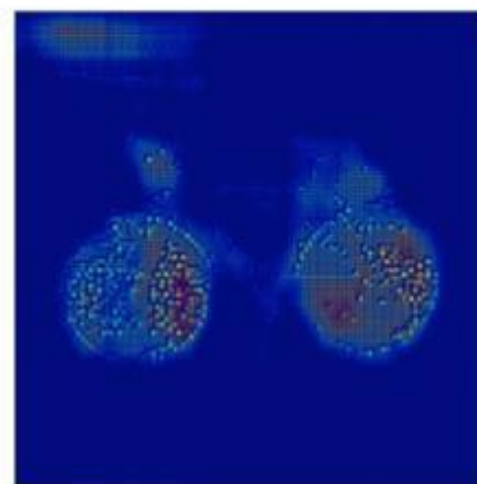
(g)

112 × 112 deconvolutional layer



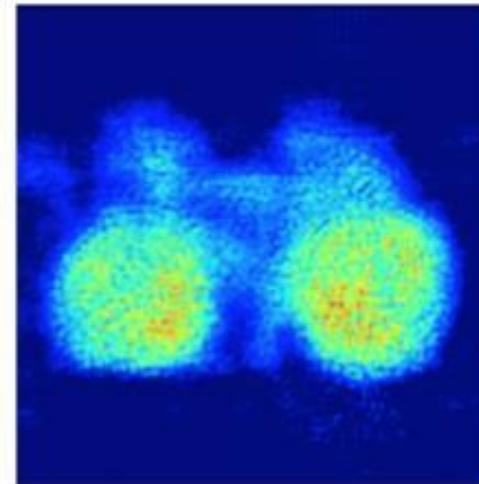
(h)

224 × 224 unpooling layer



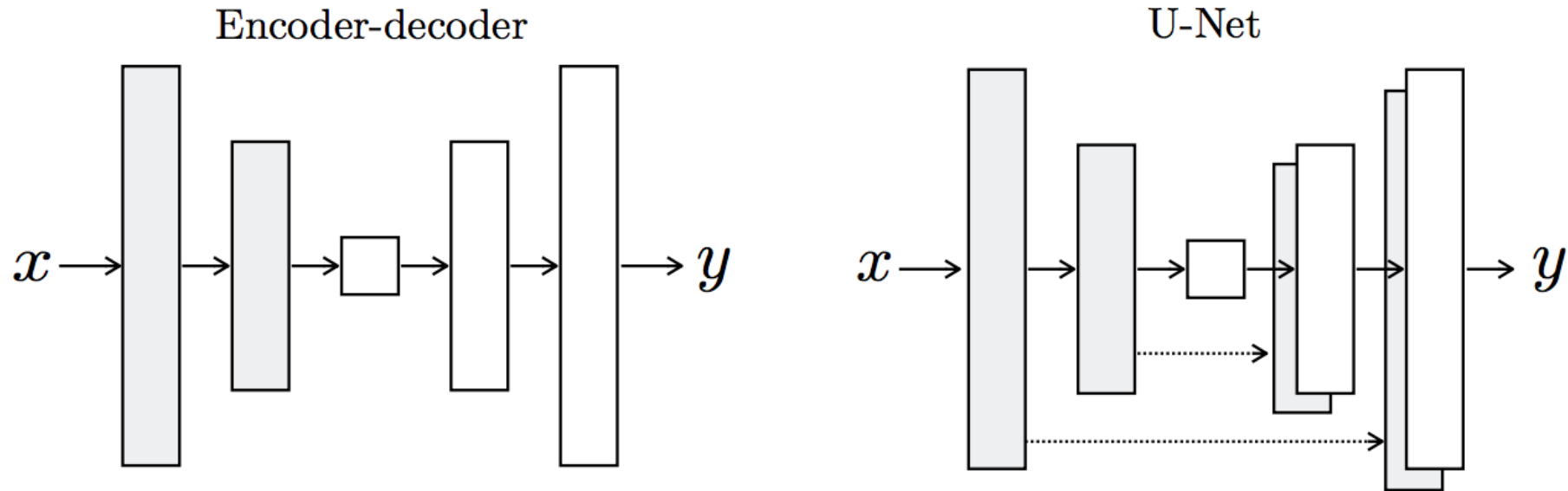
(i)

224 × 224 deconvolutional layer

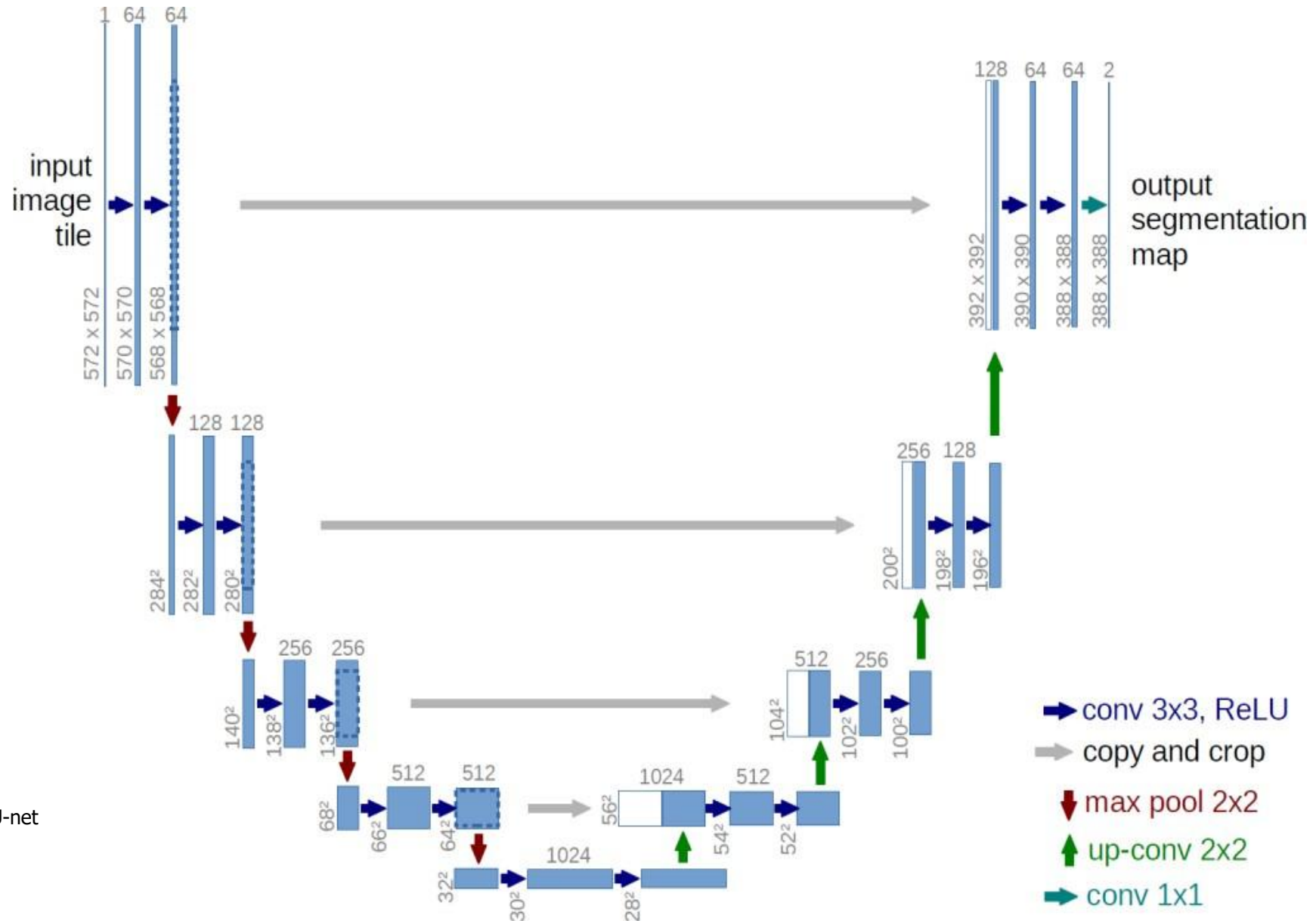


(j)

Learned upsampling architectures

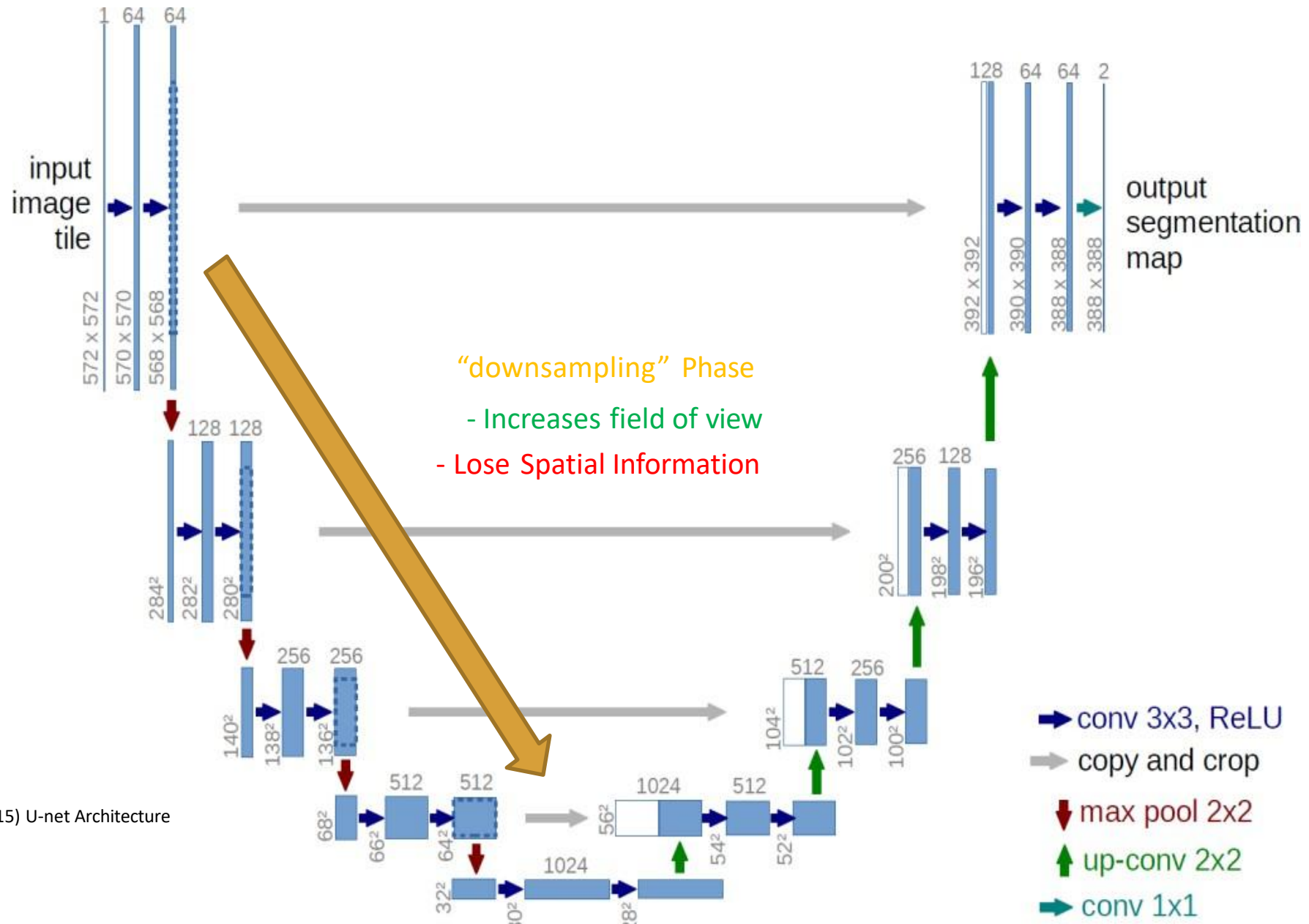


U-Net

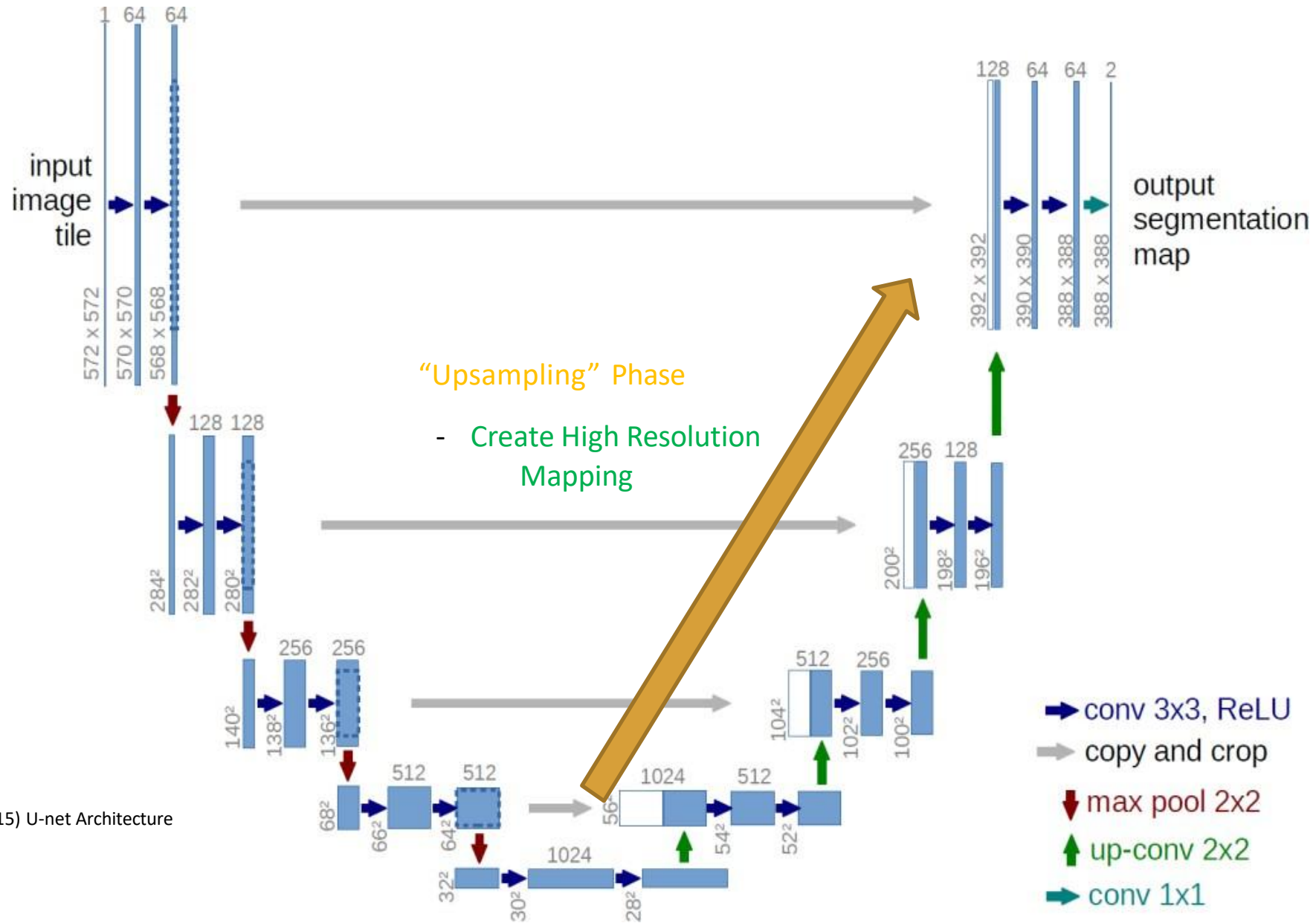


Ronneberger et al. (2015) U-net
Architecture

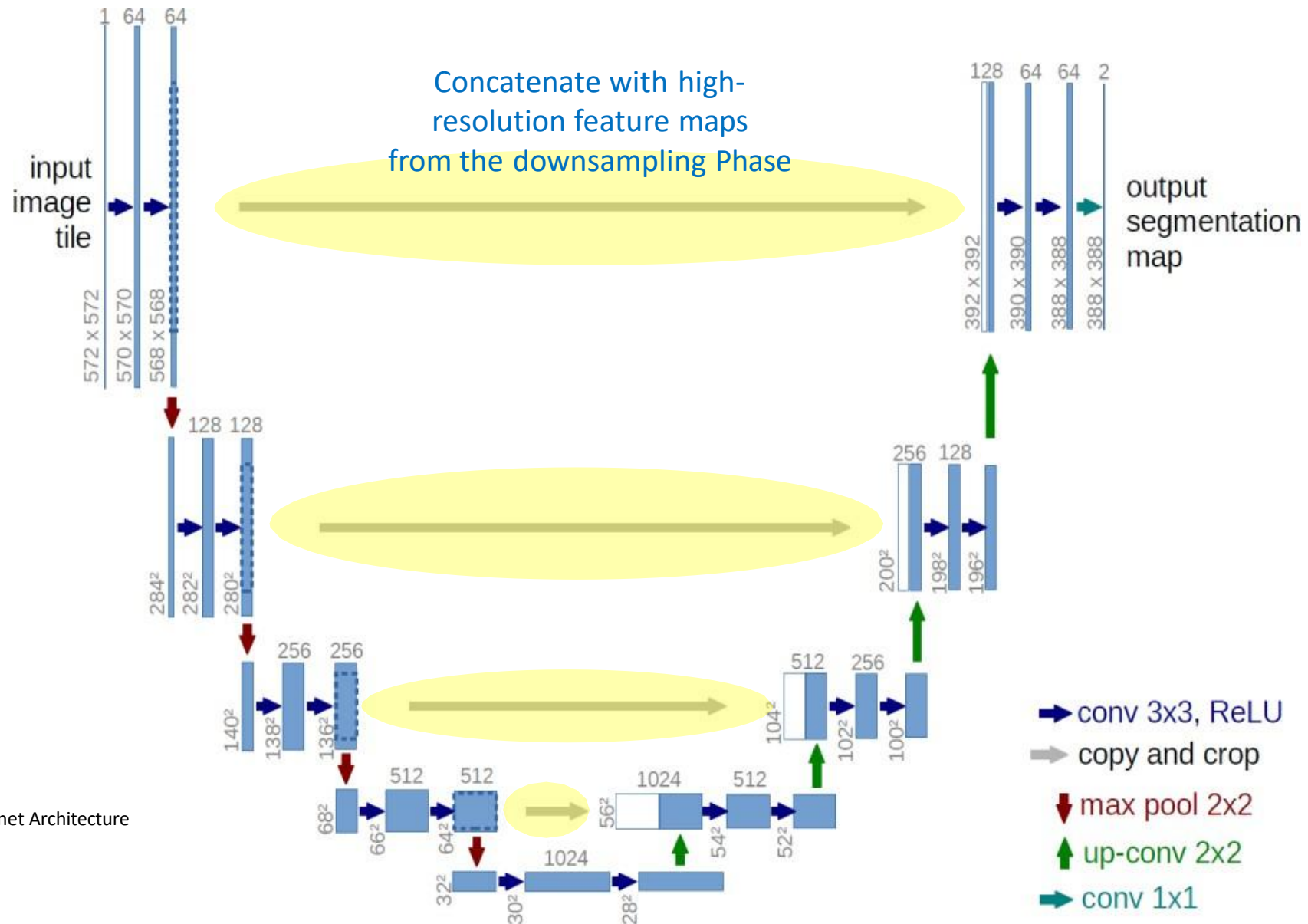
U-Net



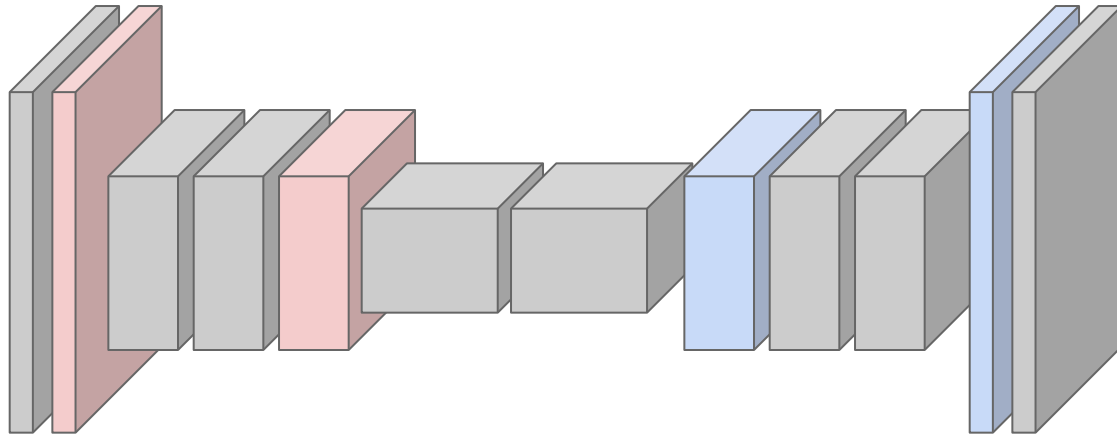
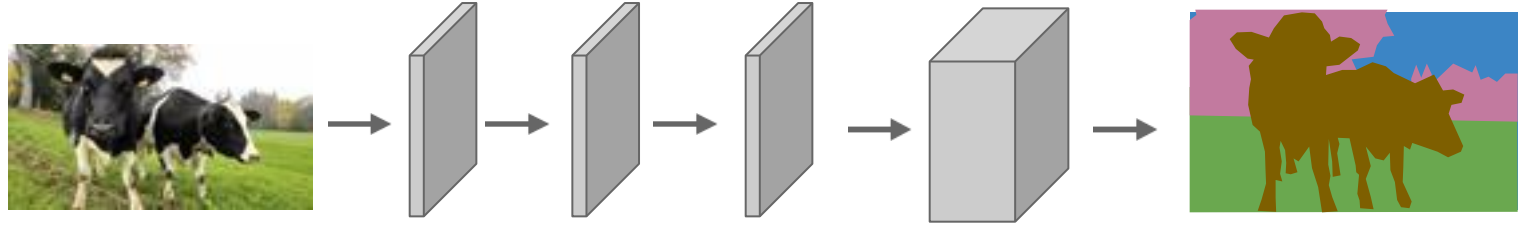
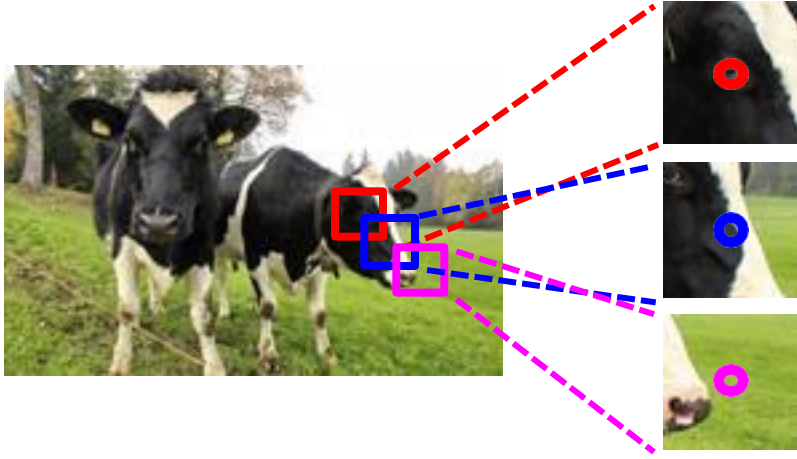
U-Net



U-Net



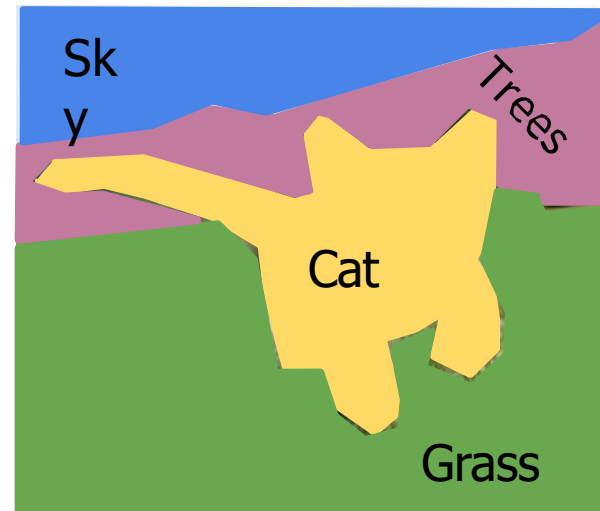
Semantic Segmentation: Summary



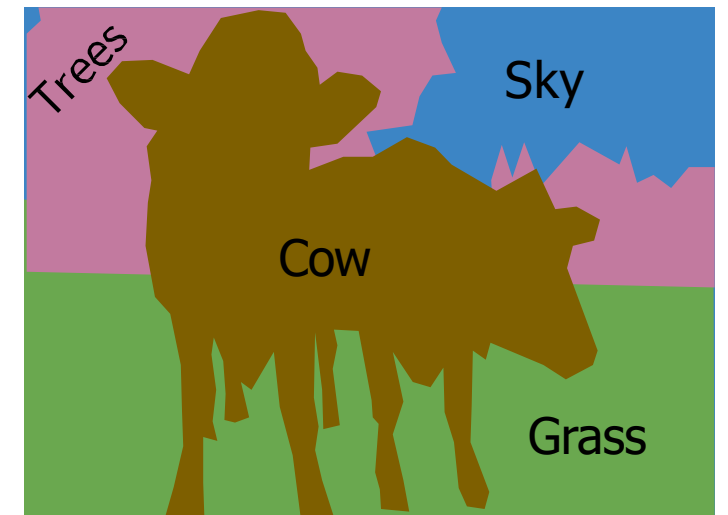
Semantic Segmentation

Label each pixel in the image with a category label

Don't differentiate instances, only care about pixels



[This image](#) is [CC0 public domain](#)



Object Detection

Object Recognition

- **Problem:** Given an image A, does A contain an image of a person?



YES

Object localization



Phil Noble / AP



Mike Hewitt / Allsport



Patrick Gardin / AP



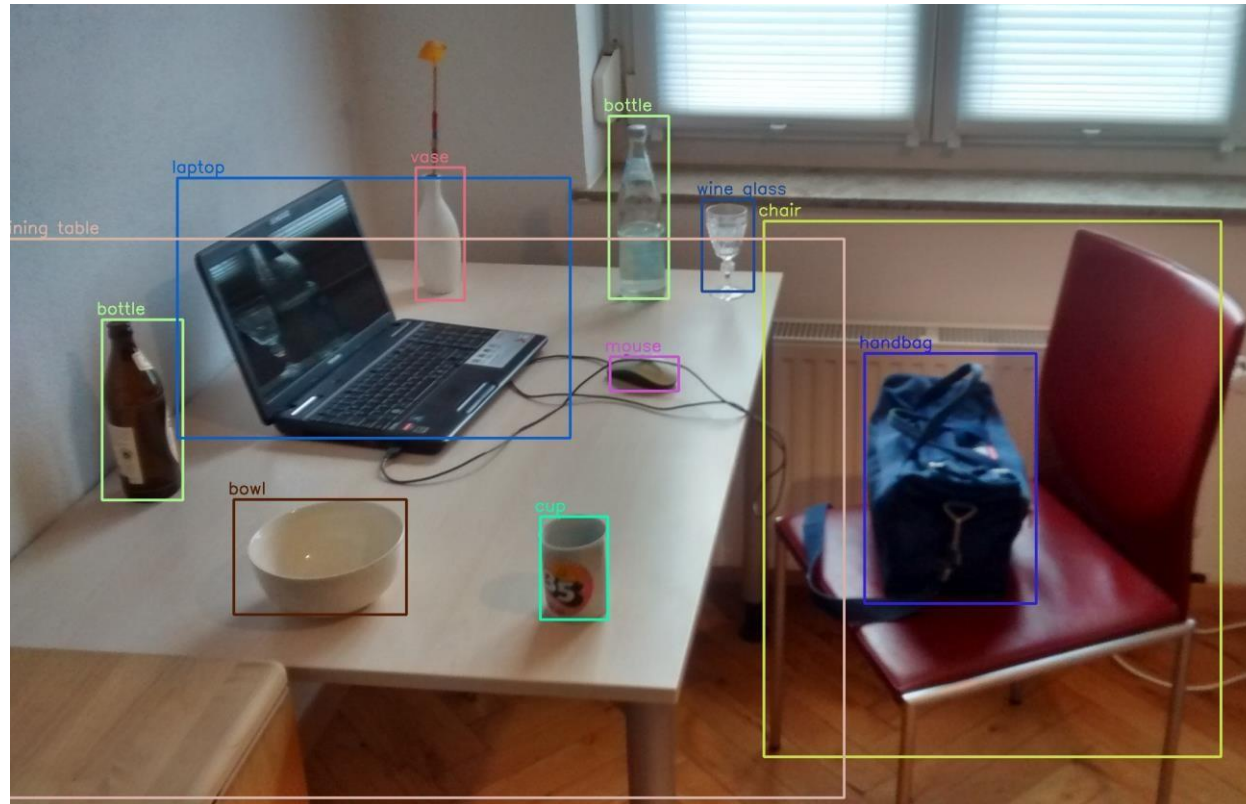
Andy Barron / Reno Gazette-Journal



Sydney Morning Herald

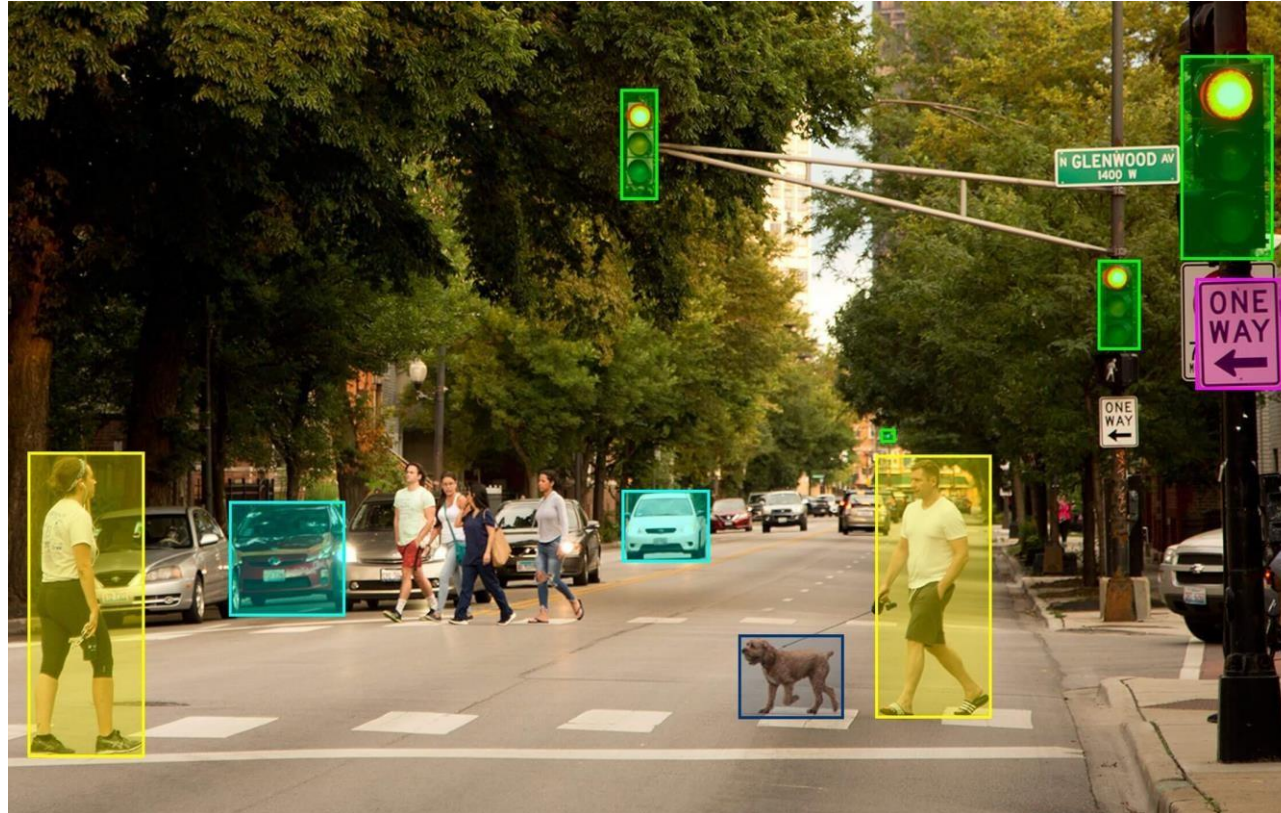
Object detection

- Multiple objects



Why detection?

- Self-driving car

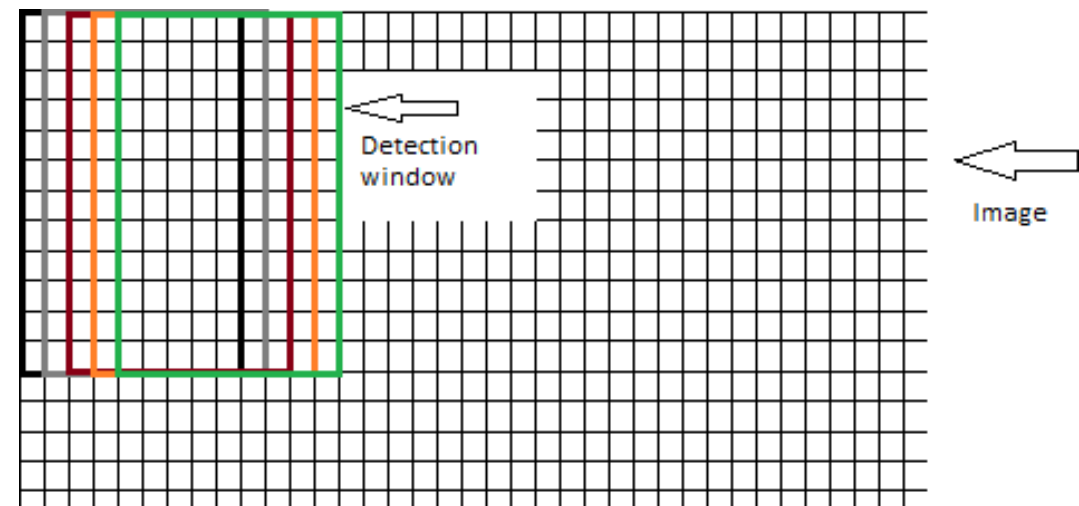


A simple solution

Sliding window

Sliding Window

- Slide a box around image and classify each crop
- There are many interesting papers
 - Viola and Jones (2001): Face detector (22K citations)
 - Dalal and Triggs (2005): HOG (33K citations)



Naïve approach: Template Matching

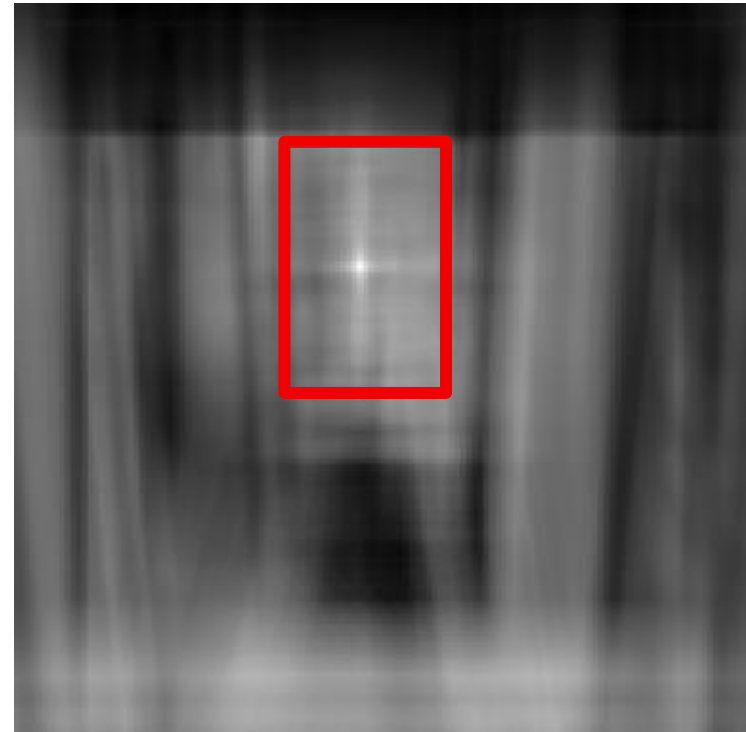
This is a chair



Find the chair in this image

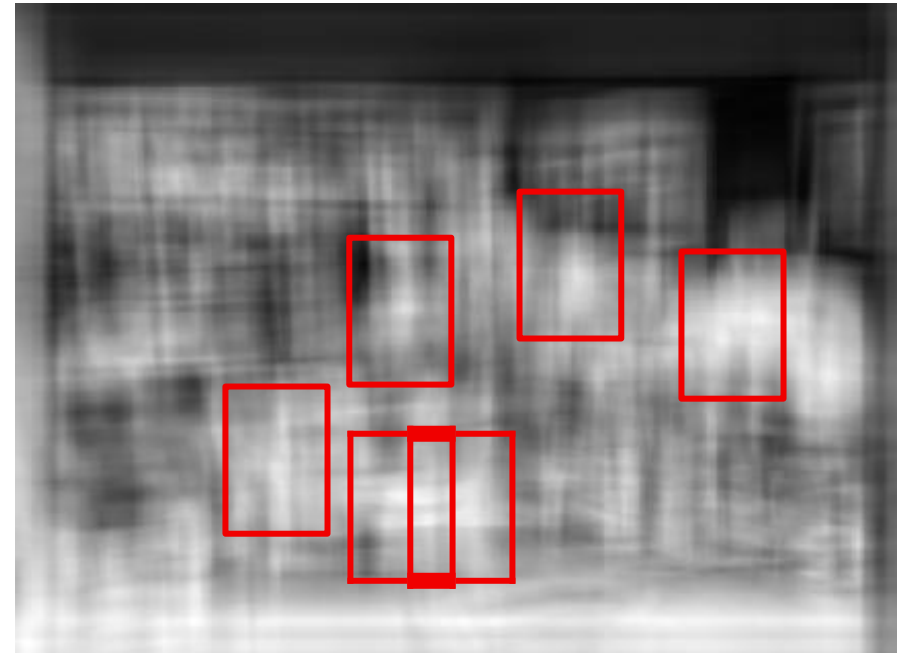
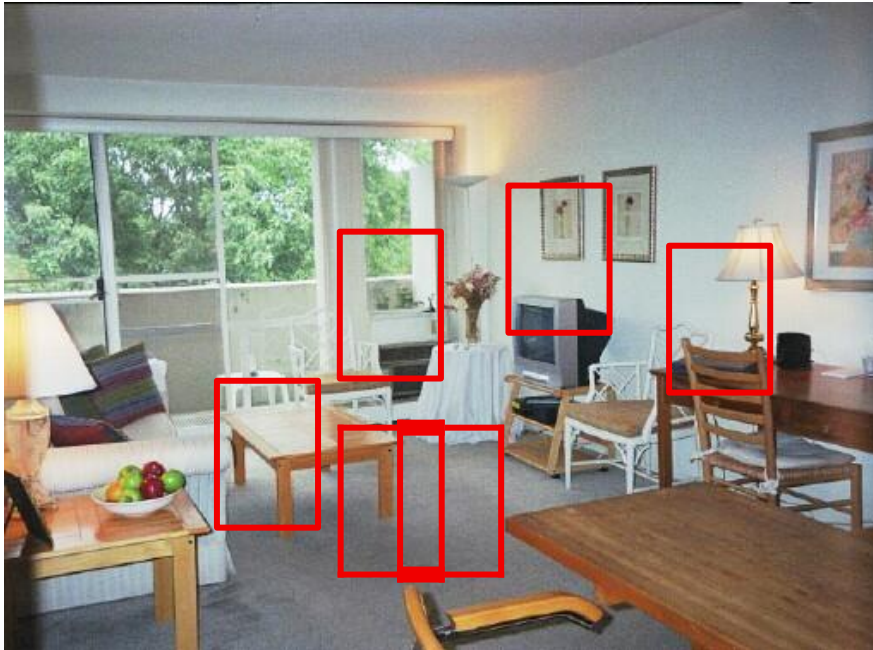


Output of correlation



Template Matching

Find the chair in this image



Epic fail!
Simple template matching is not going to make it

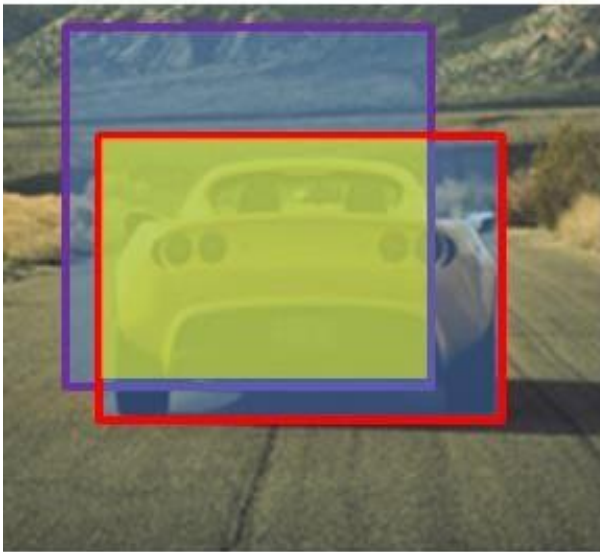
Sliding Window

- General approach
 - Scan all possible locations
 - Extract features
 - Classify features
 - Post-processing



Evaluation

- Intersection over union



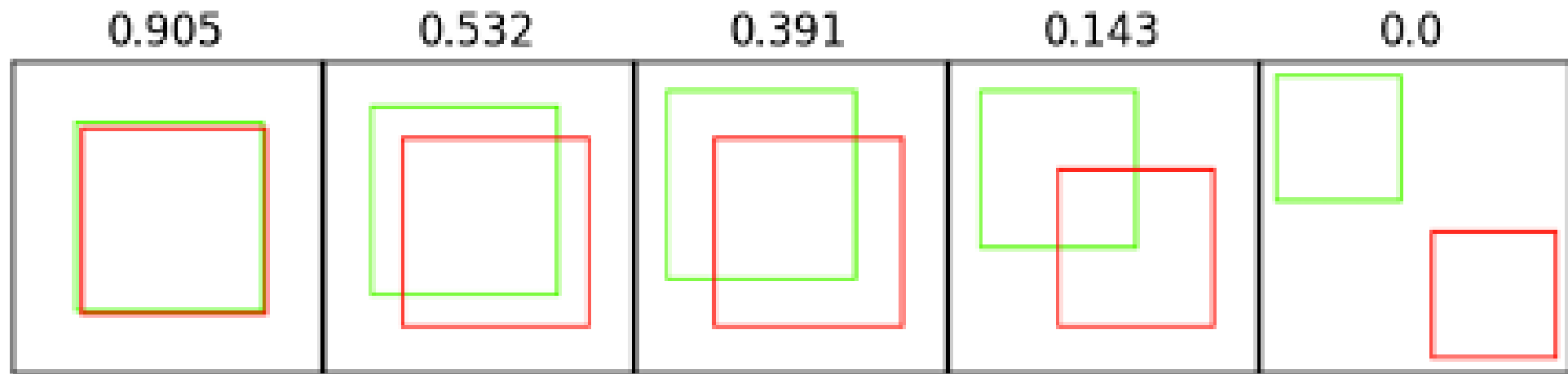
Intersection over union (IoU)

$$= \frac{\text{size of } \text{[yellow box]}}{\text{size of } \text{[blue box]}}$$

“Correct” if $\text{IoU} \geq 0,5$

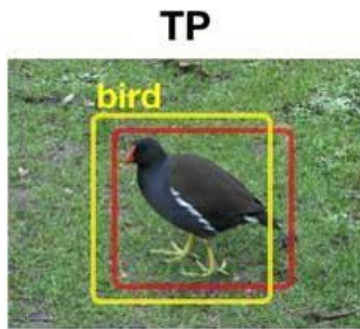
Sample IoU

Sample IoU scores



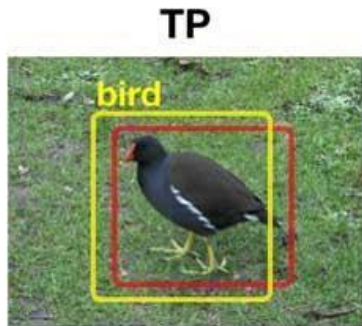
Evaluation

- True positives



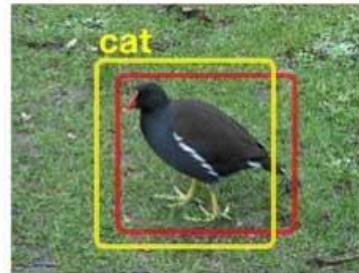
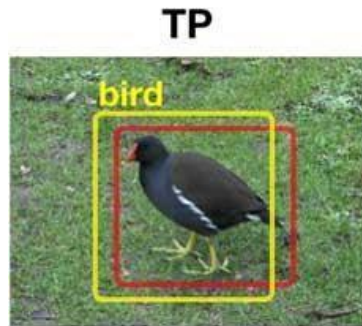
Evaluation

- True positives
- False positives

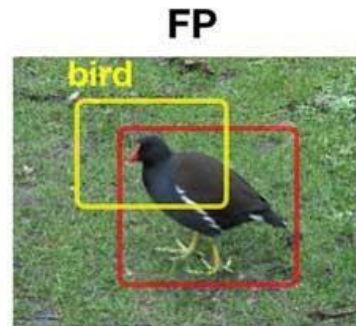


Evaluation

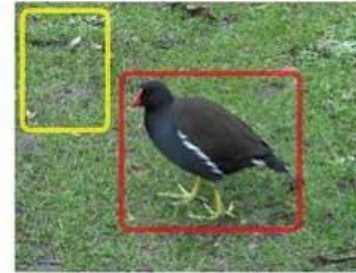
- True positives
- False positives



wrong class



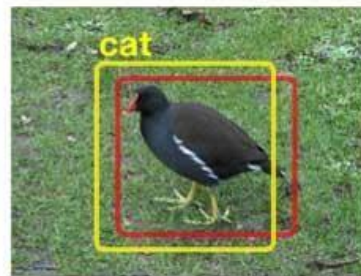
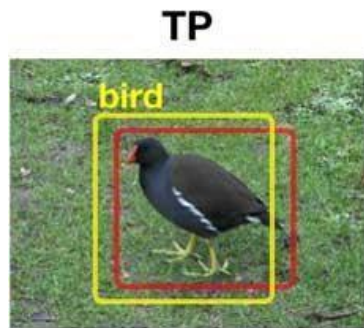
$\text{IOU} < 0.5$



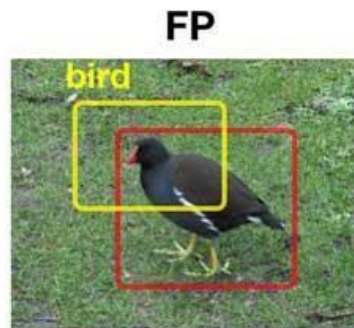
no overlap

Evaluation

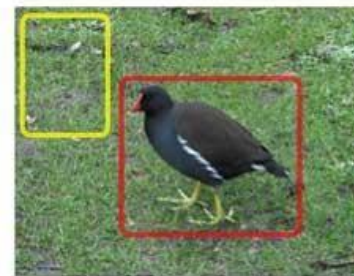
- True positives
- False positives
- False negatives



wrong class



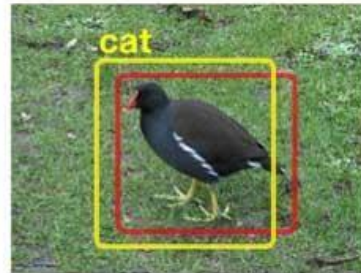
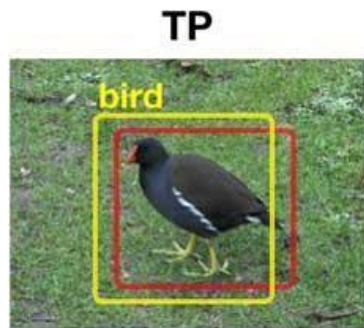
IOU < 0.5



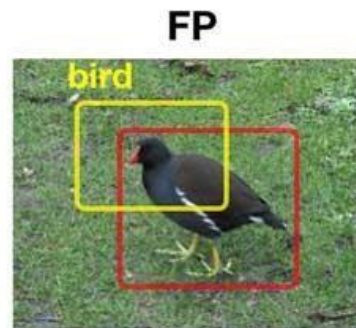
no overlap

Evaluation

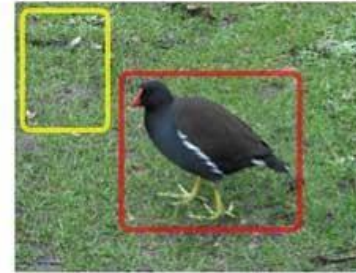
- True positives
- False positives
- False negatives



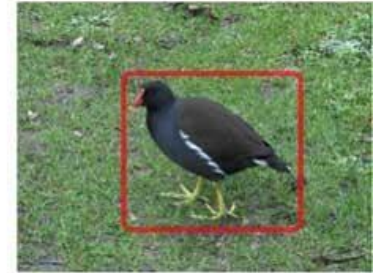
wrong class



$\text{IOU} < 0.5$



no overlap



no prediction

Evaluation

- Precision

- Precision is the ability of a model to identify **only** the relevant objects.
- It is the percentage of correct positive predictions and is given by:

$$precision = \frac{true\ positives}{true\ positives + false\ positives}$$

- Recall

- Recall is the ability of a model to find all the relevant cases
- It is the percentage of true positive detected among all relevant ground truths and is given by:

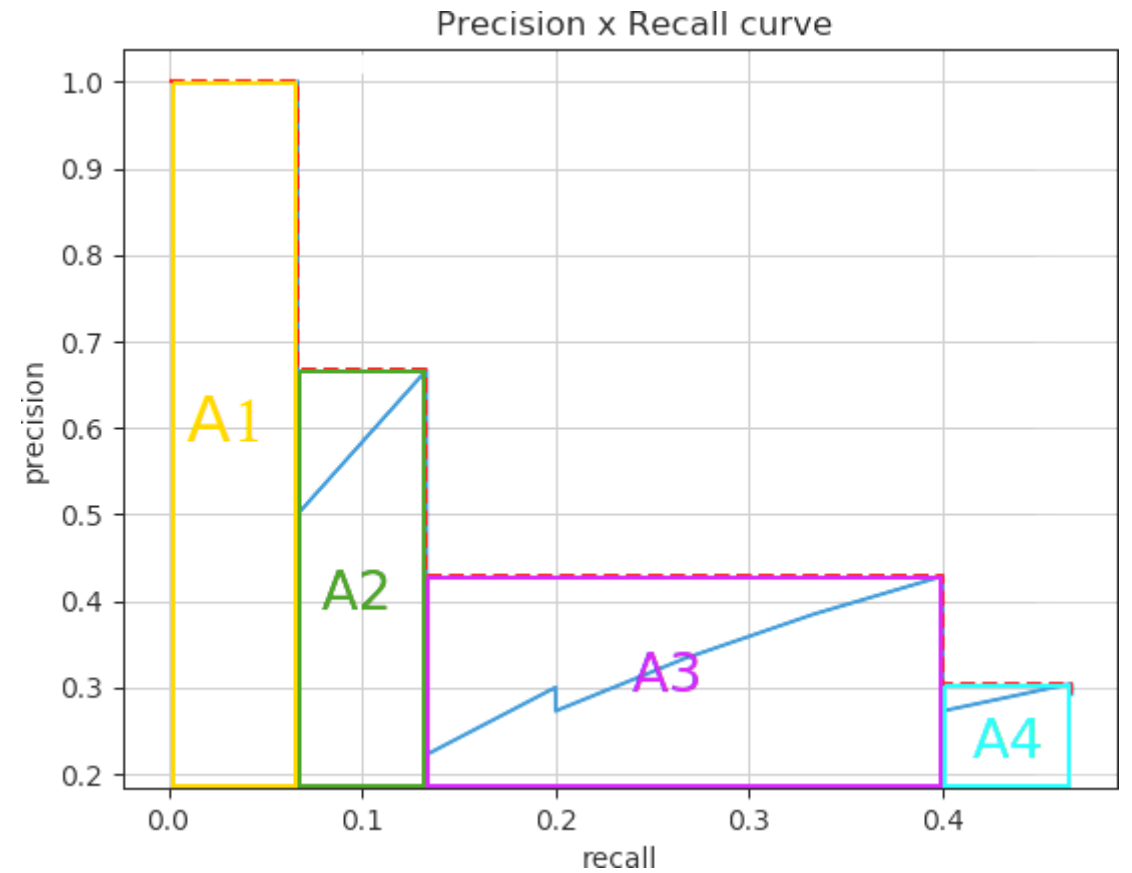
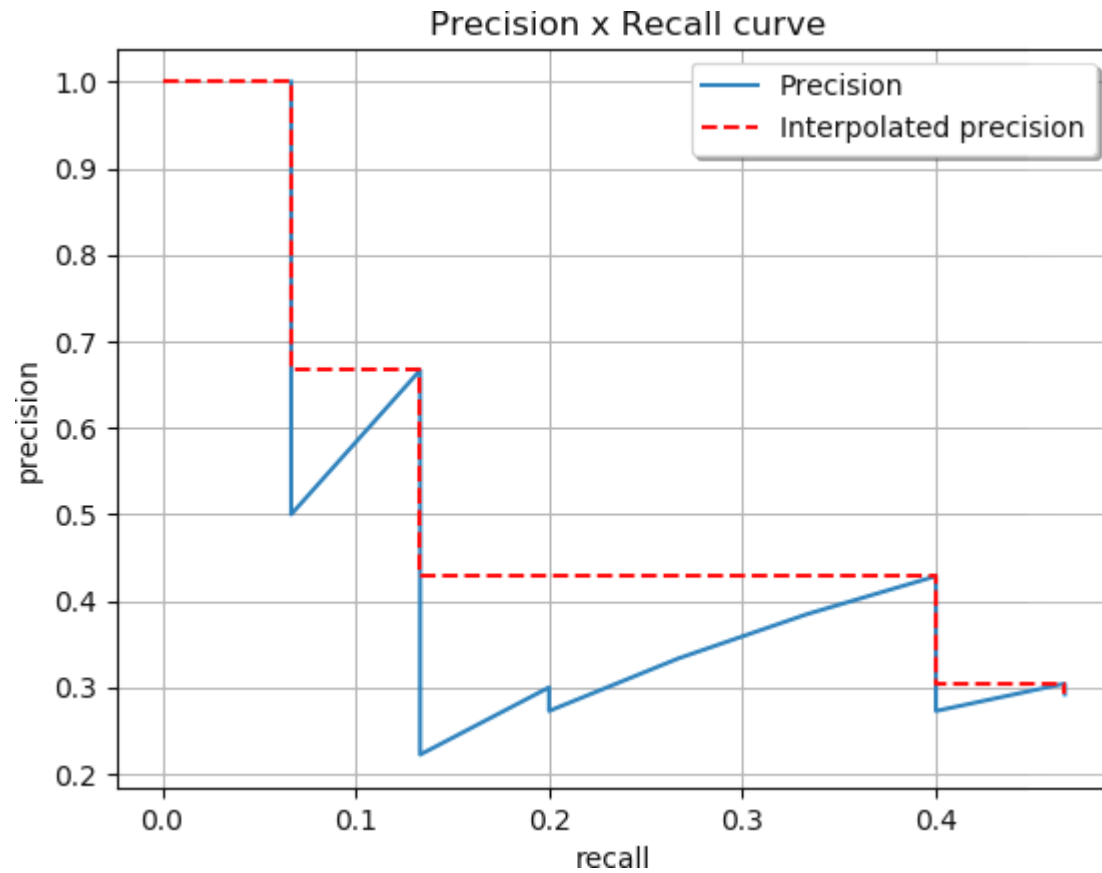
$$recall = \frac{true\ positives}{true\ positives + false\ negatives}$$

Evaluation

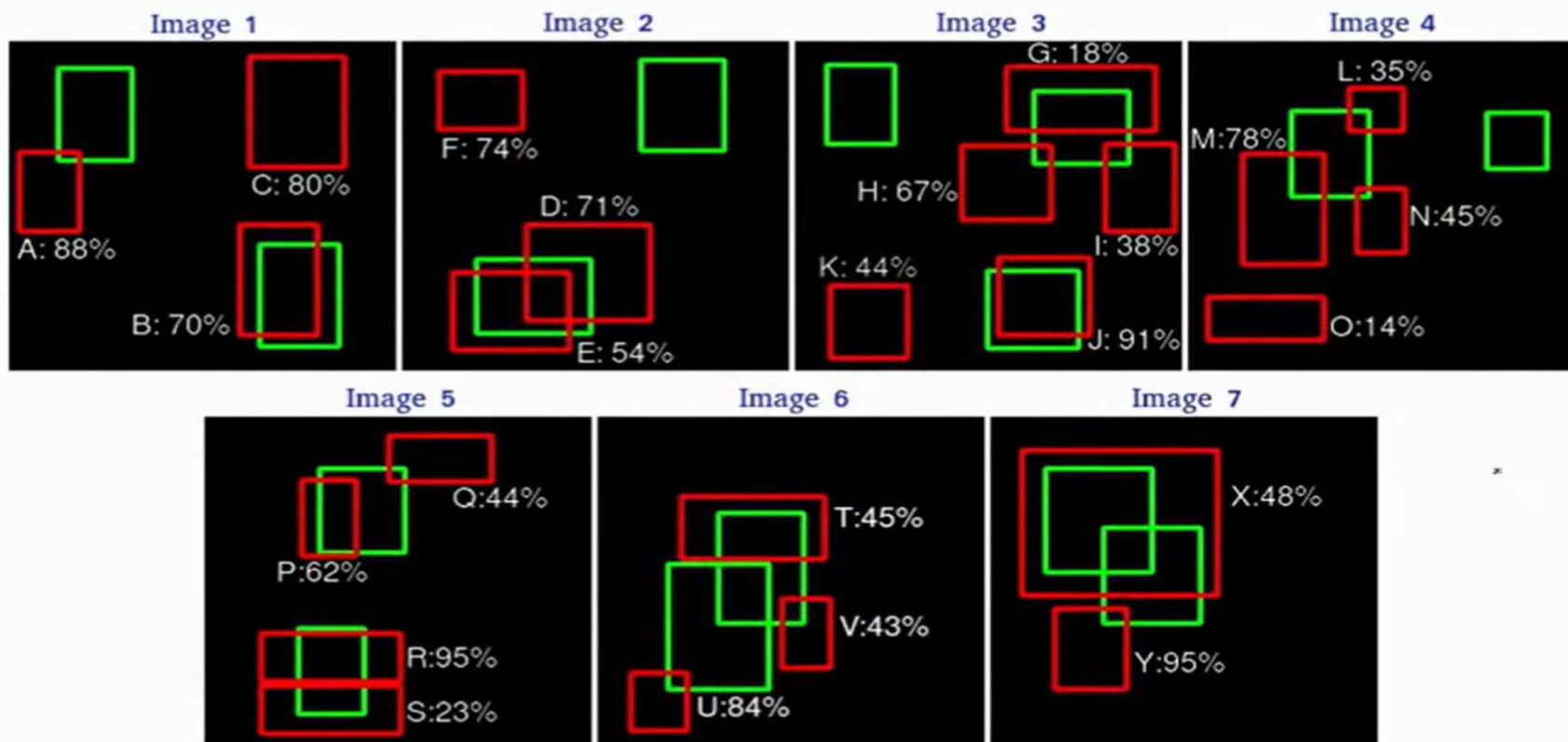
- Sort all predicted boxes (for all images)
 - According to scores
- For each k (location) in the list
 - Compute recall and precision

Images	Detections	Confidences	TP	FP	Acc TP	Acc FP	Precision	Recall
Image 5	R	95%	1	0	1	0	1	0.0666
Image 7	Y	95%	0	1	1	1	0.5	0.0666
Image 3	J	91%	1	0	2	1	0.6666	0.1333
Image 1	A	88%	0	1	2	2	0.5	0.1333
Image 6	U	84%	0	1	2	3	0.4	0.1333
Image 1	C	80%	0	1	2	4	0.3333	0.1333
Image 4	M	78%	0	1	2	5	0.2857	0.1333

Average precision (AP)



mAP: average AP over multiple classes



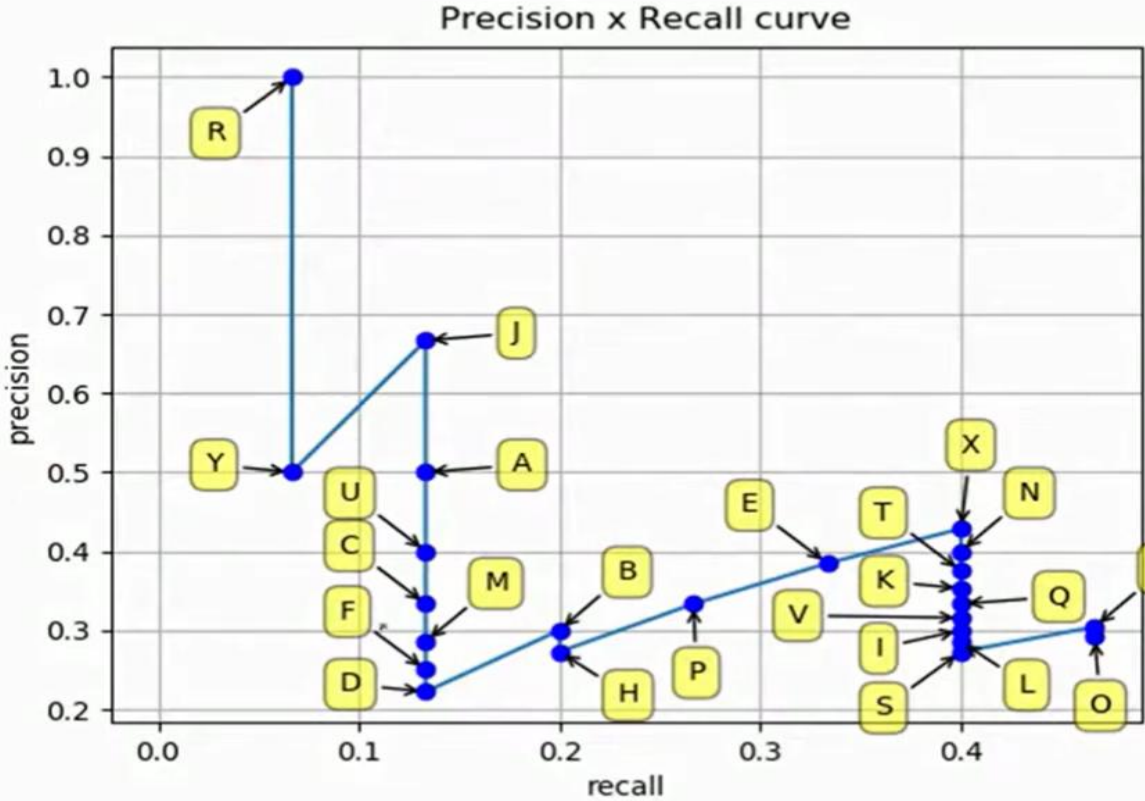
Hat Tip to Rafael: <https://github.com/rafaelpadilla/Object-Detection-Metrics>

Images	Detections	Confidences	TP or FP
Image 1	A	88%	FP
Image 1	B	70%	TP
Image 1	C	80%	FP
Image 2	D	71%	FP
Image 2	E	54%	TP
Image 2	F	74%	FP
Image 3	G	18%	TP
Image 3	H	67%	FP
Image 3	I	38%	FP
Image 3	J	91%	TP
Image 3	K	44%	FP
Image 4	L	35%	FP
Image 4	M	78%	FP
Image 4	N	45%	FP
Image 4	O	14%	FP
Image 5	P	62%	TP
Image 5	Q	44%	FP
Image 5	R	95%	TP
Image 5	S	23%	FP
Image 6	T	45%	FP
Image 6	U	84%	FP
Image 6	V	43%	FP
Image 7	X	48%	TP
Image 7	Y	95%	FP

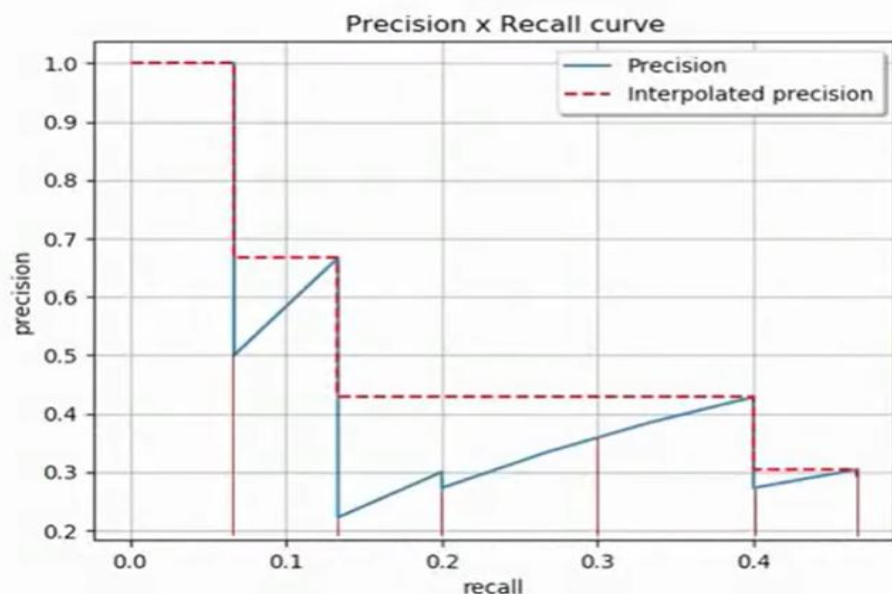
Images	Detections	Confidences	TP or FP
Image 1	A	88%	FP
Image 1	B	70%	TP
Image 1	C	80%	FP
Image 2	D	71%	FP
Image 2	E	54%	TP
Image 2	F	74%	FP
Image 3	G	18%	TP
Image 3	H	67%	FP
Image 3	I	38%	FP
Image 3	J	91%	TP
Image 3	K	44%	FP
Image 4	L	35%	FP
Image 4	M	78%	FP
Image 4	N	45%	FP
Image 4	O	14%	FP
Image 5	P	62%	TP
Image 5	Q	44%	FP
Image 5	R	95%	TP
Image 5	S	23%	FP
Image 6	T	45%	FP
Image 6	U	84%	FP
Image 6	V	43%	FP
Image 7	X	48%	TP
Image 7	Y	95%	FP

Images	Detections	Confidences	TP	FP	Acc TP	Acc FP	Precision	Recall
Image 5	R	95%	1	0	1	0	1	0.0666
Image 7	Y	95%	0	1	1	1	0.5	0.0666
Image 3	J	91%	1	0	2	1	0.6666	0.1333
Image 1	A	88%	0	1	2	2	0.5	0.1333
Image 6	U	84%	0	1	2	3	0.4	0.1333
Image 1	C	80%	0	1	2	4	0.3333	0.1333
Image 4	M	78%	0	1	2	5	0.2857	0.1333
Image 2	F	74%	0	1	2	6	0.25	0.1333
Image 2	D	71%	0	1	2	7	0.2222	0.1333
Image 1	B	70%	1	0	3	7	0.3	0.2
Image 3	H	67%	0	1	3	8	0.2727	0.2
Image 5	P	62%	1	0	4	8	0.3333	0.2666
Image 2	E	54%	1	0	5	8	0.3846	0.3333
Image 7	X	48%	1	0	6	8	0.4285	0.4
Image 4	N	45%	0	1	6	9	0.4	0.4
Image 6	T	45%	0	1	6	10	0.375	0.4
Image 3	K	44%	0	1	6	11	0.3529	0.4
Image 5	Q	44%	0	1	6	12	0.3333	0.4
Image 6	V	43%	0	1	6	13	0.3157	0.4
Image 3	I	38%	0	1	6	14	0.3	0.4
Image 4	L	35%	0	1	6	15	0.2857	0.4
Image 5	S	23%	0	1	6	16	0.2727	0.4
Image 3	G	18%	1	0	7	16	0.3043	0.4666
Image 4	O	14%	0	1	7	17	0.2916	0.4666

Images	Detections	Confidences	TP	FP	Acc TP	Acc FP	Precision	Recall
Image 5	R	95%	1	0	1	0	1	0.0666
Image 7	Y	95%	0	1	1	1	0.5	0.0666
Image 3	J	91%	1	0	2	1	0.6666	0.1333
Image 1	A	88%	0	1	2	2	0.5	0.1333
Image 6	U	84%	0	1	2	3	0.4	0.1333
Image 1	C	80%	0	1	2	4	0.3333	0.1333
Image 4	M	78%	0	1	2	5	0.2857	0.1333
Image 2	F	74%	0	1	2	6	0.25	0.1333
Image 2	D	71%	0	1	2	7	0.2222	0.1333
Image 1	B	70%	1	0	3	7	0.3	0.2
Image 3	H	67%	0	1	3	8	0.2727	0.2
Image 5	P	62%	1	0	4	8	0.3333	0.2666
Image 2	E	54%	1	0	5	8	0.3846	0.3333
Image 7	X	48%	1	0	6	8	0.4285	0.4
Image 4	N	45%	0	1	6	9	0.4	0.4
Image 6	T	45%	0	1	6	10	0.375	0.4
Image 3	K	44%	0	1	6	11	0.3529	0.4
Image 5	Q	44%	0	1	6	12	0.3333	0.4
Image 6	V	43%	0	1	6	13	0.3157	0.4
Image 3	I	38%	0	1	6	14	0.3	0.4
Image 4	L	35%	0	1	6	15	0.2857	0.4
Image 5	S	23%	0	1	6	16	0.2727	0.4
Image 3	G	18%	1	0	7	16	0.3043	0.4666
Image 4	O	14%	0	1	7	17	0.2916	0.4666



Average Precision



11 Point Interpolation

$$AP = \frac{1}{11} (1 + 0.6666 + 0.4285 + 0.4285 + 0.4285 + 0 + 0 + 0 + 0 + 0 + 0)$$

$$AP = 26.84\%$$

All Point Interpolation

Calculating the total area, we have the AP:

$$AP = A1 + A2 + A3 + A4$$

with:

$$A1 = (0.0666 - 0) \times 1 = \mathbf{0.0666}$$

$$A2 = (0.1333 - 0.0666) \times 0.6666 = \mathbf{0.04446222}$$

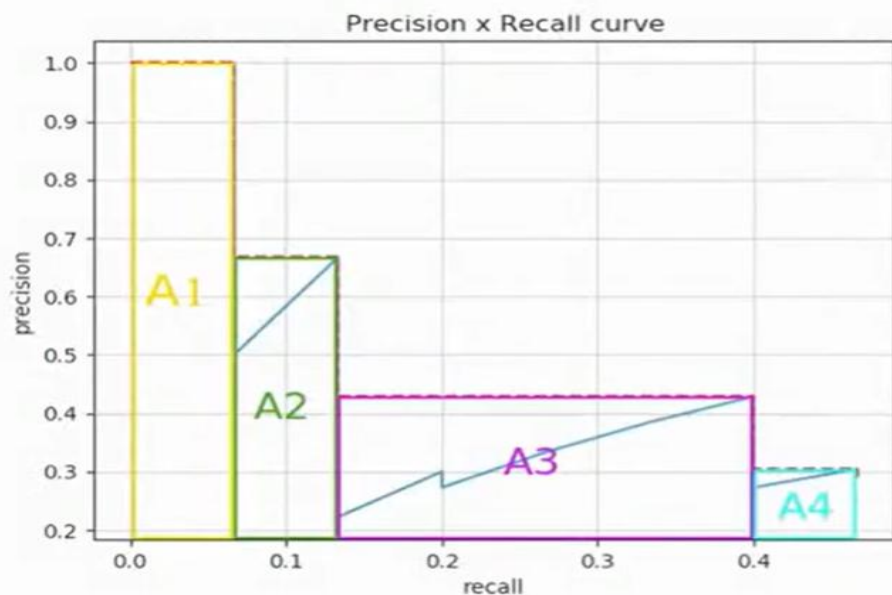
$$A3 = (0.4 - 0.1333) \times 0.4285 = \mathbf{0.11428095}$$

$$A4 = (0.4666 - 0.4) \times 0.3043 = \mathbf{0.02026638}$$

$$AP = 0.0666 + 0.04446222 + 0.11428095 + 0.02026638$$

$$AP = 0.24560955$$

$$AP = \mathbf{24.56\%}$$

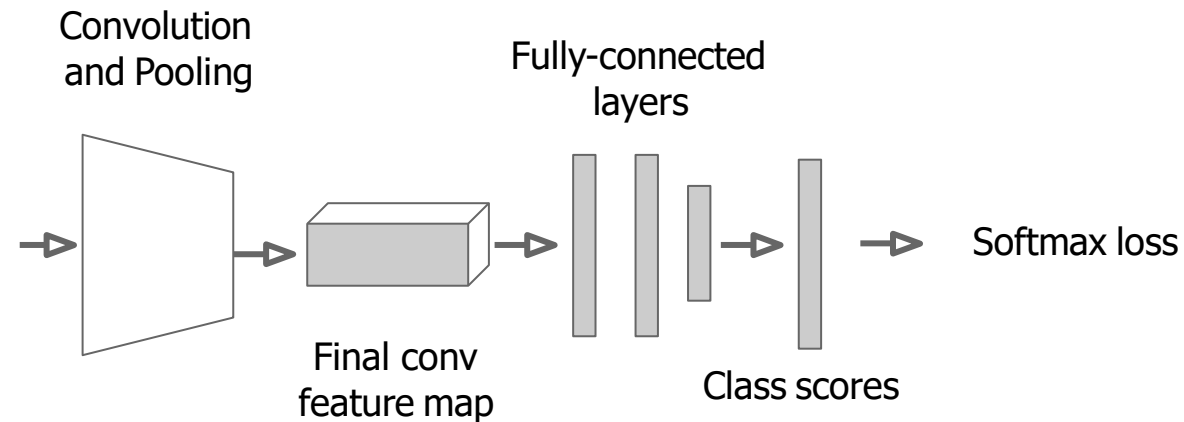


Simple Recipe for Object Detection

Step 1: Train (or download) a classification model (AlexNet, VGG, GoogLeNet)



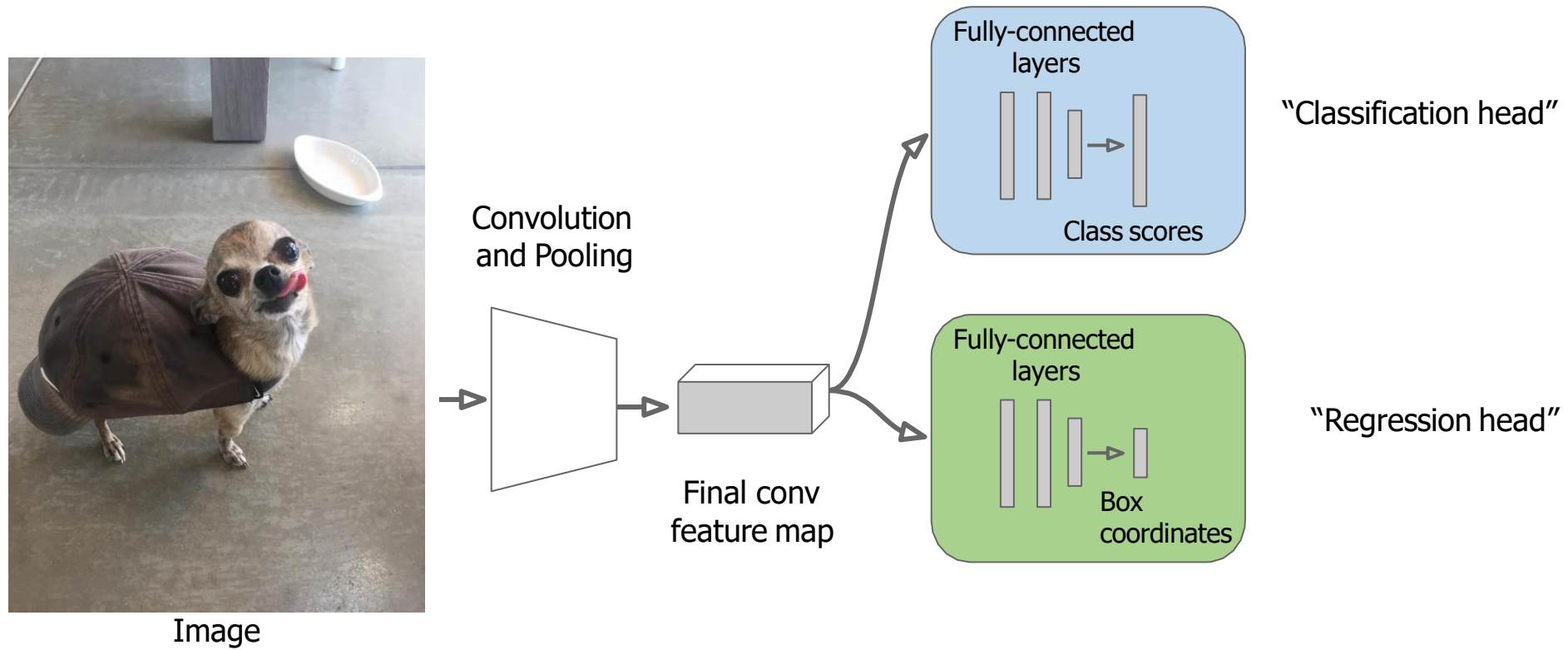
Image



Problem: Is there a **turtle** in this picture? If yes, localize!

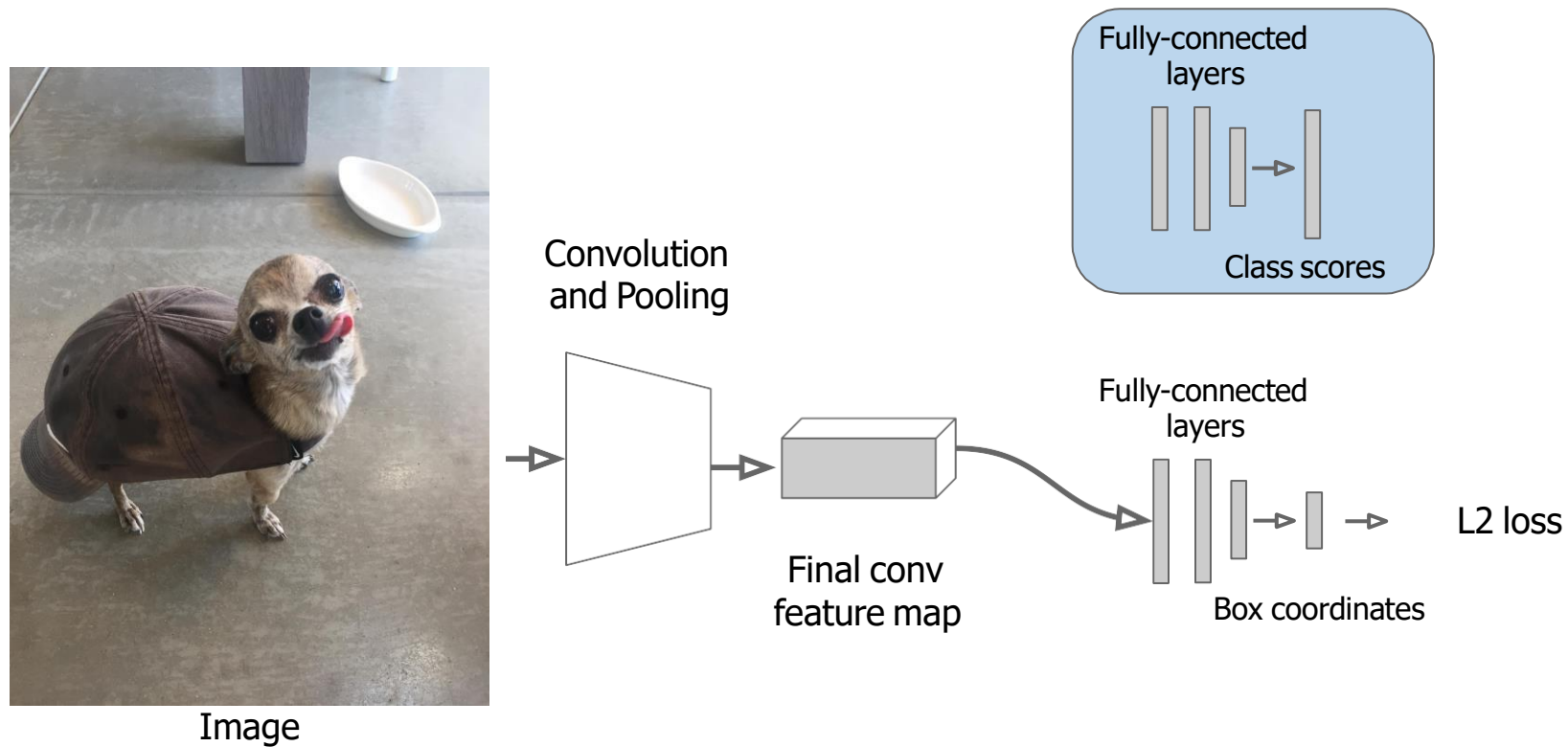
Simple Recipe for Object Detection

Step 2: Attach new fully-connected “regression head” to the network.



Simple Recipe for Object Detection

Step 3: Train the regression head only using L2 loss

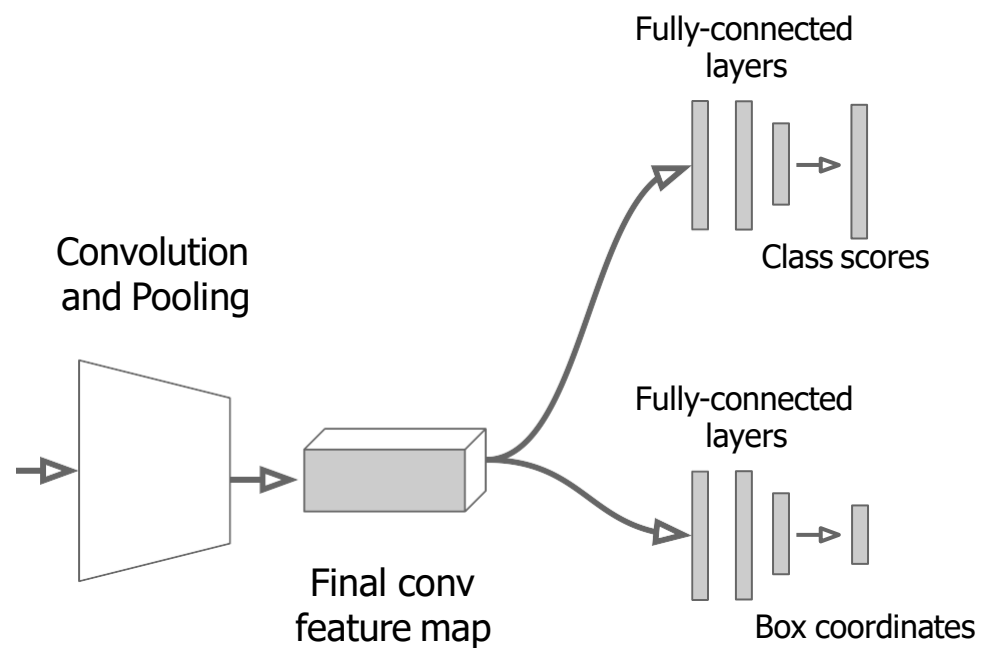


Simple Recipe for Object Detection

Step 4: At test time use both heads.

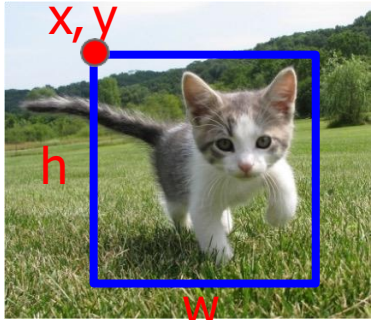


Image

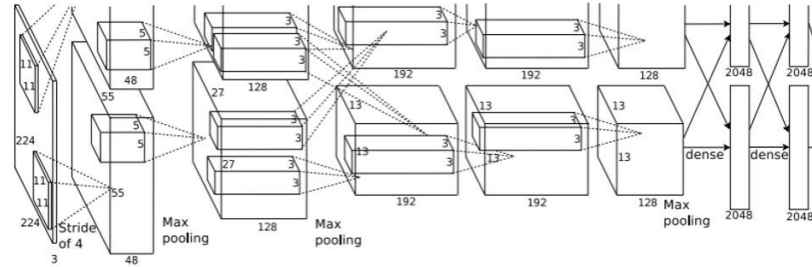


Simple Recipe for Object Detection

Object Detection: Single Object (Classification + Localization)



This image is CC0 public domain



Fully
Connected:
4096 to 1000

Class Scores

Cat: 0.9

Dog: 0.05

Car: 0.01

...

Vector:
4096

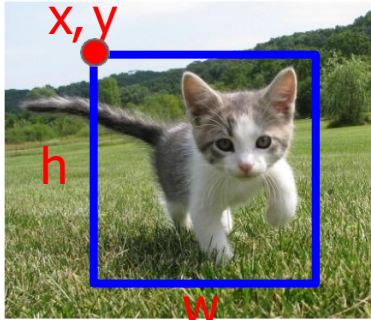
Fully
Connected:
4096 to 4

Box
Coordinates
(x, y, w, h)

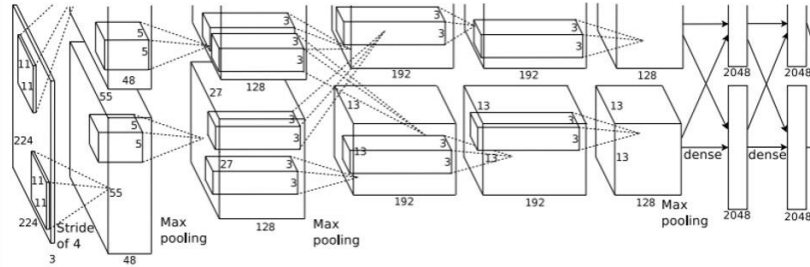
Treat localization as a
regression problem!

Simple Recipe for Object Detection

Object Detection: Single Object (Classification + Localization)



This image is CC0 public domain



Vector:
4096

Fully
Connected:
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:
Cat

Softmax
Loss

Multitask Loss

+ → Loss

Fully
Connected:
4096 to 4

Box
Coordinates
(x, y, w, h)

L2 Loss

Correct box:
(x', y', w', h')

Treat localization as a
regression problem!

Detection as Regression?



DOG, (x, y, w, h)
CAT, (x, y, w, h)
CAT, (x, y, w, h)
DUCK (x, y, w, h)

= 16 numbers

Detection as Regression?



DOG, (x, y, w, h)

CAT, (x, y, w, h)

= 8 numbers

Detection as Regression?



CAT, (x, y, w, h)

CAT, (x, y, w, h)

....

CAT (x, y, w, h)

= many
numbers

Need variable sized outputs

Detection as Classification



CAT? NO

DOG? NO

Detection as Classification



CAT? YES!

DOG? NO

Detection as Classification



CAT? NO

DOG? Yes

Detection as Classification

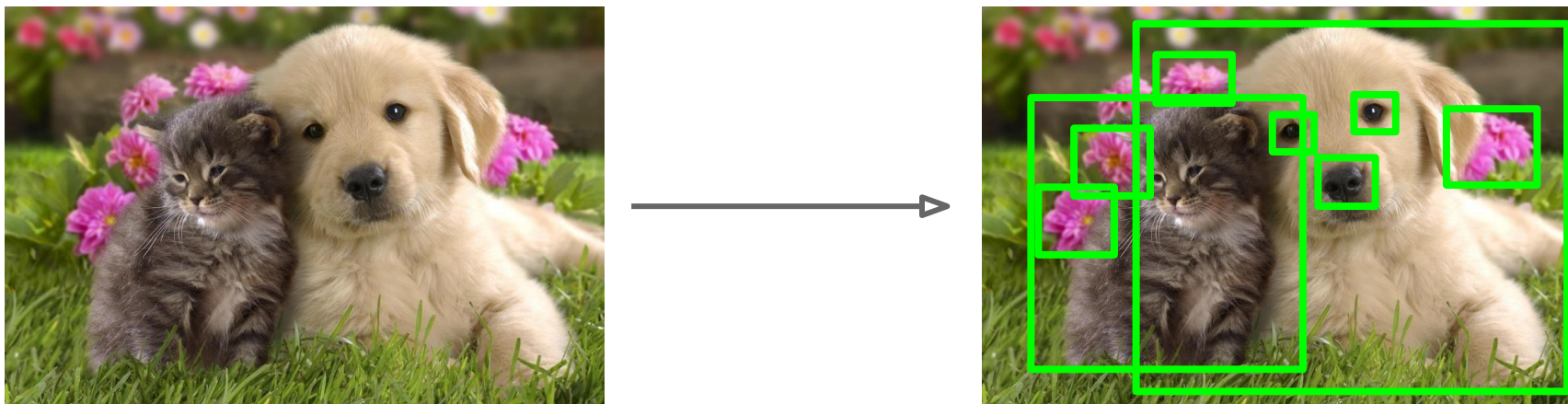
Problem:

- Need to test many positions and scales
- Use a computationally demanding classifier (CNN)
- Search at different scales
- Search at different positions

Solution: Only look at a tiny subset of possible positions

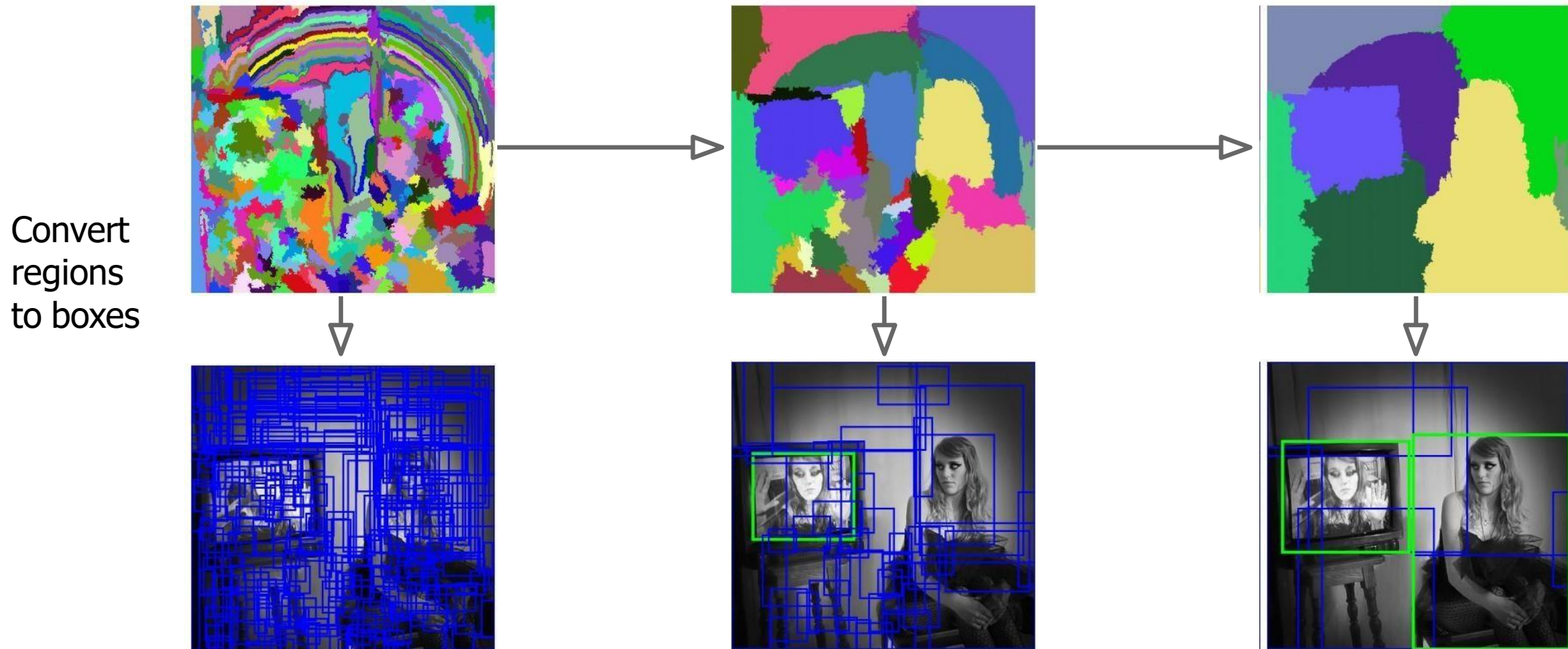
Region Proposals

- Find “blobby” image regions that are likely to contain objects
- “Class-agnostic” object detector
- Look for “blob-like” regions

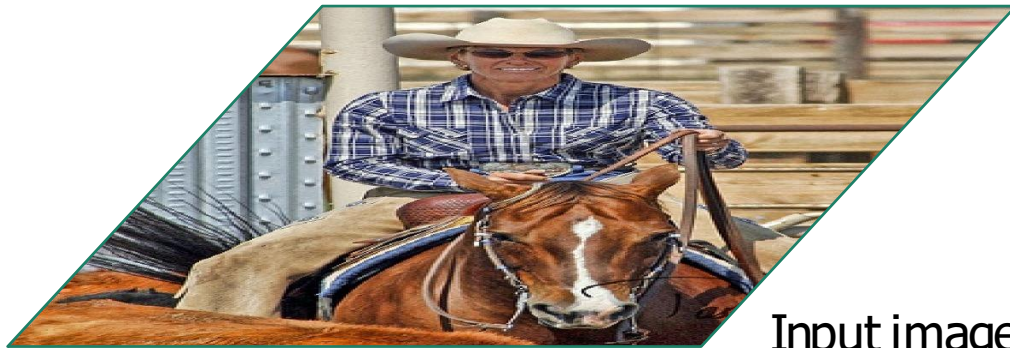


Region Proposals: Selective Search

Bottom-up segmentation, merging regions at multiple scales



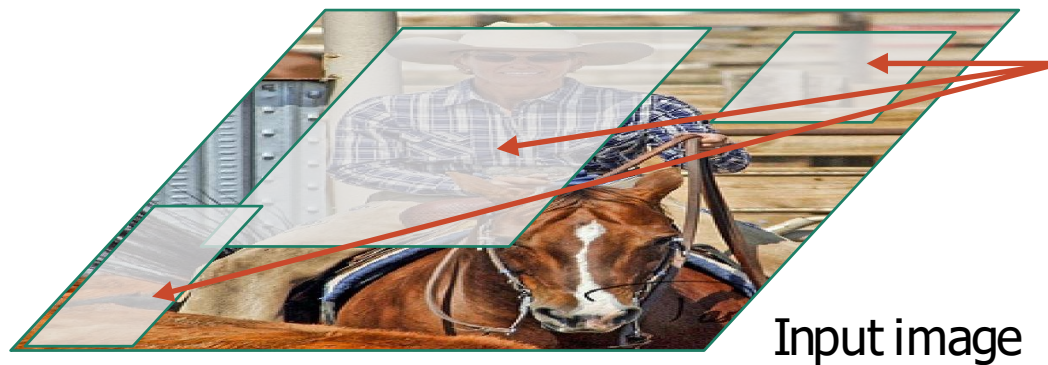
R-CNN



Input image

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

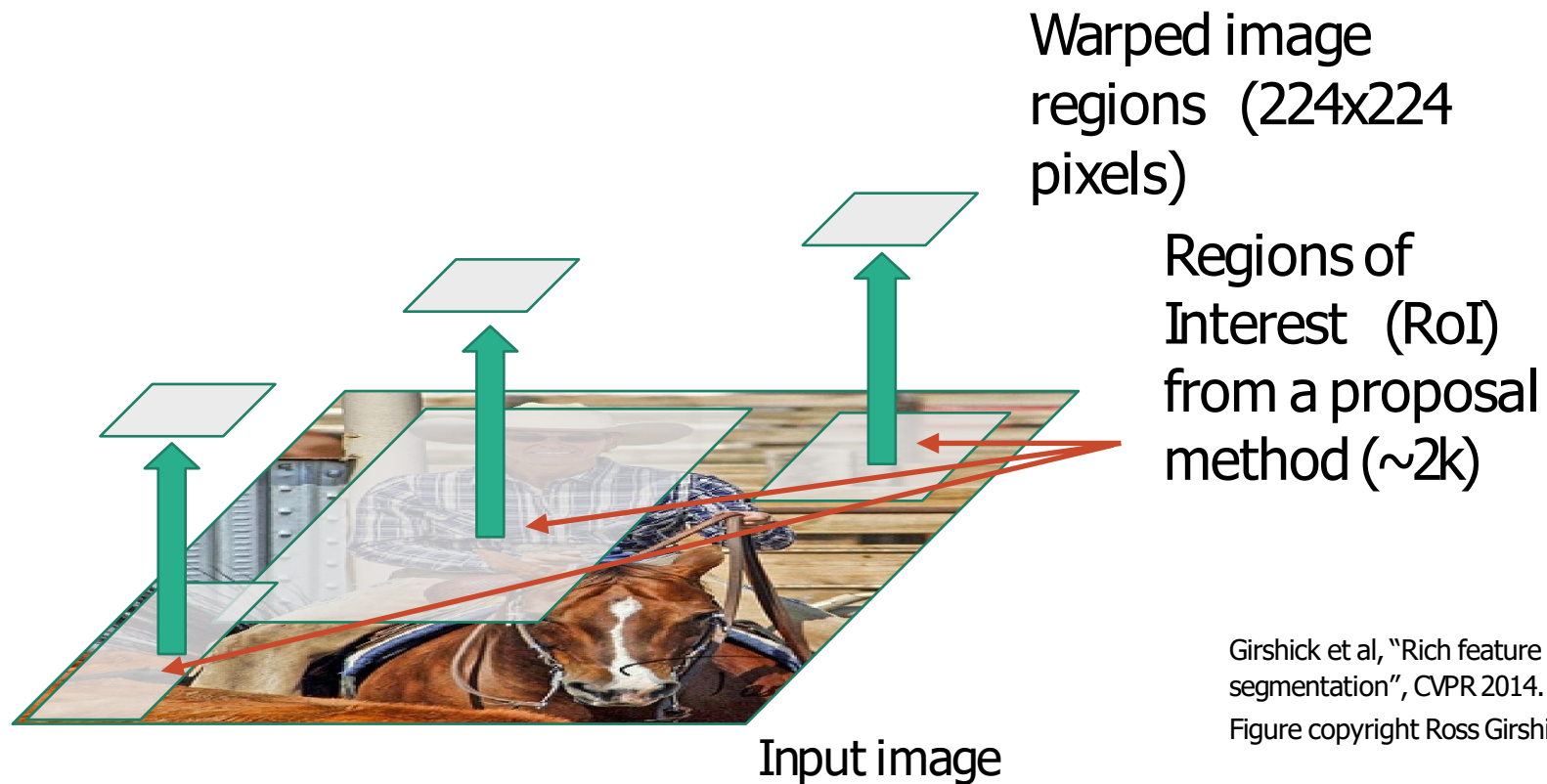
R-CNN



Regions of Interest
(RoI) from a proposal
method ($\sim 2k$)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

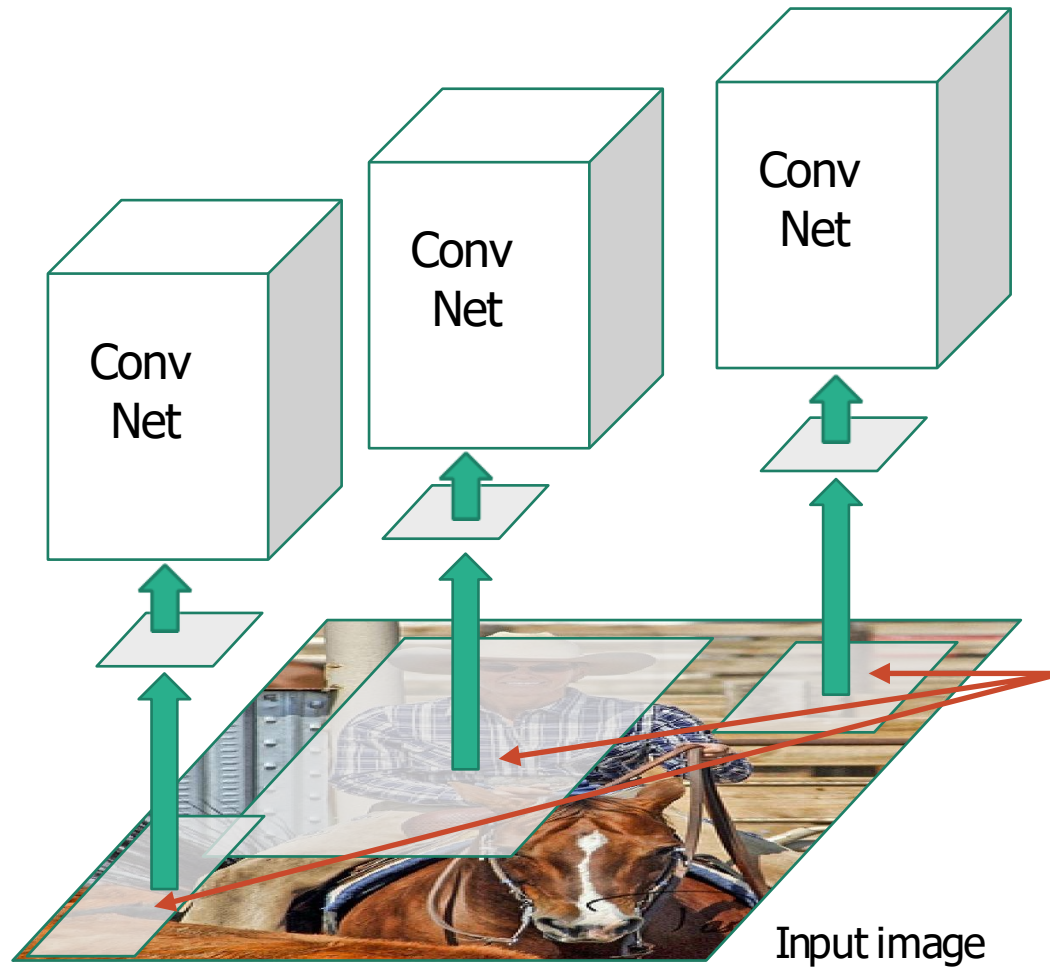
R-CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN



Forward each region through ConvNet (ImageNet-pretrained)

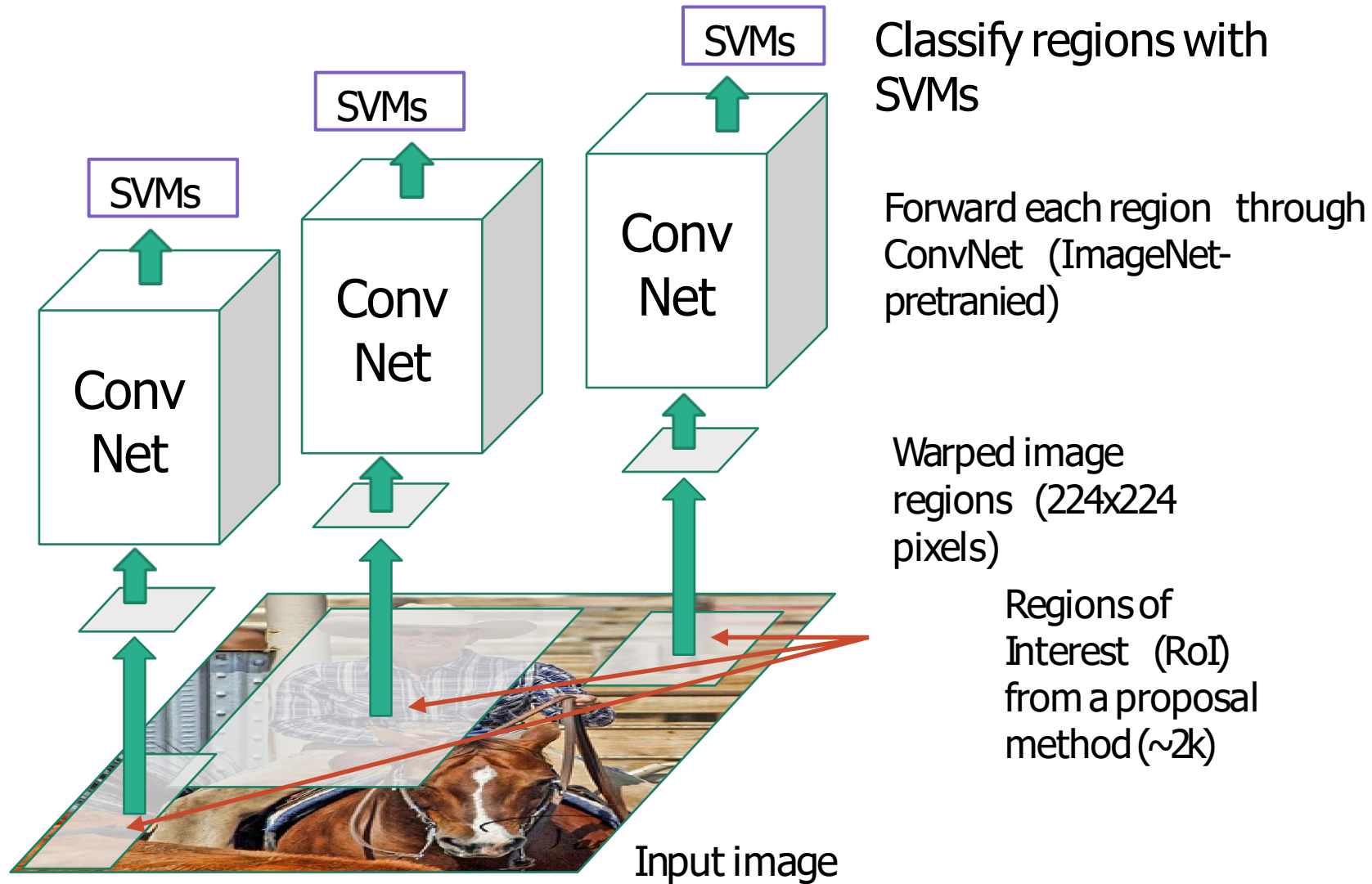
Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

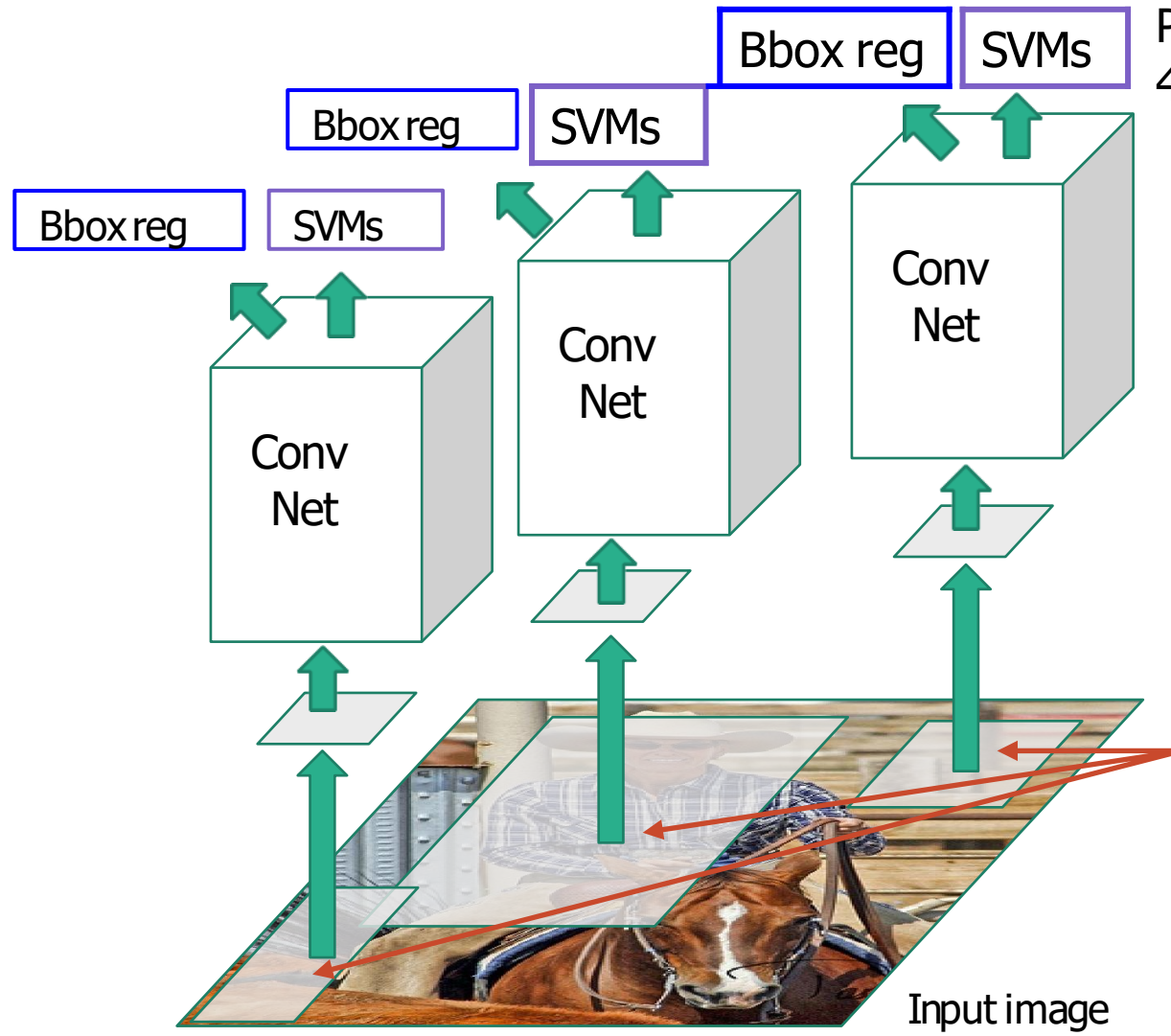
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN



R-CNN



Predict "corrections" to the RoI:
4 numbers: (dx, dy, dw, dh)

Classify regions with
SVMs

Forward each region through
ConvNet (ImageNet-
pretrained)

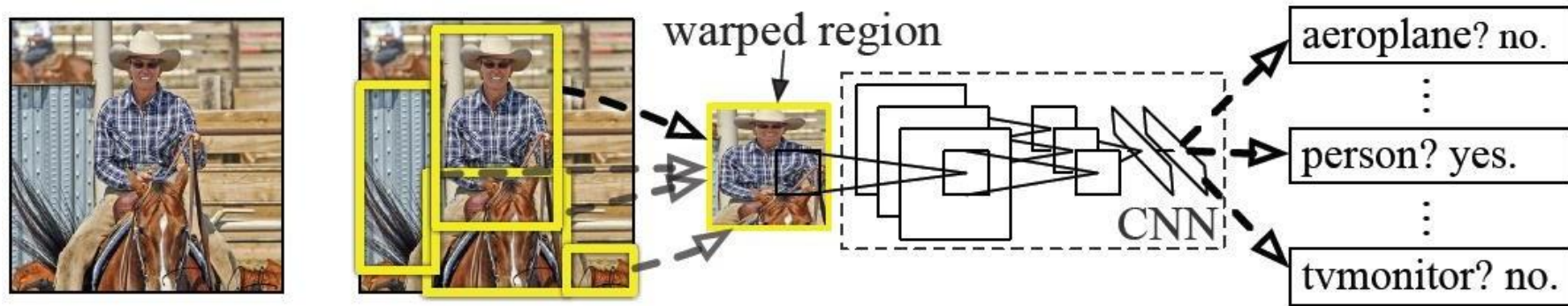
Warped image
regions (224x224
pixels)

Regions of
Interest (RoI)
from a proposal
method (~2k)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

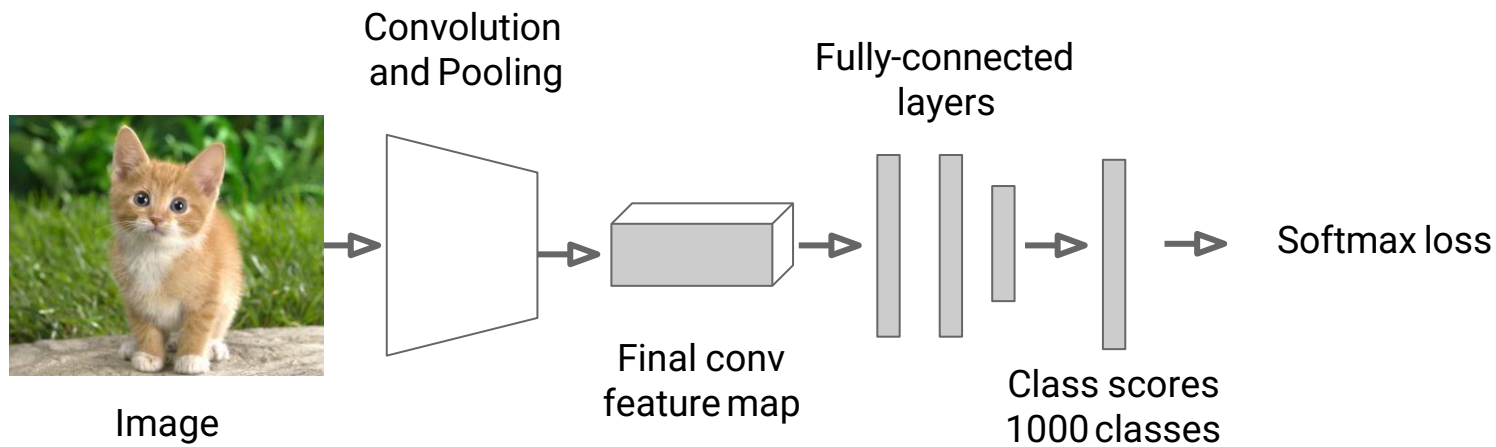
R-CNN Details



- **Regions:** uses ~ 2000 Selective Search proposals
- **Network:** uses AlexNet *pre-trained* on ImageNet (1000 classes), *fine-tuned* on PASCAL (21 classes)
- **Final detector:**
 - first warp proposal regions,
 - then extract fc7 network activations (4096 dimensions),
 - Finally, classify with linear SVM
- **Bounding box regression** is also used to refine box locations

R-CNN Training (initialization)

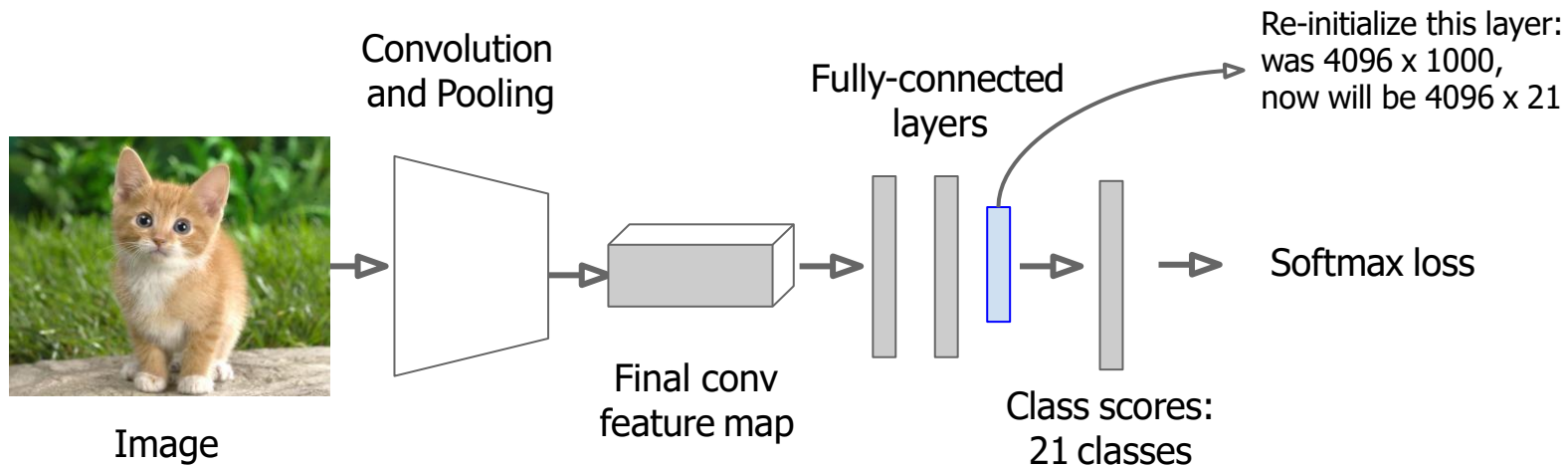
Step 1: Train (or download) a classification model for ImageNet (AlexNet)



R-CNN Training (Fine-tuning)

Step 2: "Fine-tune" model for detection

- Instead of 1000 ImageNet classes, want 20 object classes + background
- Throw away final fully-connected layer, reinitialize from scratch
- Keep training model using positive / negative regions from detection images



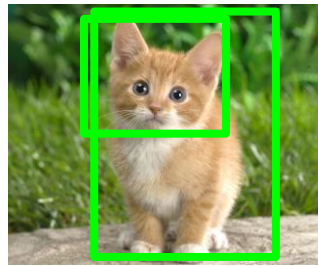
R-CNN Training (feature extraction)

Step 3: Extract features

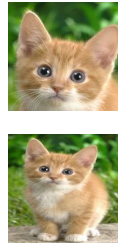
- Extract region proposals for all images
- For each region: warp to CNN input size, run forward through CNN, save pool5 features to disk



Image

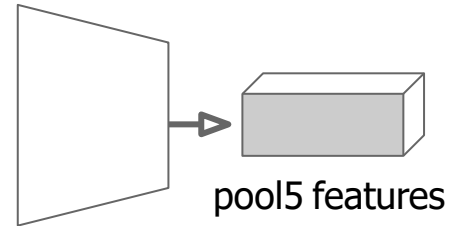


Region Proposals



Crop + Warp

Convolution
and Pooling



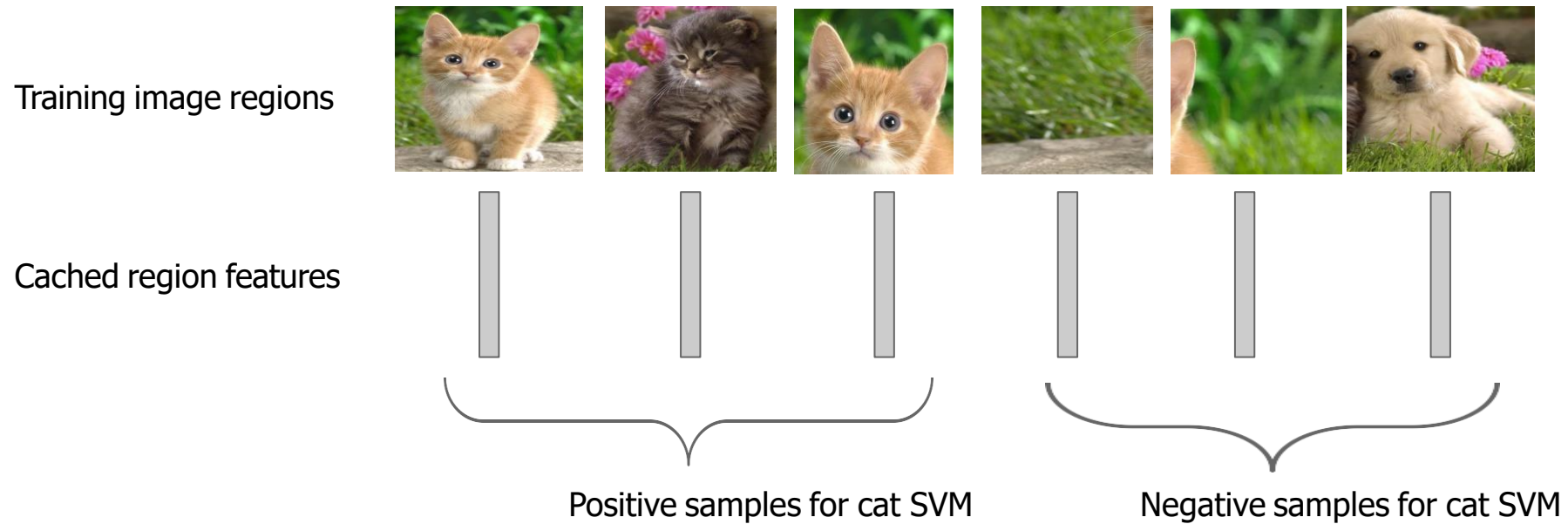
Forward pass



Save to disk

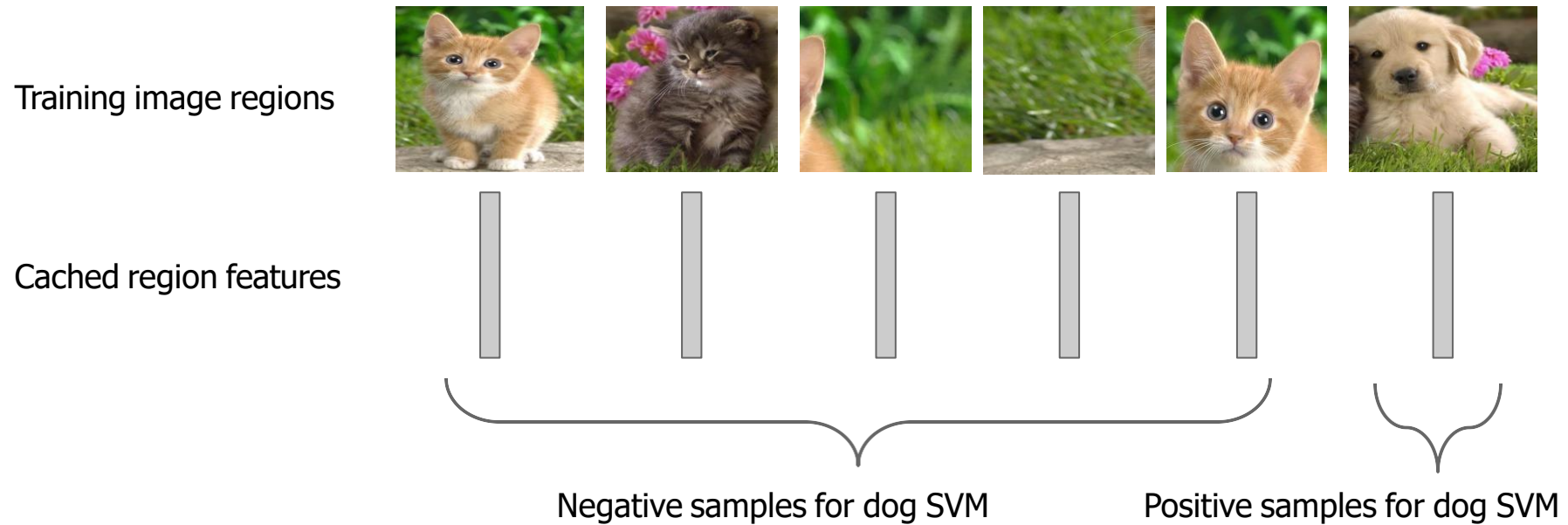
R-CNN Training (train classifier)

Step 4: Train one binary SVM per class to classify region features



R-CNN Training (train classifier)

Step 4: Train one binary SVM per class to classify region features



R-CNN Training (bounding box regression/prediction)

Step 5 (bbox regression): **For each class**, train a linear **regression** model to map from cached features to offsets to GT boxes to make up for “slightly wrong” proposals

Training image
regions:



Cached region features

Regression **targets:**
(dx, dy, dw, dh)
Normalized coordinates

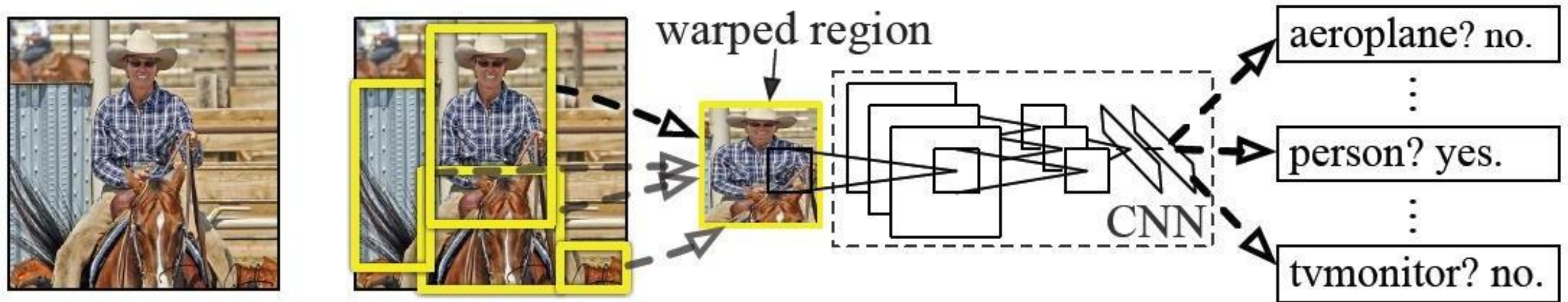
(0, 0, 0, 0)
Proposal is good

(0.25, 0, 0, 0)
Proposal too
far to left

(0, 0, -0.125, 0)
Proposal too
wide

Issue #1 with R-CNN

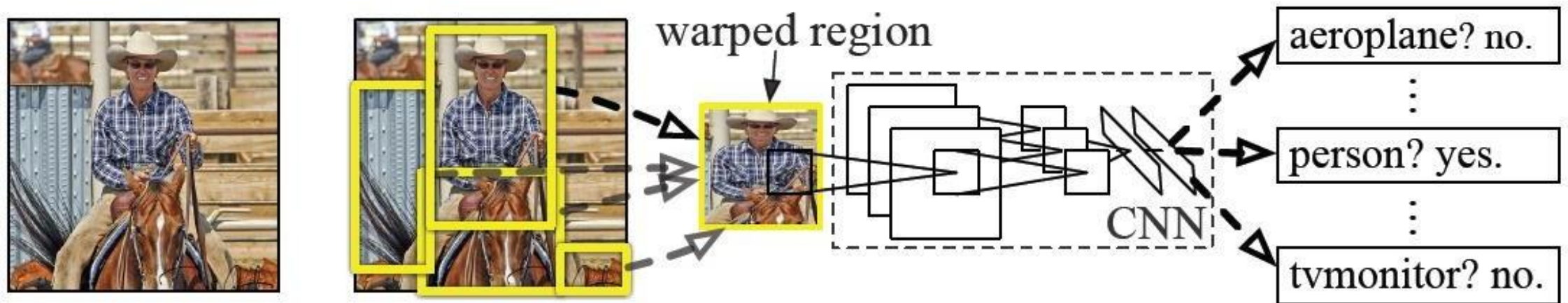
- Slow in run-time
 - Multiple forward passes for each proposal
 - There are thousands of proposals



- Solution
 - Single forward pass for each image?

Issue #2 with R-CNN

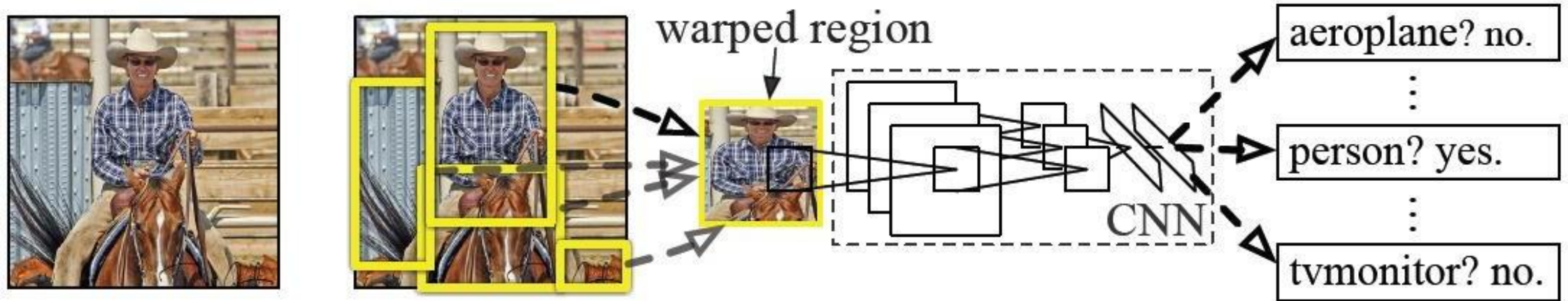
- Separate classifier training
 - CNN feature extractor is not trained with classifier and regressor



- Solution
 - End-to-end training?

Issue #3 with R-CNN

- Complex training pipeline
 - Proposals
 - Feature extraction
 - Classification

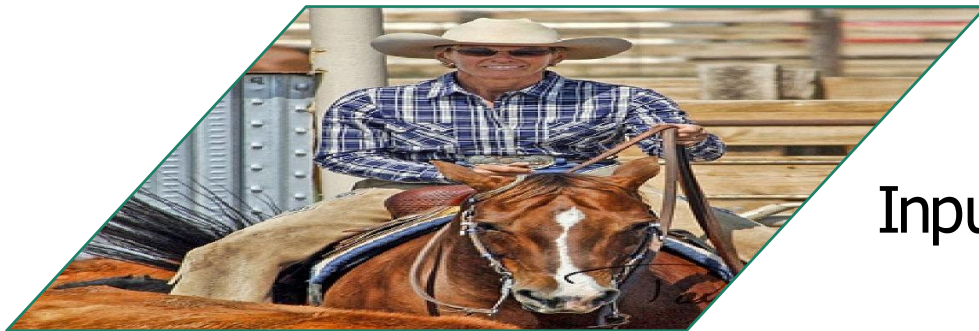


- Solution
 - Single forward pass for each image?

Solution

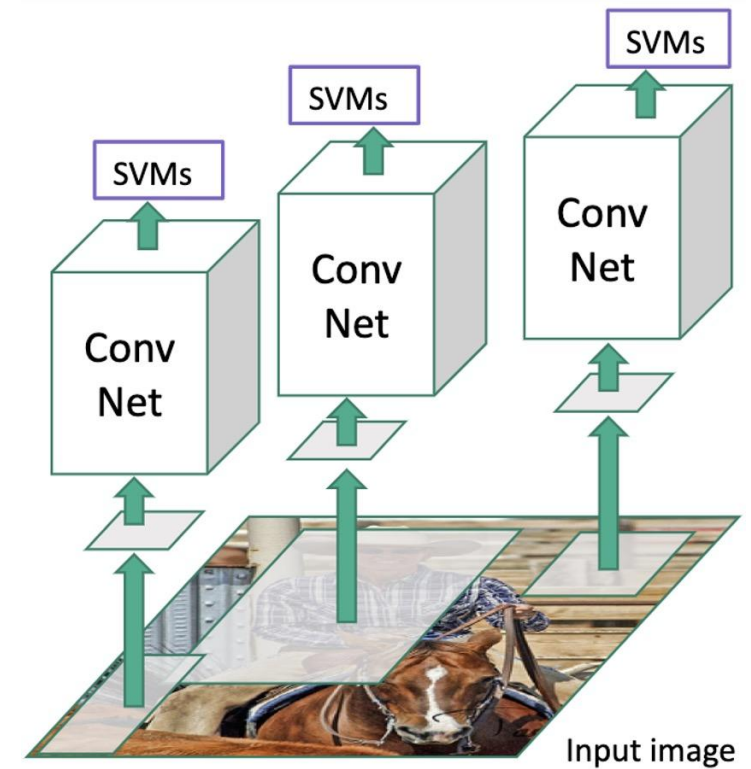
- Fast R-CNN
 - Single forward pass for each image
 - No separate classifier
 - End-to-end training

Fast R-CNN

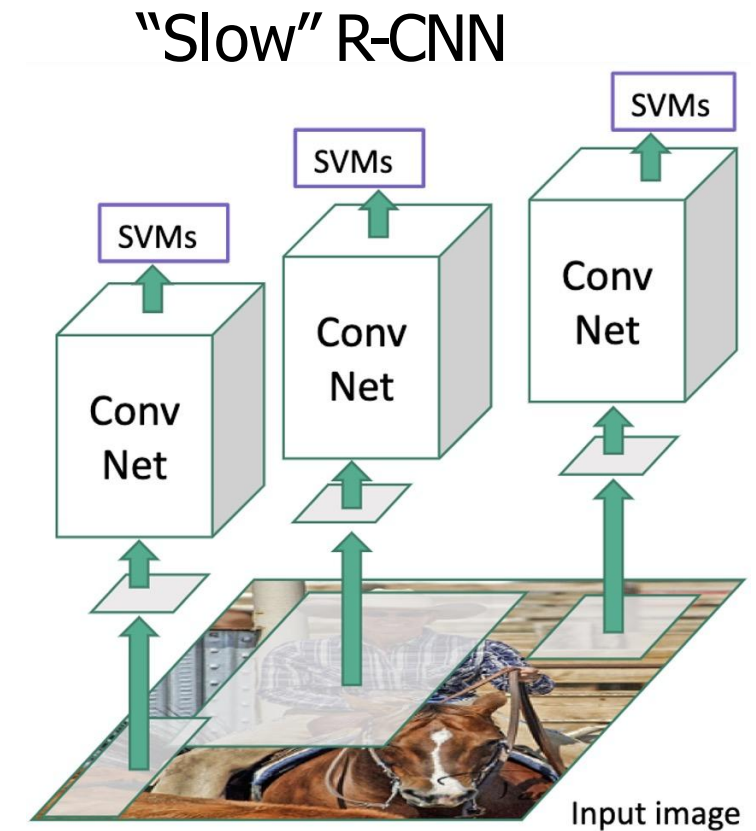
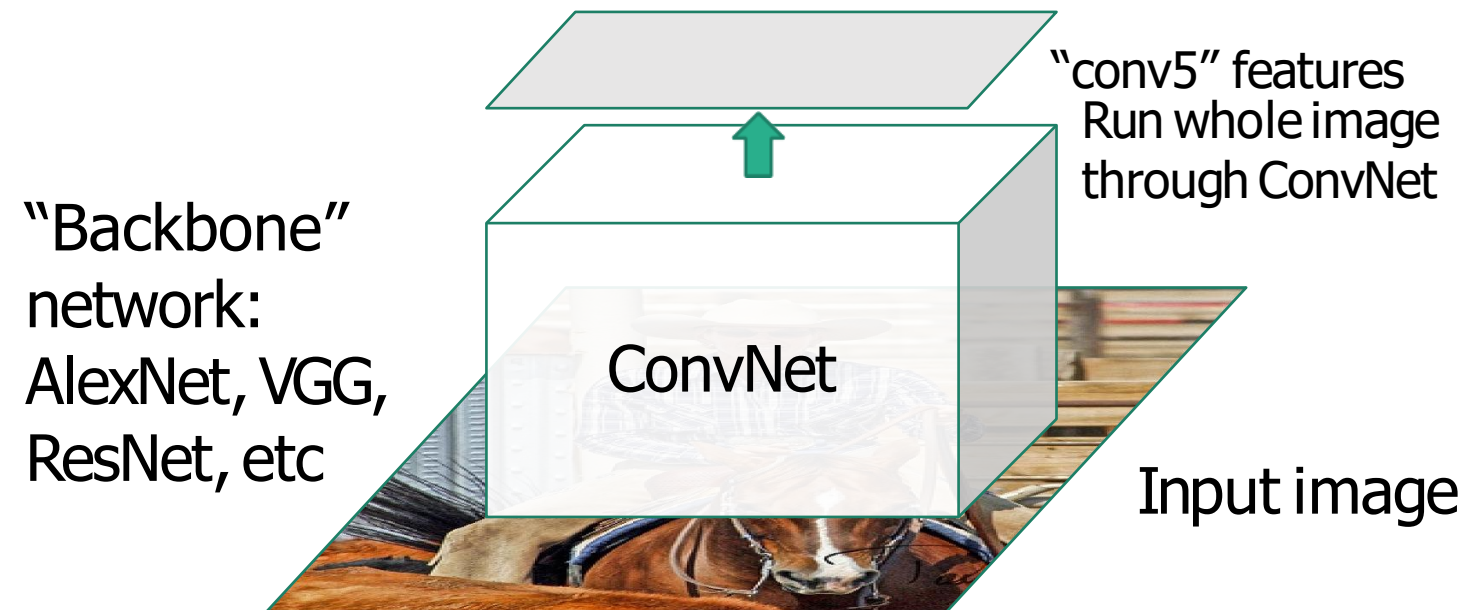


Input image

"Slow" R-CNN



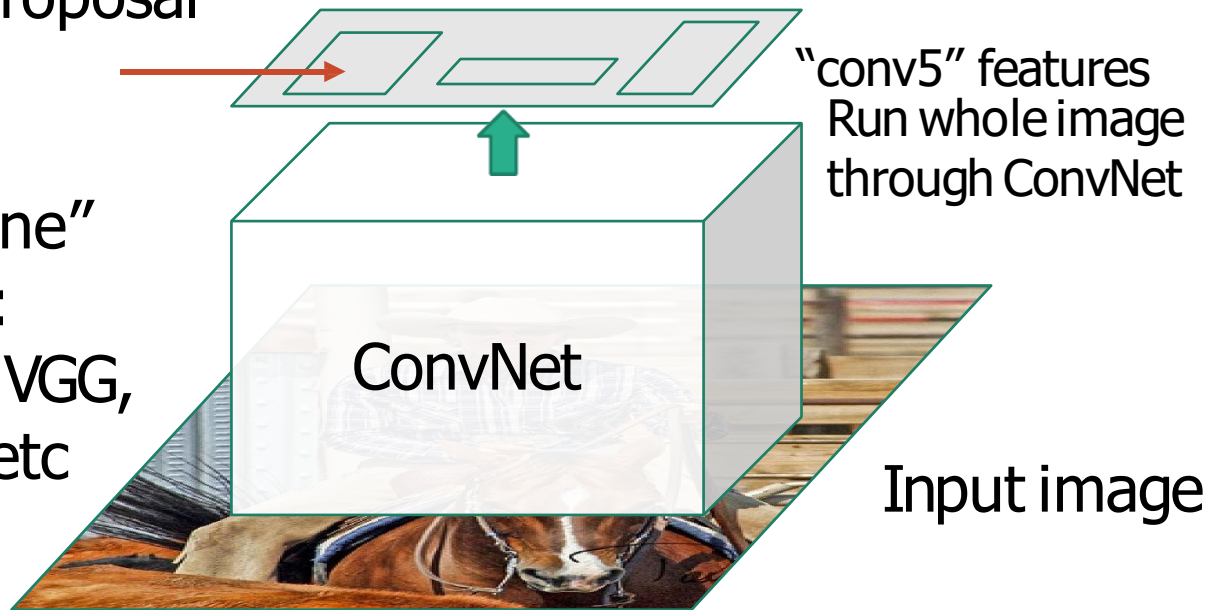
Fast R-CNN



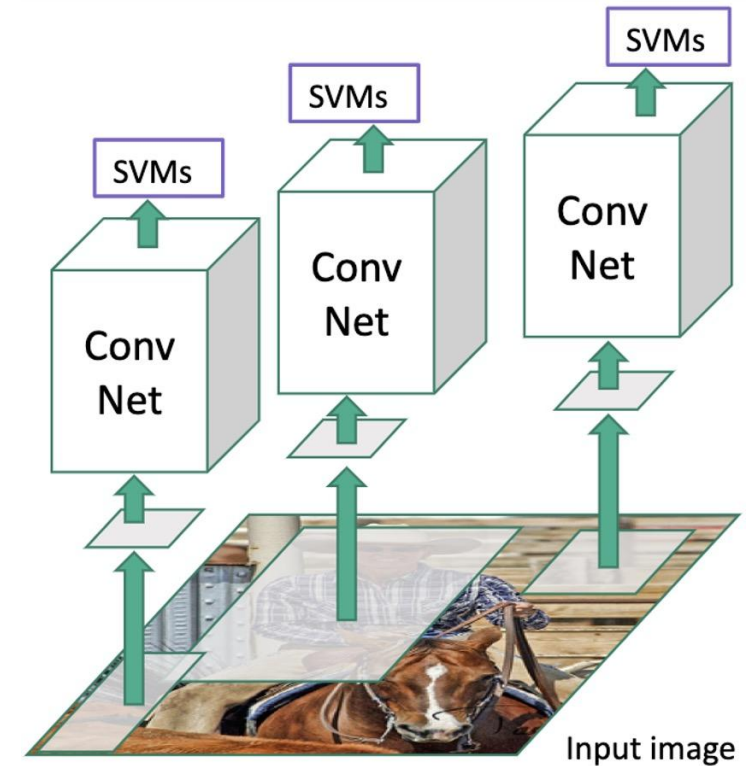
Fast R-CNN

Regions of Interest (RoIs) from a proposal method

“Backbone” network:
AlexNet, VGG, ResNet, etc



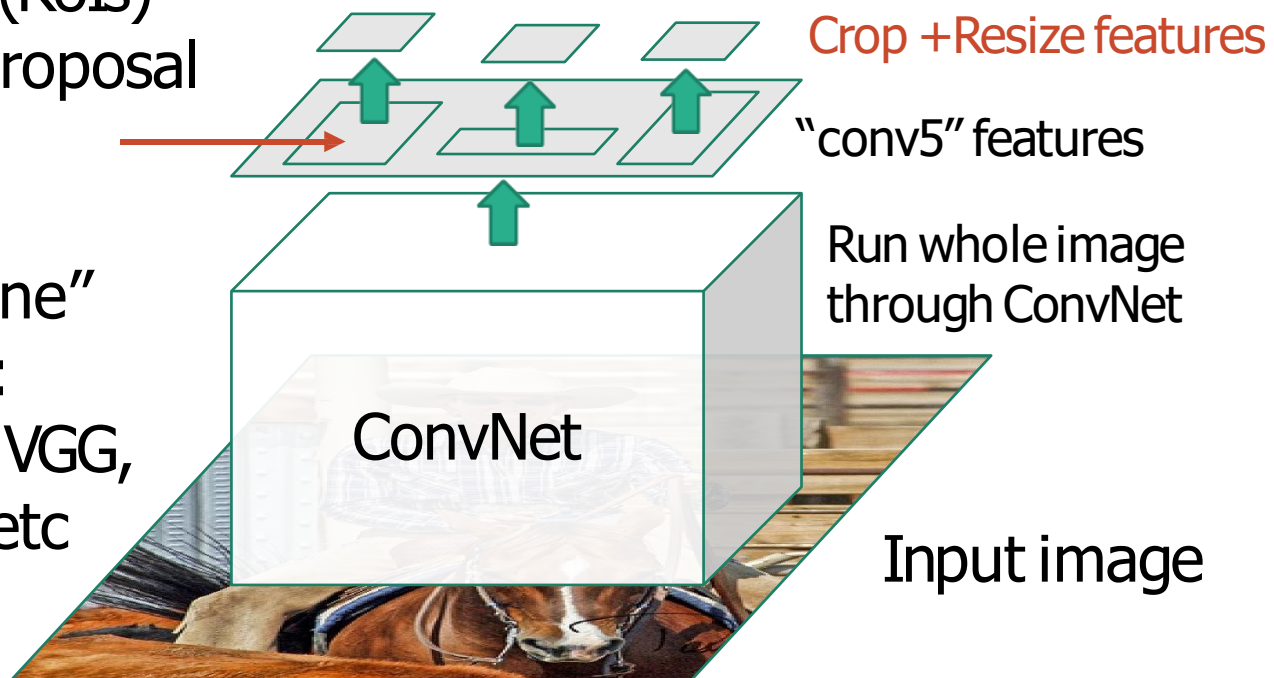
“Slow” R-CNN



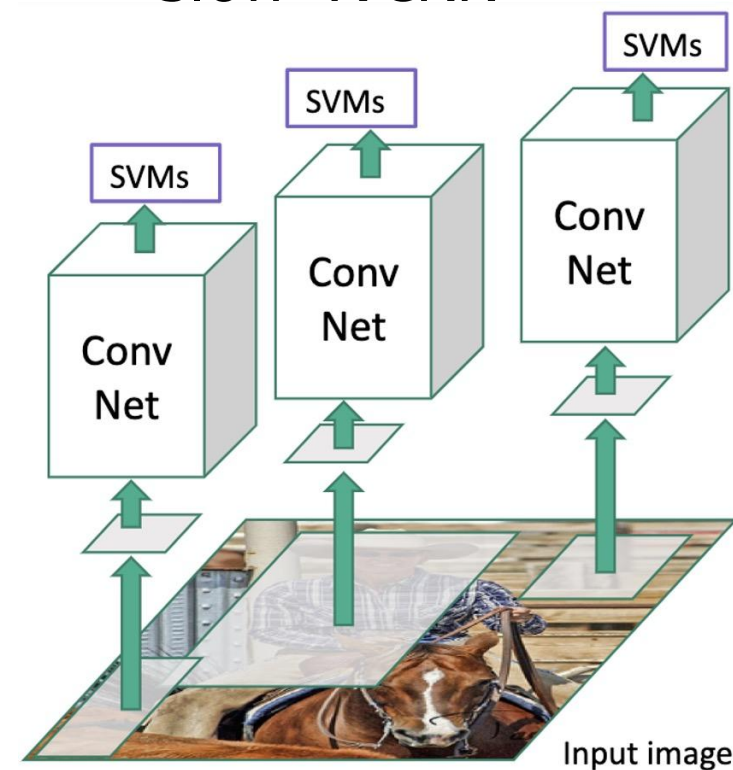
Fast R-CNN

Regions of Interest (RoIs) from a proposal method

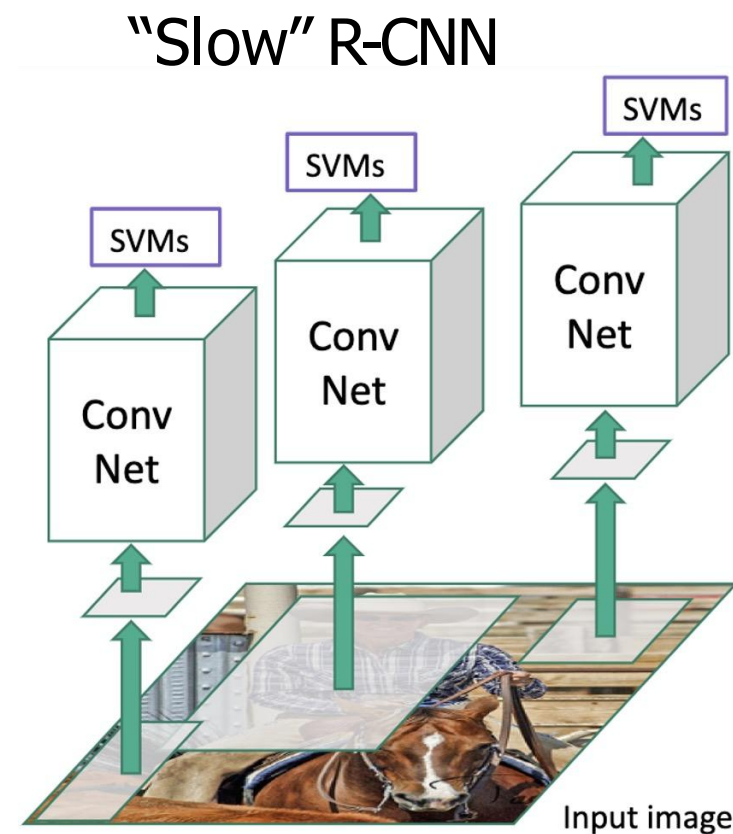
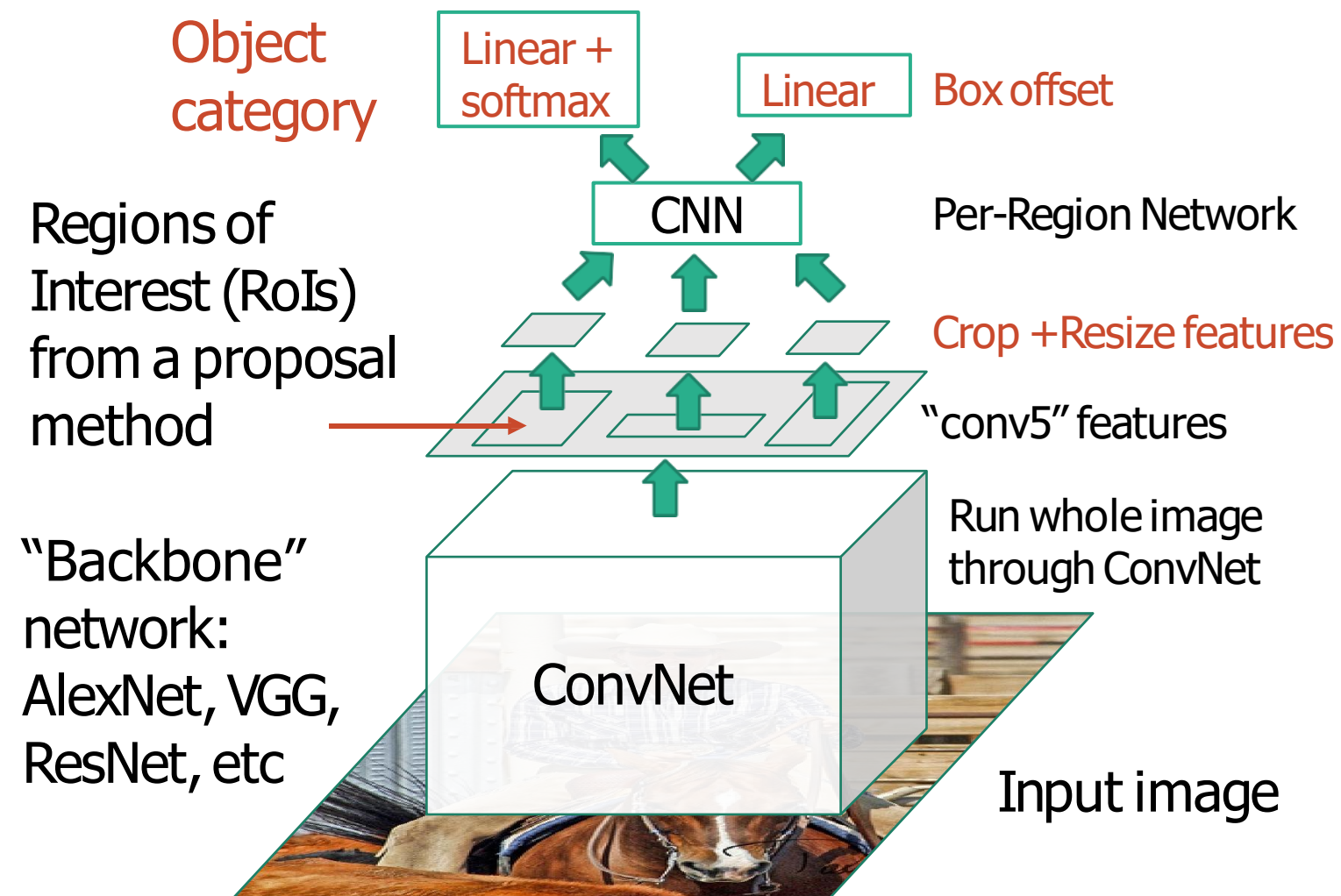
“Backbone” network:
AlexNet, VGG, ResNet, etc



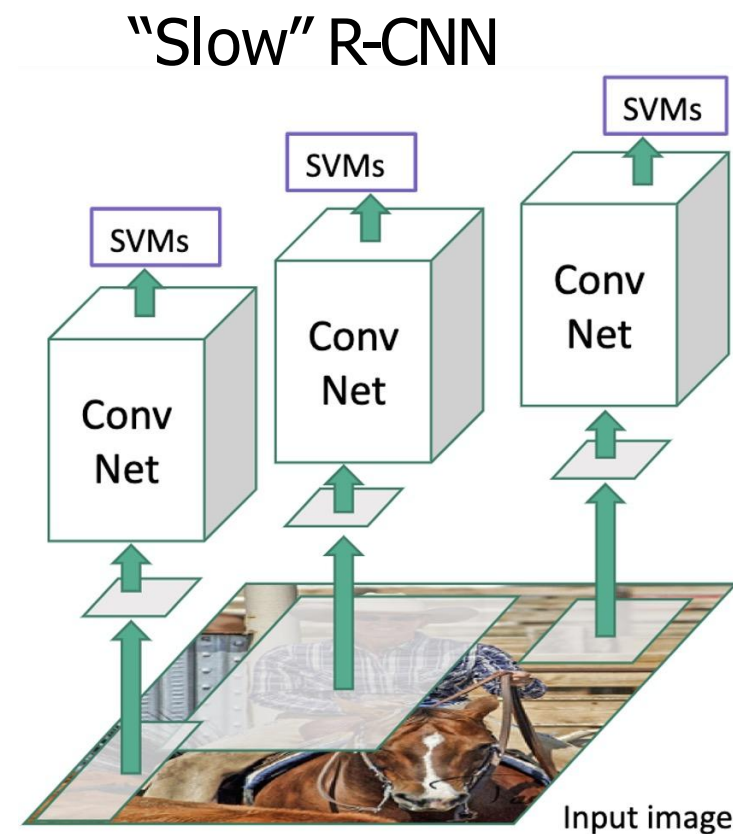
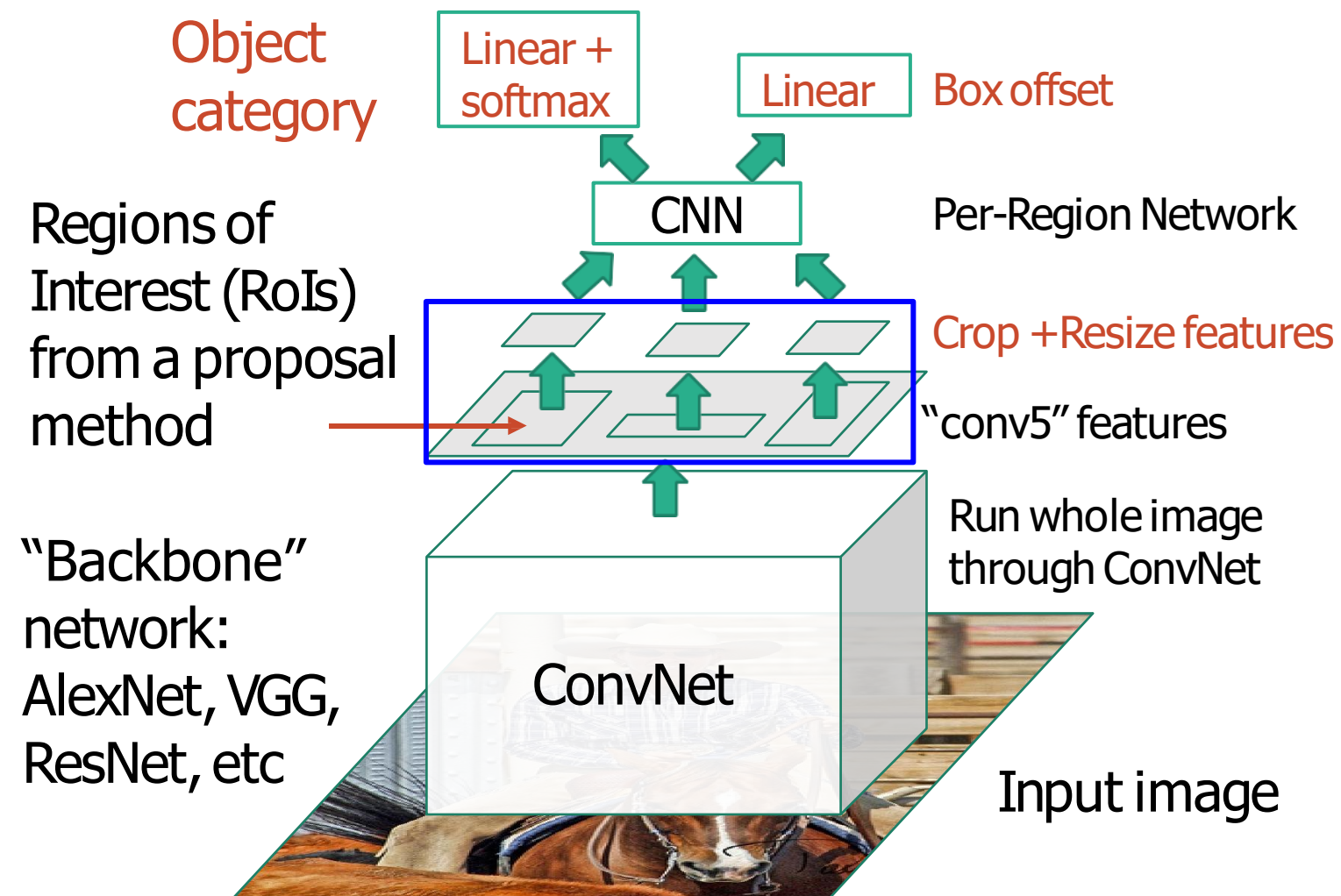
“Slow” R-CNN



Fast R-CNN



Fast R-CNN



Fast R-CNN: Another view

